



Escuela Técnica Superior
de Ingeniería Informática

Grado en Ingeniería Informática

Curso 2025-2026

Trabajo Fin de Grado

**DIAMOND INSIGHTS: MASTER THE DIAMOND
WITH DATA**

Autor: Alfonso Rodríguez Gutt

Tutor: Iván Chicano Capelo y Óscar Soto Sánchez



©2025 Alfonso Rodríguez Gutt

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional” de Creative Commons,
disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Agradecimientos

Después de 1 año de trabajo constante, de días enteros de programación, hoy por fin escribo estas líneas con la satisfacción de haber llegado a la meta. Escribir este Trabajo de Fin de Grado ha sido un proceso de aprendizaje intenso que ha ido mucho más allá de lo académico; ha sido una etapa de crecimiento personal que me ha transformado. Por ello, no puedo dar por cerrada esta etapa sin detenerme a agradecer a quienes, de una forma u otra, han sido parte de este camino y lo han hecho posible.

En el ámbito académico, mi gratitud es para mi tutor, Iván Chicano. Gracias por su disponibilidad y por esa guía tan valiosa que me ha servido de brújula en los momentos de incertidumbre. Sus correcciones, siempre constructivas, y apoyo técnico han sido las herramientas definitivas para poder completar este trabajo de manera satisfactoria. Gracias por creer en la viabilidad de este proyecto desde que era solo un borrador.

A mis padres y mi novia, gracias por ser mi pilar fundamental. Sus consejos sabios en los momentos de agobio y comprensión ante mis ausencias y preocupaciones han sido lo que me ha mantenido a flote. Saber que siempre estaban ahí, al otro lado del teléfono o en casa, dándome el apoyo que necesitaba sin pedir nada a cambio, ha sido el motor que me ha permitido seguir adelante.

Finalmente, a mis amigos. A veces se olvida que para terminar un trabajo de este calibre hace falta saber cuándo parar, y ustedes han sido mis válvulas de escape. Gracias por el apoyo mutuo en los días difíciles, pero sobre todo, gracias por esas charlas de horas sobre cualquier cosa que no tuviera nada que ver con la universidad.

A todos ustedes, gracias por formar parte de este logro.

Resumen

En esta memoria se detallará el proceso de desarrollo seguido para llevar a cabo la aplicación ***Diamond Insights***. El propósito de esta memoria es documentar cada aspecto del desarrollo de la aplicación, desde aquellos aspectos de más alto nivel como la planificación y objetivos, a aquellos más técnicos como la arquitectura y detalles de implementación.

El documento inicia indicando la motivación y propósito que impulsa el desarrollo de la aplicación, identificando los sectores a los que va dirigido, al igual que el impacto que se pretende generar con ella. Seguido de esto, se detalla la planificación seguida durante el desarrollo, la forma en la que se dividió y organizó el trabajo junto a las fechas de finalización de cada etapa. Finalizando con la parte de la memoria orientada a la planificación, se describen en alto nivel todas las funcionalidades que posee la aplicación, divididas en sus tres tipos: básicas, intermedias y avanzadas.

Llegando al cuerpo técnico del documento, se describe en detalle la arquitectura cliente - servidor adoptada por la aplicación, al igual que la arquitectura de despliegue que se utilizó para hacer público el producto, acompañado de las tecnologías y herramientas que se emplearon durante el proceso de desarrollo.

Se dedicó un apartado exclusivo a los detalles de implementación más relevantes del desarrollo, explicando la lógica detrás de algunas funcionalidades más avanzadas e importantes junto a sus respectivos anexos donde se muestra los respectivos fragmentos de código, además, se detallan las dificultades encontradas y la forma en la que se afrontaron.

A continuación, se describen los controles de calidad empleados para garantizar una correcta funcionalidad, consistencia e integridad en cada etapa de desarrollo, explicando el propósito de cada tipo de prueba que hay presente tanto en el cliente como en el servidor. Para finalizar la parte técnica del documento, se describe la metodología adoptada para el control de versiones, siendo este aspecto uno de los más importantes en cualquier desarrollo *software*. En esta sección se entra en profundidad en la forma en la que se empleó *GitHub Flow* junto al flujo de integración continua para garantizar no solo en cada etapa del desarrollo, sino en cada cambio, la correcta integración, distribución y despliegue de una versión estable e íntegra del producto.

Para concluir el trabajo, se realiza una pequeña reflexión final sobre los resultados obtenidos, la curva de aprendizaje que llevó el desarrollo de un proyecto de este calibre, junto a algunas de las múltiples funcionalidades que se podrían incluir para aumentar el valor de *Diamond Insights*.

Para facilitar la navegación, el contenido se organizó siguiendo la estructura que encontrará a continuación:

- **Capítulo 1: Introducción y Objetivos:** Se define el propósito del proyecto, motivación, y los objetivos funcionales y técnicos.
- **Capítulo 2: Metodología:** La planificación que se adoptó durante el desarrollo.
- **Capítulo 3: Descripción funcional:** Descripción en alto nivel de las funcionalidades que ofrece la aplicación, clasificadas como: básicas, intermedias y avanzadas.
- **Capítulo 4: Descripción informática:** Contiene los aspectos técnicos del desarrollo, explicando las tecnologías y herramientas empleadas, arquitectura adoptada, aspectos más destacados sobre la implementación, controles de calidad y gestión de versiones.
- **Capítulo 5: Conclusiones y trabajos futuros:** Reflexión final del trabajo, donde se realiza una valoración del producto obtenido, conocimiento aprendido, áreas de mejora, y algunas funcionalidades que se podrían añadir para continuar con el desarrollo.

Palabras clave:

- *MLB*
- *Framework*
- *Java*
- *Angular*
- *TypeScript*
- *PostgreSQL*
- Cliente
- Servidor
- *SPA*
- *Spring Boot*
- *API*
- *API REST*
- Caché

- *Cloudinary*
- Resiliencia
- Integración continua (CI)
- *Git*
- *GitHub*
- *GitHub Flow*
- *Workflows*
- *Grafana*
- *Prometheus*
- *Docker*
- *Railway*

Índice de contenidos

Índice de tablas

Índice de figuras

Índice de códigos **1**

1 Introducción y objetivos 3

1.1 Contexto 3

1.1.1 Motivación y justificación del proyecto 3

1.1.2 Utilidad 4

1.1.3 Público objetivo 4

1.1.4 Funcionalidades principales 5

1.2 Estado del arte 5

1.3 Objetivos 6

1.3.1 Objetivos funcionales 6

1.3.2 Objetivos técnicos 6

2 Metodología 9

3 Descripción funcional 12

3.1 Funcionalidades básicas 12

3.1.1 Funcionalidades comunes 12

3.1.2 Funcionalidades de usuarios registrados 14

3.1.3 Funcionalidades de administradores 14

3.2 Funcionalidades intermedias 17

3.2.1 Borrado de cuenta y gestión de notificaciones 17

3.2.2 Soporte 18

3.2.3 Venta de entradas y gestión de eventos 18

3.3 Funcionalidades avanzadas 20

3.3.1 Información analítica 20

3.3.2 Clasificación de jugadores 21

3.3.3 Estadísticas de equipos 22

4 Descripción Informática 24

4.1	Lenguajes	28
4.2	Tecnologías	28
4.2.1	Cliente	28
4.2.2	Servidor	29
4.2.3	Base de Datos (<i>PostgreSQL</i>)	29
4.2.4	Despliegue	30
4.3	Herramientas	30
4.4	Arquitectura	32
4.4.1	Entornos de Ejecución	32
4.4.2	Arquitectura de Despliegue	33
4.4.3	Modelo del dominio	34
4.4.4	Arquitectura del servidor	38
4.4.5	Arquitectura del cliente	40
4.4.6	Grafana	41
4.5	Implementación	42
4.5.1	Gestión de datos externos	42
4.5.2	Resiliencia	43
4.5.3	Asincronía	45
4.5.4	Patrones de Diseño	45
4.5.5	<i>Gestión de lotes en la creación de entradas</i>	47
4.5.6	Sistema de mensajería interno	48
4.5.7	Algoritmos de consulta avanzada	48
4.5.8	Optimizaciones	50
4.5.9	<i>Mecanismo de sondeo</i>	52
4.5.10	Problemas más destacados	52
4.6	Control de calidad	53
4.6.1	Servidor	53
4.6.2	Cliente	56
4.7	Despliegue	58
4.7.1	Empaquetado con <i>Docker</i>	58
4.7.2	Despliegue con <i>Railway</i>	59
4.8	Proceso de desarrollo	60
4.8.1	Gestión de tareas	60
4.8.2	<i>Git</i> y <i>GitHub</i>	60
4.8.3	Integración y entrega continua	62
4.8.4	Despliegue continuo y versionado	65
5	Conclusiones y trabajos futuros	68
5.1	Conclusiones	68
5.2	Trabajos futuros	69
	Bibliografía	71

Anexos	75
A Funcionalidades detalladas	77
A.1 Funcionalidades básicas	77
A.1.1 Registro de usuario	77
A.1.2 Iniciar sesión	78
A.1.3 Cerrar sesión	79
A.1.4 Añadir equipo favorito	80
A.1.5 Eliminar equipo favorito	80
A.1.6 Editar ajustes del perfil	81
A.1.7 Actualizar información del equipo	82
A.1.8 Editar información del jugador	83
A.1.9 Editar información del estadio	84
A.1.10 Crear estadio	85
A.1.11 Crear jugador	86
A.1.12 Eliminar jugador	87
A.2 Funcionalidades intermedias	88
A.2.1 Eliminar cuenta	88
A.2.2 Compra de entradas	89
A.2.3 Notificaciones por correo electrónico	90
A.2.4 Contactar con soporte por correo electrónico	91
A.3 Funcionalidades avanzadas	92
A.3.1 Analíticas de visibilidad de la aplicación	92
A.3.2 Analíticas de rendimiento de la <i>API</i>	92
A.3.3 Gestión de la caché	93
A.3.4 Estadísticas de los equipos favoritos de los usuarios	94
A.3.5 Clasificaciones de jugadores	94
A.3.6 Estadísticas de equipos	95
B Instrucciones para ejecutar la aplicación	97
B.1 Requisitos para <i>Windows</i> y <i>MacOS</i>	97
B.2 Requisitos para <i>Linux</i>	98
B.3 Instrucciones de ejecución	98
C Ejecución y edición de código	99
C.1 Clonación del repositorio	99
C.2 Requerimientos	100
C.3 Base de datos	100
C.3.1 Local - Perfil <i>dev</i>	101
C.3.2 <i>Docker</i> - Perfil <i>docker</i>	101
C.3.3 Producción - Perfil <i>prod</i>	101
C.4 Servidor	102

C.5	Cliente	103
C.6	Generación del ejecutable	104
D	Instrucciones de ejecución de las pruebas	105
D.1	Servidor	105
D.2	Cliente	105
E	<i>Endpoints</i> de la <i>API REST</i> de la aplicación	107
E.1	Recurso Authentication	107
E.2	Recurso Users	109
E.3	Recurso Teams	111
E.4	Recurso Players	115
E.5	Recurso Stadium	119
E.6	Recurso Matches	122
E.7	Recurso Events	126
E.8	Recurso Tickets	131
E.9	Recurso Support	132
E.10	Recurso Admin Support	132
E.11	Recurso Analytics	134
E.12	Recurso Cache	136
F	Extensión arquitectura del servidor	137
G	Extensión arquitectura del cliente	141
H	Implementación <i>MatchImportService</i>	148
I	Implementación <i>TeamImportService</i>	151
J	Implementación <i>PlayerImportService</i>	154
K	Configuración de <i>Grafana</i> y <i>Prometheus</i>	156
K.1	Configuración de <i>Prometheus</i>	156
K.2	Configuración de <i>Grafana</i>	156
K.3	Despliegue	157

Índice de tablas

4.1	Tecnologías y herramientas	27
4.2	Relaciones de la entidad <i>Team</i>	35
4.3	Relaciones de la entidad <i>Player</i>	36
4.4	<i>Schedulers</i>	43
4.5	Patrones de diseño del servidor	45
4.6	Patrones de diseño del cliente	46
4.7	Atributos indexados en la base de datos	51
4.8	Impacto de la optimización con <i>EntityGraph</i> en la latencia de consultas	52
4.9	Resumen estadístico del conjunto de pruebas y cobertura del servidor	54
4.10	Resumen estadístico del conjunto de pruebas y cobertura del cliente	56
4.11	Métricas de actividad del repositorio	61
4.12	Historial de versiones y funcionalidades desarrolladas	65
C.1	Requisitos para la ejecución de la aplicación	100
K.1	Credenciales de acceso a <i>Grafana</i>	157

Índice de figuras

2.1	Cronograma del proyecto (diagrama de Gantt)	10
3.1	<i>Widget</i> de partidos del día	13
3.2	<i>Widgets</i> del menú principal	13
3.3	Pestaña "Favorite Teams"	14
3.4	Pestaña <i>Edit Menu</i>	15
3.5	Edición de equipo	16
3.6	Pestaña <i>Create Player</i>	16
3.7	Página de perfil del usuario	17
3.8	Creación de evento	19
3.9	Selección de entrada	19
3.10	Compra de entrada	19
3.11	Página <i>Analytics</i>	21
3.12	Analíticas de crecimiento de la aplicación	21
3.13	Sección <i>Stats</i> de la página del equipo	22
4.1	Entornos de ejecución de la aplicación	33
4.2	Diagrama de secuencia de la arquitectura de la aplicación	34
4.3	Diagrama relacional parte 1	36
4.4	Diagrama relacional parte 2	37
4.5	Diagrama relacional parte 3	37
4.6	Diagrama de clases del módulo de equipos y partidos del servidor	39
4.7	Diagrama de clases de la gestión de importación de datos	39
4.8	Diagrama de clases del módulo de partidos del cliente	40
4.9	Diagrama de clases del módulo de equipos del cliente	41
4.10	Máquina finita de estados del <i>@CircuitBreaker</i>	44
4.11	Diagrama de actividad del funcionamiento del sistema de caché local	50
4.12	Resultado de la ejecución de las pruebas del servidor	55
4.13	Análisis estático de código con Sonar	55
4.14	Reporte de cobertura de las pruebas en el cliente	57
4.15	Panel de despliegue de <i>Railway</i>	59
4.16	Métricas de código por <i>GitHub</i>	61
4.17	Cantidad de <i>commits</i> semanales por <i>GitHub</i>	62

4.18	Diagrama del flujo de integración continua	66
F.1	Diagrama de clases del módulo de gestión de entradas en el servidor	137
F.2	Diagrama de clases del módulo de gestión de usuarios	138
F.3	Diagrama de clases del módulo de modificación de entidades en el servidor	138
F.4	Diagrama de clases del módulo de soporte en el servidor	139
F.5	Diagrama de clases del módulo de analíticas en el servidor	139
F.6	Diagrama de clases del módulo de clasificación de jugadores en el servidor	139
F.7	Diagrama de clases del módulo de las estadísticas del equipo en el servidor	140
G.1	Diagrama de clases del módulo de modificación de entidades en el cliente	141
G.2	Diagrama de clases del módulo de gestión de entradas en el cliente	142
G.3	Diagrama de clases del módulo de la barra de navegación	143
G.4	Diagrama de clases del módulo de clasificación de jugadores en el cliente	144
G.5	Diagrama de clases del módulo de soporte en el cliente	145
G.6	Diagrama de clases del módulo de autenticación	146
G.7	Diagrama de clases del módulo de perfil de usuario	146
G.8	Diagrama de clases del módulo de analíticas en el cliente	147
G.9	Diagrama de clases del módulo de las estadísticas del equipo en el cliente	147
K.1	Mapa de despliegue del ecosistema de <i>Grafana</i>	157
K.2	<i>Dashboard</i> de <i>Grafana</i> de la aplicación	158

Índice de códigos

4.1	Implementación del <i>SeatBatchService</i>	47
4.2	Obtención de la clasificación de una métrica concreta	49
B.1	Plantilla del archivo <i>.env</i>	98
C.1	Configuración de la conexión en local con la base de datos	101
C.2	Configuración de la conexión en <i>docker</i> con la base de datos . . .	101
C.3	Configuración de la conexión en producción con la base de datos .	101
C.4	Fragmento del archivo de configuración global de la aplicación . .	102
H.1	Implementación de <i>getOfficialMatches</i>	148
H.2	Implementación de <i>verifyMatchStatus</i>	149
H.3	Implementación de <i>MatchScheduler</i>	150
I.1	Implementación de <i>getTeamStats</i>	152
I.2	Implementación de <i>TeamScheduler</i>	153
J.1	Implementación de <i>getTeamRoster</i>	154
J.2	Implementación de <i>getPlayerStatsFromAPI</i>	155
J.3	Implementación de <i>savePlayerPicture</i>	155
J.4	Implementación de <i>PlayerSecheduler</i>	155

1

Introducción y objetivos

En este capítulo se presentará la aplicación, mostrando la idea que propulsó el desarrollo junto a su propósito y el público al que pretende cautivar, además, se describirán los objetivos funcionales y técnicos planteados al inicio del proceso de desarrollo.

1.1 Contexto

En esta sección se explicará la motivación que llevó al desarrollo de este proyecto, junto a su propósito y el público que pretende cautivar.

1.1.1 Motivación y justificación del proyecto

Diamond Insights es una aplicación dedicada a todos los aficionados del béisbol con la intención de proporcionar un seguimiento en tiempo real de la temporada actual de la ***Major League Baseball (MLB)***, acompañado de estadísticas y analíticas sobre el desempeño de los equipos y jugadores de una manera única y adaptada a los gustos de cada fanático. La intención principal es ofrecer una experiencia que combine el seguimiento de cada jornada con una profundidad analítica, que, hasta ahora, es difícil de encontrar de forma unificada en otros portales.

1.1.2 Utilidad

La importancia de implementar una aplicación de estas características reside en la evolución del perfil del aficionado moderno. En los últimos años, el consumo de este deporte ha evolucionado, ahora, tanto equipos como aficionados buscan interactuar y estar al tanto de las analíticas que rodean la temporada, habiendo cada vez más formas de medir lo que sucede en el campo. *Diamond Insights* busca satisfacer las necesidades de estos perfiles al brindar acceso a métricas detalladas, permitiendo que el usuario no solo consulte resultados o clasificaciones, sino que también comprenda el desempeño de equipos y jugadores mediante estadísticas avanzadas y personalizables a sus preferencias.

1.1.3 Público objetivo

La aplicación ofrece una variedad de funcionalidades que van dirigidas a tres perfiles principales de usuario:

- **Aficionado casual:** Aquel que solo busca una herramienta que le proporcione información diaria en tiempo real. Para este perfil, la aplicación ofrece funcionalidades como la consulta de partidos, clasificaciones, venta de entradas, entre otras.
- **Analista de datos:** Aquellos usuarios que buscan ir más allá de lo tradicional. Para este perfil, la aplicación ofrece funcionalidades que permiten analizar el rendimiento dentro de unas determinadas estadísticas, mediante una serie de filtros que el usuario tendrá a su disposición para personalizar sus búsquedas y análisis.
- **Cazatalentos o scouts:** Más allá del aficionado, la aplicación pretende posicionarse como una herramienta de apoyo para *scouts* de diferentes equipos de la *MLB*, proporcionando métricas precisas y personalizables sobre el desempeño de los jugadores. *Diamond Insights* facilitaría la toma de decisiones estratégicas en los procesos de búsqueda y contratación de jugadores.

1.1.4 Funcionalidades principales

Para cumplir con el propósito mencionado anteriormente y alcanzar al público objetivo, la aplicación se construye en base a una serie de funcionalidades de las cuales destacan las siguientes:

- **Seguimiento en tiempo real:** Ofrece un seguimiento en vivo de la temporada actual, no solo de los partidos del día, sino también de las clasificaciones y estadísticas.
- **Venta de entradas:** Ofrece a los aficionados la posibilidad de comprar entradas para asistir a los partidos de sus equipos favoritos.
- **Personalización:** La aplicación está construida en base a la personalización, desde poder filtrar las estadísticas y clasificaciones según los gustos del usuario, a priorizar la información que se muestra a través de la selección de equipos como favoritos.
- **Panel analítico:** Ofrece un espacio dedicado al análisis de métricas de rendimiento de equipos y jugadores a lo largo de la temporada.

1.2 Estado del arte

Debido a que esta aplicación surgió de una motivación y propósito personal, no se encontraron múltiples aplicaciones o plataformas con un alto grado de similitud a la desarrollada, sin embargo, las que más destacan son las siguientes:

- **Aplicación oficial de la *MLB*:** Al ser la plataforma oficial de la *MLB* existen múltiples similitudes, como el formato de las clasificaciones o partidos del día, sin embargo, *Diamond Insights* difiere de esta en la parte analítica, puesto que además de aportar las métricas de forma numérica, también se representan de manera gráfica, aportando también la posibilidad de realizar filtrados adaptados a los gustos del usuario.
- ***Driblab PRO*:** Es una plataforma web orientada al análisis exhaustivo de datos en las múltiples ligas de fútbol del mundo. La principal diferencia es el sector al que va dirigido, puesto que la aplicación desarrollada va dirigida al aficionado del béisbol, *Driblab* orienta sus servicios hacia el perfil del aficionado futbolístico. Ambas ofrecen servicios similares, no obstante, el grado de detalle o de funcionalidad de esta última es mucho mayor que la desarrollada.

1.3 Objetivos

En esta sección se describirán los objetivos funcionales y técnicos planteados al inicio del desarrollo de la aplicación.

1.3.1 Objetivos funcionales

El principal objetivo funcional de esta aplicación es ofrecer a los aficionados al béisbol un portal completo y personalizado para la *Major League Baseball (MLB)*, que les permita acceder a información general de la liga, datos de equipos y jugadores, clasificaciones y resultados de partidos, así como acceder a servicios interactivos como la compra de entradas, la personalización de contenido según sus equipos favoritos y mantenerse al día sobre las últimas novedades de la liga mediante notificaciones por correo electrónico. Además, para aquellos fanáticos interesados en la parte más analítica de la temporada, se ofrecen métricas sobre el desempeño de jugadores y equipos de la temporada actual. A continuación, se mencionan alguna de ellas:

- Ofrecer información actualizada de la temporada actual.
- Ofrecer información estadística personalizable y detallada sobre equipos y jugadores.
- Venta de entradas para todos los usuarios registrados
- Ofrecer a los administradores información analítica sobre el rendimiento y alcance de la aplicación.

1.3.2 Objetivos técnicos

La aplicación se desarrollará como una plataforma web con arquitectura cliente-servidor, utilizando el protocolo *HTTPS* para garantizar la seguridad, la protección de datos y la comunicación segura.

El servidor se implementará con *Spring Boot* para gestionar la lógica de negocio, la comunicación con la base de datos *PostgreSQL* y la exposición de servicios mediante una *API REST*.

El cliente se implementará con *HTML*, *CSS* (con *TailwindCSS* [1]), *TypeScript* y *Angular*, ofreciendo una interfaz amigable para los usuarios. Se integrará la biblioteca *ng2-charts* para mostrar datos estadísticos de manera gráfica.

REST Client (extensión de *Visual Studio*) se utilizará durante el desarrollo y las pruebas de los servicios *REST*, y el flujo de trabajo incluirá control de versiones

con *Git*. Además, *Docker* se utilizará como parte de los procesos de Integración Continua (*CI*), Entrega Continua (*CD*) y Despliegue Continuo (*CD*), lo que garantiza que la aplicación se pueda compilar, empaquetar, probar y desplegar de forma consistente en diferentes entornos.

Para garantizar la calidad del *software*, se realizarán pruebas unitarias y de integración utilizando *Mockito* y *AssertJ*, así como pruebas de sistema/de extremo a extremo (e2e) utilizando *RestAssured* en el servidor, mientras que en el cliente se utilizará *Jasmine* y *Karma* para pruebas unitarias y de integración, y *Cypress* para pruebas de sistema.

- *Spring Boot* como *framework* del servidor para implementar la lógica de negocio y exponer la *API REST*.
- Se utilizarán *HTML*, *TailwindCSS*, *TypeScript* y *Angular* para implementar el cliente, garantizando una interfaz amigable.
- Almacenamiento y gestión de datos mediante una base de datos relacional *PostgreSQL*.
- Control de versiones con *Git*.

2

Metodología

En esta sección se describirán las distintas fases por las que ha pasado el desarrollo de la aplicación.

- **Fase 1 - Definición de funcionalidades:** Las funcionalidades de la aplicación se definirán y clasificarán en básicas, intermedias o avanzadas. Además, se definirán las entidades y sus relaciones, así como los diferentes tipos de usuario y sus permisos correspondientes dentro de la aplicación. **Inicio:** 16/06/2025. **Final:** 03/09/2025.
- **Fase 2 - Configuración del repositorio:** En esta fase, se creará el repositorio junto con la implementación básica necesaria para conectar el cliente, el servidor y la base de datos. Además, se implementarán las primeras pruebas automatizadas y se configurará el sistema de integración continua (*CI*). **Inicio:** 09/09/2025. **Final:** 30/09/2025.
- **Fase 3 - Versión 0.1:** Se implementarán las funcionalidades básicas junto con sus correspondientes pruebas automatizadas, y la aplicación se empaquetará con *Docker*. Además, se integrará en el flujo de *CI* la fase de entrega continua (*CD*), automatizando el despliegue del contenedor. Finalmente, se lanzará la **versión 0.1**. Esta fase, junto a la 4 y 5, seguirá un proceso de desarrollo **iterativo e incremental**, lo que significa que la aplicación se construirá por partes o incrementos, añadiendo funcionalidades gradualmente hasta completarla. Cada incremento se revisará y perfeccionará mediante iteraciones sucesivas (fases 4 y 5). **Inicio:** 30/09/2025. **Final:** 19/12/2025.

- **Fase 4 - Versión 0.2:** Se implementarán las funcionalidades intermedias junto con sus correspondientes pruebas automatizadas. Finalmente, la aplicación será desplegada en los servidores de *Railway*, y se lanzará la **versión 0.2**. **Inicio:** 22/12/2025. **Final:** 24/02/2026.
- **Fase 5 - Versión 1.0:** Se implementarán las funcionalidades avanzadas junto con sus correspondientes pruebas automatizadas. En esta etapa, la aplicación se considerará completa y se lanzará la **versión 1.0**. **Inicio:** 25/02/2026. **Final:** 20/04/2026.
- **Fase 6 - Desarrollo de la memoria:** Se elaborará el primer borrador de la memoria. **Inicio:** 21/04/2026. **Final:** 24/05/2026.
- **Fase 7 - Preparación de la presentación:** Se empezará con la preparación de la defensa del trabajo de fin de grado (TFG) para finalmente presentarla. **Inicio:** 25/05/2026. **Final pronosticado:** 15/06/2026

En la **Figura 2.1** se muestra un **diagrama de Gantt** reflejando la planificación mencionada anteriormente.

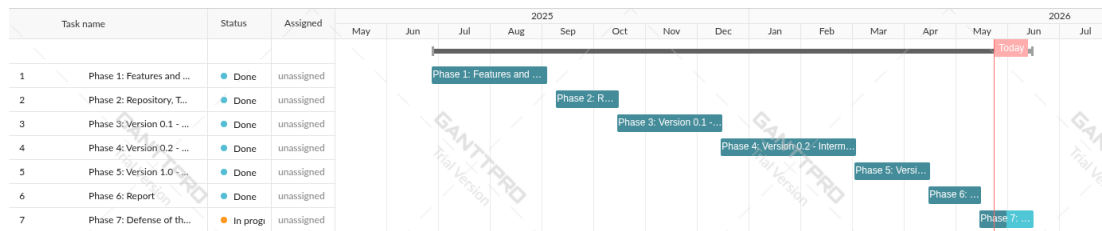


Figura 2.1: Cronograma del proyecto (diagrama de Gantt)

3

Descripción funcional

En este capítulo se realizará una descripción de las principales funcionalidades que ofrece la aplicación.

En el Anexo A podrá encontrar más detalles sobre cada una de las funcionalidades que se describirán a continuación.

Para una mejor comprensión visual de las funcionalidades, se incluye el siguiente enlace a un video demostrativo en *YouTube*: <https://youtu.be/6Edkjj7vElQ>.

3.1 Funcionalidades básicas

En esta versión se desarrollaron las principales funcionalidades de la aplicación. Estas incluyen el proceso completo de autenticación (incluida la recuperación de contraseña), la página de inicio con todas sus secciones (partidos diarios y el *widget* de clasificación), así como las páginas de clasificaciones y de equipos.

3.1.1 Funcionalidades comunes

La página de inicio incluye los partidos del día y el *widget* de la clasificación, que permite al usuario acceder rápidamente a esta información. Además, todos los usuarios tienen acceso a una página dedicada a cada equipo, donde encontrarán información relevante como campeonatos, detalles del estadio, estadísticas de la

temporada, plantilla y calendario.

En cuanto a la barra de navegación, su contenido varía según el tipo de usuario. Los usuarios también pueden alternar entre el modo oscuro y el modo claro en cualquier momento.

Las Figuras 3.1 y 3.2 muestran los *widgets* presentes en el menú principal.

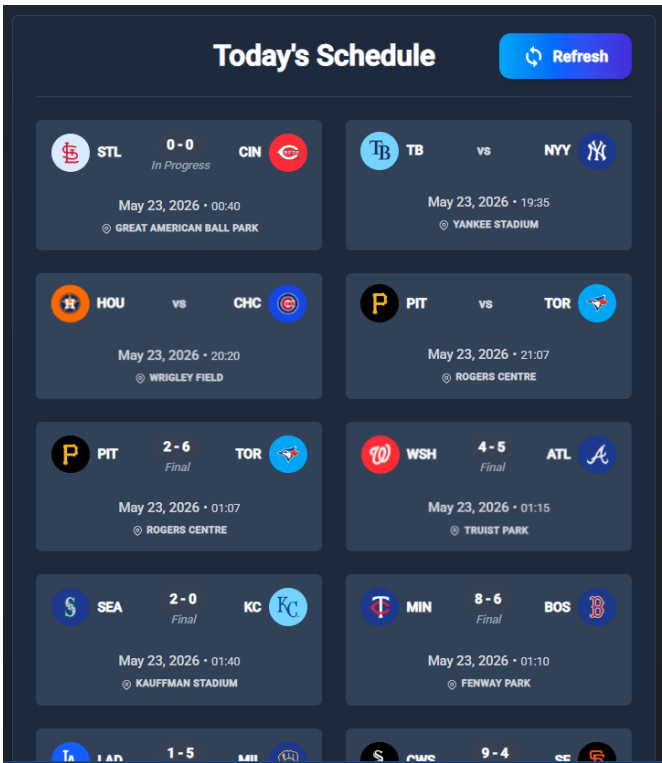


Figura 3.1: *Widget* de partidos del día

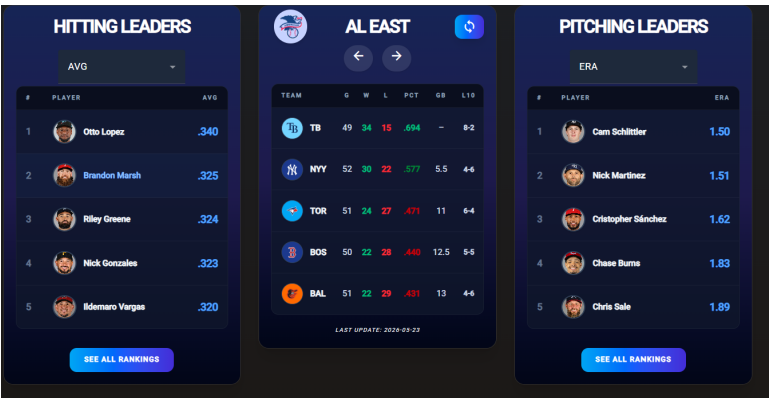


Figura 3.2: *Widgets* del menú principal

3.1.2 Funcionalidades de usuarios registrados

El usuario registrado puede gestionar sus equipos favoritos para personalizar la información que se muestra en la aplicación. De esta forma, al consultar los partidos del día o la clasificación, los primeros partidos y la clasificación que aparecerán serán los de sus equipos favoritos.

Para seleccionar un equipo como favorito, se hace visible un panel con todos los equipos de la *MLB*, donde el usuario puede seleccionar un equipo para añadirlo a su lista de favoritos. Para evitar que los usuarios seleccionen un equipo que ya aparece en la página principal, dicho panel solo muestra los equipos no seleccionados como favoritos por el usuario, actualizándose dinámicamente cada vez que se añade o elimina uno de la lista.

La **Figura 3.3** muestra la página de *Favorite Teams* de los usuarios registrados.

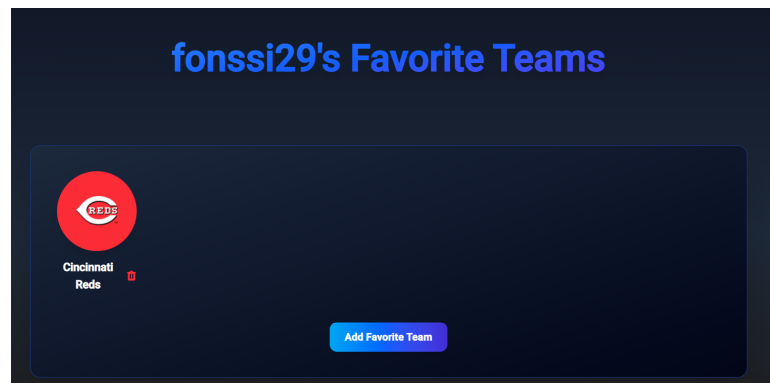


Figura 3.3: Pestaña "Favorite Teams"

3.1.3 Funcionalidades de administradores

Los administradores pueden modificar la información del equipo, el estadio y los jugadores. Para centralizar todas estas funciones en una sola sección, pueden acceder a la página *Edit Menu*, donde pueden buscar y seleccionar el tipo de entidad que desean editar.

- **Editar equipo:** Los campos editables de un equipo incluyen la ciudad, información general, campeonatos y estadio. Dado que el nombre del equipo sigue el patrón ciudad—nombre, cualquier cambio en la ciudad actualiza automáticamente el nombre del equipo. Por ejemplo, si un administrador cambia la ciudad de los *Yankees* de Nueva York a Caracas, el nombre actualizado será *Caracas Yankees*.

- **Editar jugador:** Los administradores pueden editar la información del jugador sin inconvenientes, pero al modificar el equipo al que pertenece, se abre un panel con la lista de todos los equipos disponibles de la *MLB*. En este contexto, un equipo disponible se define como aquel que no tiene una plantilla completa (menos de 26 jugadores). Esto garantiza que un administrador no pueda asignar un equipo que no tenga espacio para un jugador más.
- **Editar estadio:** Los administradores pueden editar las imágenes de un estadio. Las restricciones son que las imágenes deben tener un tamaño inferior a 1MB y que se permite un máximo de cinco imágenes por estadio. Esta última restricción se gestiona mediante el diseño de la interfaz de usuario, ya que la opción para cargar una nueva imagen solo es visible cuando el número de imágenes subidas es inferior a cinco.
- **Borrar jugador:** Eliminar a un jugador significa que se ha retirado de la *MLB*. Antes de realizar la operación, aparecerá un mensaje de confirmación para asegurar que el usuario realmente desea eliminar al jugador seleccionado de la base de datos, evitando así posibles errores.
- **Crear estadio:** Los usuarios administradores pueden crear un estadio; la única información que deberán proporcionar es el nombre y la fecha de inauguración.

Las **Figuras 3.4, 3.5 y 3.6** muestran la edición de equipos y creación de jugadores.

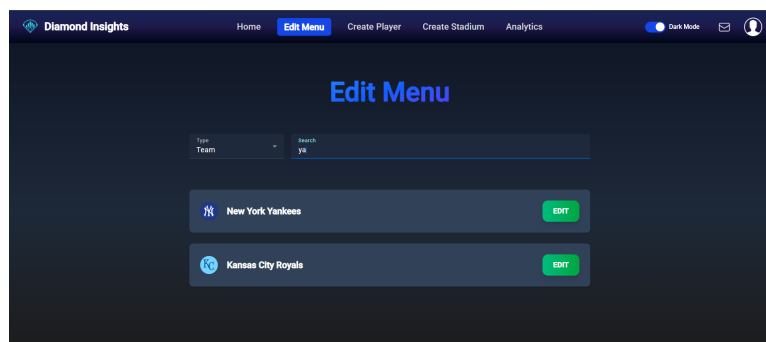


Figura 3.4: Pestaña *Edit Menu*

Edit New York Yankees



**NY Yankees**
Abbreviation

**AL**
League

**EAST**
Division

52
Games

30
Wins

22
Losses

0.577
Win %

4-6
Last Ten

City

City

New York

Stadium ⓘ

Yankee Stadium

General Info

General Information

Founded in 1901, the New York Yankees are the most successful franchise in Major League Baseball history. They have captured 27 World Series championships, featuring legendary players like Babe Ruth, Lou Gehrig, and Derek Jeter. Yankee Stadium is an iconic landmark where generations of fans have witnessed historic moments. The team is synonymous with excellence, tradition, and a culture of winning that spans over a century. Their pinstripe uniforms and global recognition make the Yankees a symbol of baseball greatness.

Championships

1923 1927 1928 1932 1936 1937 1938 1939 1941

1943 1947 1949 1950 1951 1952 1953 1956 1958

1961 1962 1977 1978 1996 1998 1999 2000 2009

Championship

Go Back

Confirm Changes

Figura 3.5: Edición de equipo

Create Player ⓘ

Name

Player #

Name

Number

Team

Team Name

Select Team

Position

Positions

Go Back

Confirm Changes

Figura 3.6: Pestaña *Create Player*

3.2 Funcionalidades intermedias

Esta segunda versión de la aplicación incluye las funcionalidades intermedias que van dirigidas a usuarios registrados y administradores, que se detallan a continuación:

3.2.1 Borrado de cuenta y gestión de notificaciones

Esta versión completa la sección de perfil al incorporar las funciones de **Eliminar cuenta** y **Notificaciones**. El nuevo servicio de notificaciones permite a los usuarios con equipos favoritos recibir recordatorios por correo electrónico 10 minutos antes del inicio del partido. Esta función se puede gestionar directamente desde la configuración del perfil del usuario.

La **Figura 3.7** muestra la página de perfil del usuario.

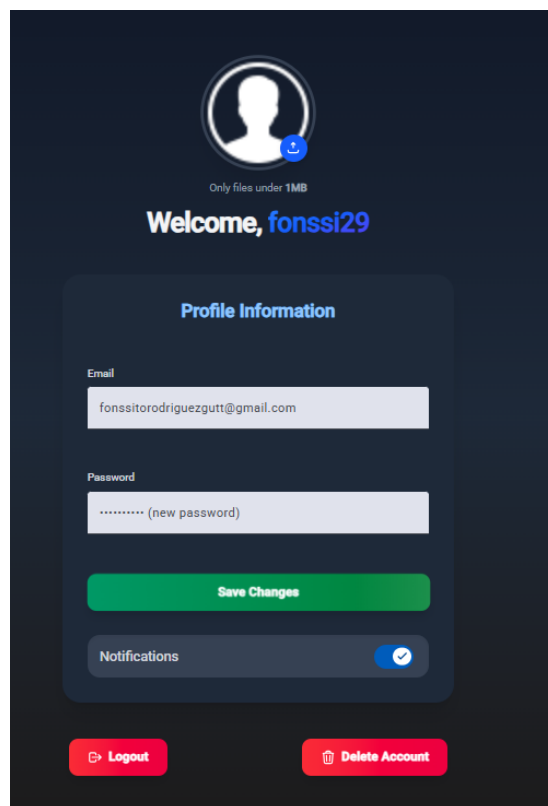


Figura 3.7: Página de perfil del usuario

3.2.2 Soporte

Con esta función, los usuarios ahora pueden contactar a los administradores de la aplicación para reportar cualquier problema que experimenten, y los administradores les podrán responder. Estos problemas se consideran *tickets* de soporte, y los usuarios pueden abrir uno directamente desde el pie de página.

Cuando se crea un *ticket*, los administradores reciben una notificación por correo electrónico y pueden acceder a dicha incidencia directamente desde su bandeja de entrada.

3.2.3 Venta de entradas y gestión de eventos

La principal novedad de esta versión es la compra de entradas para cualquier partido. Pueden acceder a la página de compra directamente desde la página del equipo, desplazándose hasta la sección ***Upcoming Home Games*** o haciendo *click* en el botón ***Calendar*** y seleccionando un partido. Esto los redirigirá a la página de compra, que está dividida en dos secciones: la de la izquierda muestra un mapa del estadio del equipo local, que se puede ampliar haciendo *click* sobre él, y el área debajo del cursor del ratón se amplía; la de la derecha permite al usuario introducir la cantidad de entradas deseadas, el sector y los asientos.

A continuación, el usuario debe introducir los datos de su tarjeta de crédito para completar la transacción. Recibirá un correo electrónico de confirmación con información sobre las entradas compradas en formato PDF. También puede consultar todas las entradas compradas en la sección ***My Tickets***.

Además, los administradores pueden crear, editar y eliminar eventos directamente desde la página del equipo, dependiendo de si el partido tiene un evento asignado o no:

- Si el partido no tiene ningún evento asignado → Botón Crear Evento.
- Si el partido tiene un evento asignado: Botón Editar y Eliminar Evento.

La aplicación no utiliza un servicio externo de procesamiento de pagos, por lo que el importe mostrado es ficticio y no se realizará ningún cargo. Si bien los datos de la tarjeta de crédito son ficticios, deben ajustarse al patrón y la estructura de una tarjeta real. Esto se debe a que el sistema simula el proceso de validación utilizando el algoritmo de *Luhn*, el método estándar que emplean las plataformas reales para verificar la integridad del número de la tarjeta.

Las **Figuras 3.8, 3.9 y 3.10** muestran las páginas de creación, selección y compra de entrada.

Boston Red Sox vs New York Yankees

Yankee Stadium
Saturday, June 6, 2026

Create New Event

Match ID: #28603

PLEASE ENTER THE NUMBER OF SECTORS FOR THIS EVENT

0

SECTOR NAMES (N/)

PRICES (N/)

CAPACITIES (N/)

Sector

Price

Capacity

Ready to generate event with sectors

GENERATE EVENT

Figura 3.8: Creación de evento

Boston Red Sox vs New York Yankees

Yankee Stadium
Saturday, June 6, 2026

STADIUM MAP

YANKEE STADIUM SEATING MAP

Ticket Selection

TICKET AMOUNT

Tickets

CHOOSE SECTOR

Select a Sector...

Select a Sector...

ESTIMATED TOTAL

\$0.00

Proceed to Payment

Figura 3.9: Selección de entrada

Diamond Insights

OFFICIAL TICKETING

EVENT DETAILS

Boston Red Sox

at New York Yankees

SELECTED SEATS

3 x 3

TOTAL TO PAY

\$100.00

GO BACK

SECURE CHECKOUT

Payment Details

CARDHOLDER NAME

Owner*

CARD NUMBER

Card Number*

EXPIRY DATE

MM/YY

CVV

CVV**

CONFIRM PAYMENT - \$100.00

Figura 3.10: Compra de entrada

3.3 Funcionalidades avanzadas

La versión final (1.0) de la aplicación se centra en el desarrollo de funcionalidades avanzadas orientadas al análisis de datos. Estas son:

3.3.1 Información analítica

Esta nueva sección de la aplicación, ubicada en la sección de ***Analytics***, permite a los administradores analizar los datos analíticos de la aplicación, incluyendo:

- **Estadísticas de crecimiento:** Modela datos relacionados con el crecimiento de la aplicación, específicamente mediante gráficos que muestran el número de visitas, nuevos usuarios y usuarios perdidos diariamente. El intervalo de tiempo se puede personalizar según las preferencias del usuario.
- **Rendimiento de la *API*:** Gráficos que muestran las solicitudes exitosas y fallidas, el número total de solicitudes y la latencia diaria, así como los 5 *endpoints* más utilizados por los usuarios. El intervalo de tiempo en el que se representan estos datos puede ser la última hora, día, semana o mes.
- **Gestión de caché:** El servidor utiliza múltiples *cachés* para mejorar los tiempos de respuesta y reducir el acceso a la base de datos. Los administradores pueden restaurar manualmente dichas *cachés* en caso de haber alguna discrepancia de datos.
- **Equipos más seleccionados como favoritos:** Modela los equipos que los usuarios han seleccionado con mayor frecuencia como favoritos.

Además, para ampliar este alcance analítico, la aplicación ahora incluye un panel de **Grafana** que muestra datos analíticos más detallados sobre el rendimiento del sistema. Se puede acceder a este panel de control a través del siguiente enlace: <https://diamond-insights-grafana.up.railway.app>.

Las Figuras 3.11 y 3.12 muestran el menú de analíticas de la aplicación, y la sección de analíticas de crecimiento.

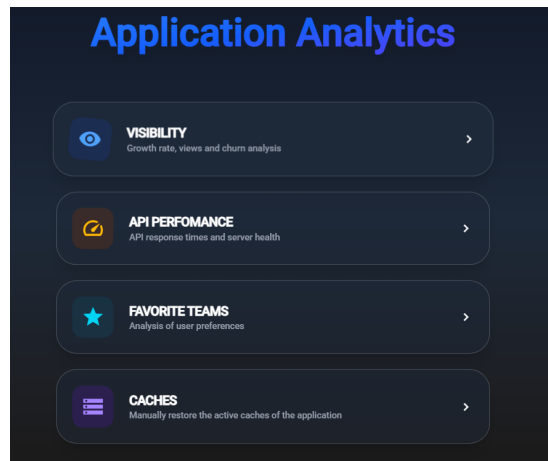


Figura 3.11: Página *Analytics*

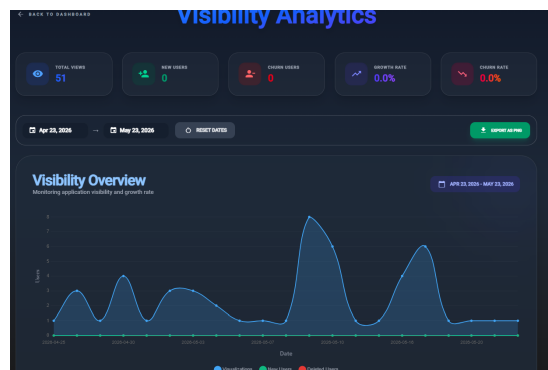


Figura 3.12: Analíticas de crecimiento de la aplicación

3.3.2 Clasificación de jugadores

Con el inicio de la temporada 2026 y la actualización diaria de las estadísticas de los jugadores, la aplicación ahora incluye la posibilidad de ver las clasificaciones de los jugadores para cada estadística. Esta función está disponible en el menú principal como un *widget*, lo que permite a los usuarios filtrar por la métrica especificada, o en la barra de navegación de la nueva sección de **Top Players**, donde pueden aplicar varios filtros, como liga, división y equipos, para acotar la lista de jugadores según sus preferencias. Además, cada clasificación ofrece la opción de visualizar la información de manera gráfica, haciendo *click* en el botón de la esquina superior derecha.

3.3.3 Estadísticas de equipos

Como resultado de esta función, la página del equipo tiene un nuevo diseño. Ahora, la información se divide en secciones (***Home, Stats, Roster y Matches***), con el objetivo de mejorar la experiencia del usuario y evitar una página demasiado extensa.

Esta nueva función, que se encuentra en la sección de ***Stats***, permite a los usuarios ver información estadística más detallada sobre el equipo seleccionado, como la evolución de la posición en su respectiva división a lo largo del tiempo, la distribución de victorias (como local y visitante), el total de carreras anotadas y permitidas, y su récord comparado con los rivales seleccionados.

Toda esta información (excepto la distribución de victorias) se puede comparar con la de otros equipos, que se pueden seleccionar manualmente o mediante filtros. De esta manera, los usuarios pueden comparar el rendimiento del equipo seleccionado con el de equipos de su misma división, liga o rivales con un porcentaje de victoria mayor o igual al 50%.

La **Figura 3.13** muestra la sección de estadísticas dentro de la página de equipo.

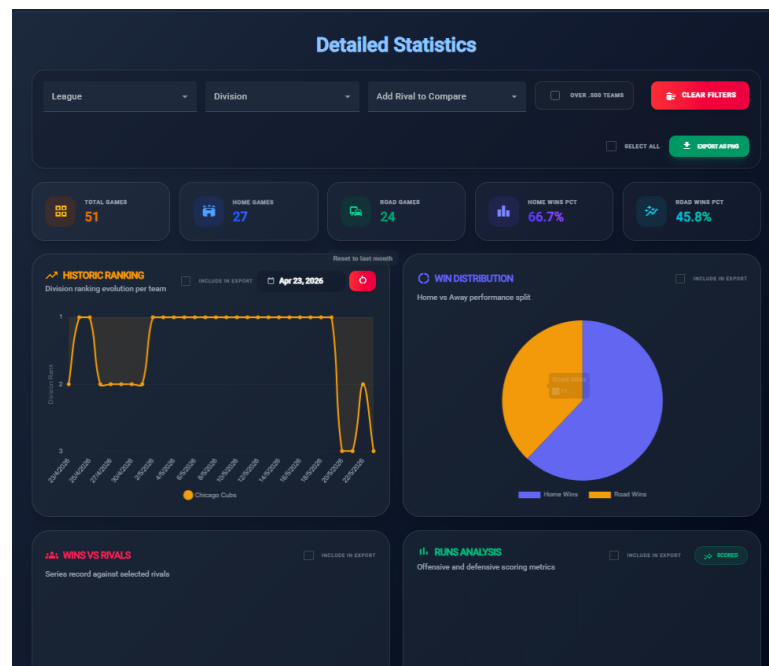


Figura 3.13: Sección *Stats* de la página del equipo

4

Descripción Informática

Contenido del capítulo

4.1	Lenguajes	28
4.2	Tecnologías	28
4.2.1	Cliente	28
4.2.2	Servidor	29
4.2.3	Base de Datos (<i>PostgreSQL</i>)	29
4.2.4	Despliegue	30
4.3	Herramientas	30
4.4	Arquitectura	32
4.4.1	Entornos de Ejecución	32
4.4.2	Arquitectura de Despliegue	33
4.4.3	Modelo del dominio	34
4.4.4	Arquitectura del servidor	38
4.4.5	Arquitectura del cliente	40
4.4.6	Grafana	41
4.5	Implementación	42
4.5.1	Gestión de datos externos	42
4.5.2	Resiliencia	43
4.5.3	Asincronía	45
4.5.4	Patrones de Diseño	45

4.5.5	<i>Gestión de lotes en la creación de entradas</i>	47
4.5.6	Sistema de mensajería interno	48
4.5.7	Algoritmos de consulta avanzada	48
4.5.8	Optimizaciones	50
4.5.9	<i>Mecanismo de sondeo</i>	52
4.5.10	Problemas más destacados	52
4.6	Control de calidad	53
4.6.1	Servidor	53
4.6.2	Cliente	56
4.7	Despliegue	58
4.7.1	Empaquetado con <i>Docker</i>	58
4.7.2	Despliegue con <i>Railway</i>	59
4.8	Proceso de desarrollo	60
4.8.1	Gestión de tareas	60
4.8.2	<i>Git</i> y <i>GitHub</i>	60
4.8.3	Integración y entrega continua	62
4.8.4	Despliegue continuo y versionado	65

En este capítulo se encontrarán los aspectos más técnicos e informáticos de la aplicación.

Todo el código desarrollado de esta aplicación, se puede encontrar en un repositorio de *GitHub* a través del siguiente enlace: <https://github.com/codeurjc-students/2025-MyMLB>.

La aplicación ***Diamond Insights*** está construida bajo una arquitectura monolítica, lo que significa que tanto el lado del servidor (*backend*) como el lado del cliente (*frontend*) se desarrollan, gestionan y despliegan de manera unificada. Dentro de esta estructura, el cliente se implementa mediante una ***Single Page Application (SPA)***. Una *SPA* es una aplicación web que carga un único archivo *HTML* y actualiza su contenido dinámicamente mediante *JavaScript*, en lugar de solicitar páginas completas al servidor con cada navegación. Esta solución del lado del cliente se ha desarrollado utilizando *Angular* 20 [2, 3].

En el lado del servidor, se desarrolló con ***Spring Boot*** [4], que expone una ***API REST*** como método de comunicación con el cliente.

Para la gestión de datos, la aplicación utiliza una base de datos ***PostgreSQL*** [5].

Para el despliegue y la *contenerización*, se utilizó ***Docker***.

En la **Tabla 4.1** se detallan todas las tecnologías y herramientas empleadas durante el proceso de desarrollo.

Tabla 4.1: Tecnologías y herramientas

Componente	Descripción
Tipo de aplicación	Web SPA con API REST
Arquitectura de la aplicación	Monolítica
Cliente	Angular, TailwindCSS
Servidor	Spring Boot
Base de datos	PostgreSQL
Sistema de caché local	Caffeine
Lenguajes	Java, SQL y TypeScript
IDEs	Visual Studio Code e IntelliJ
Herramientas auxiliares	DBeaver, REST Client (Extensión de Visual Studio), JaCoCo, Git y GitHub
Servicios externos	Cloudinary y statsAPI
Tests	Unitarios, Integración y Sistema (extremo a extremo)
Librerías de <i>testing</i>	JUnit, AssertJ, Mockito, REST Assured, Jasmine, Karma y Cypress
Tecnología de contenerización	Docker
Despliegue (PaaS)	Railway
Proceso de desarrollo	Iterativo e incremental, control de versiones con Git, y CI/CD con GitHub Actions
Métricas y analíticas	Actuator y Grafana
Resiliencia	Resilience4J

Este capítulo detalla el ecosistema tecnológico en el que se basó el desarrollo de la aplicación. Inicialmente, se mencionarán los lenguajes de programación, tecnologías y herramientas utilizadas, para luego profundizar en la arquitectura del sistema, desglosando el modelo de datos y la organización de las capas de cliente y servidor. Asimismo, se detallarán los aspectos más relevantes sobre la implementación, al igual que los problemas encontrados y sus respectivas soluciones. Seguido de ello, se explicarán los mecanismos de control de calidad aplicados, para finalmente explicar el proceso de despliegue y control de versiones.

4.1 Lenguajes

Para el desarrollo de la aplicación, se utilizaron los siguientes lenguajes:

- **Java:** Lenguaje de programación encargado de implementar la lógica de negocio, la conexión y persistencia con la base de datos y la gestión de tareas programadas, es decir, se encargó de implementar el servidor en su totalidad.
- **TypeScript:** Lenguaje de programación principal utilizado en el cliente para poder utilizar el *framework* de *Angular* [6].
- **SQL:** Lenguaje de consulta utilizado para interactuar con la base de datos en operaciones de escritura y lectura de datos.

4.2 Tecnologías

La aplicación utiliza las siguientes tecnologías para su ejecución:

4.2.1 Cliente

Node.js

Entorno de ejecución que permite ejecutar código *JavaScript* fuera del navegador. Esto hace que sea ideal para construir aplicaciones de *backend* rápidas, escalables y en tiempo real. Además, posee una arquitectura asíncrona orientada a eventos [7].

Node Package Manager (npm)

Gestor de paquetes por defecto de *Node.js*, utilizado para instalar, compartir y gestionar dependencias en proyectos *JavaScript* [8].

Angular

Es un *framework* de desarrollo web de código abierto basado en *TypeScript*, diseñado para construir aplicaciones *SPA* y escalables. Utiliza una arquitectura basada en componentes que facilita el desarrollo rápido y estructurado, junto a herramientas integradas para enrutamiento, comunicación entre cliente y servidor, etc. [9].

4.2.2 Servidor

Maven

Herramienta de gestión y comprensión de proyectos *software*, principalmente usada en *Java* para automatizar la construcción, empaquetado y documentación, basándose en un único archivo *pom.xml* [10].

Spring Boot

Framework de código abierto basada en Java que simplifica la creación, configuración y despliegue de aplicaciones web y microservicios [4]. Se divide en varios módulos, pero los principales son:

- ***Spring MVC***: Para desarrollar aplicaciones web basadas en la arquitectura Modelo Vista Controlador (MVC).
- ***Spring Data***: Para interactuar con la base de datos.
- ***Spring Security***: Para gestionar la autenticación y autorización.

Cloudinary

Plataforma *SaaS* (*Software as a Service*) basada en la nube dedicada al almacenamiento, administración y optimización de contenido multimedia (imágenes y videos) [11].

Resilience4J

Biblioteca ligera de tolerancia a fallos para aplicaciones *Java*. Ayuda a la aplicación a responder correctamente cuando se degrada un servicio externo del que depende [12, 13].

4.2.3 Base de Datos (*PostgreSQL*)

Sistema de gestión de base de datos relacionales, que destaca por su alta estabilidad, flexibilidad, robustez y cumplimiento de estándares. Permite almacenar y gestionar datos complejos de forma segura [14, 5].

4.2.4 Despliegue

Docker

Plataforma que permite a los desarrolladores empaquetar, distribuir y ejecutar aplicaciones dentro de contenedores ligeros e independientes. Dichos contenedores contienen todo lo necesario para que la aplicación sea portable (librerías, dependencias, etc) [15].

Railway

Plataforma que ofrece servicios de despliegue en la nube (PaaS) que simplifica el despliegue de aplicaciones, bases de datos y servidores, en forma de contenedores [16].

4.3 Herramientas

Durante el desarrollo de la aplicación se utilizaron los siguientes entornos de desarrollo integrados (IDE) y herramientas auxiliares:

IDEs

- **Visual Studio Code:** Editor de código abierto ligero con soporte para extensiones. Se utilizó para el desarrollo del cliente [17].
- **IntelliJ:** Potente entorno de desarrollo integrado (IDE) para *Java* y otros lenguajes de programación. Se utilizó para el desarrollo del servidor [18].

Base de Datos

- **DBeaver:** Una herramienta de gestión de bases de datos para explorar, consultar y administrar bases de datos *SQL* [19].

Herramientas Auxiliares

- **REST Client:** Extensión de Visual Studio Code que permite enviar solicitudes *HTTP* y ver las respuestas directamente dentro del editor.
- **Git:** Un sistema de control de versiones distribuido que se utiliza para realizar un seguimiento de los cambios en el código fuente durante el proceso de desarrollo *software* [20].

- **GitHub:** Una plataforma basada en la nube que aloja repositorios *Git* y añade funciones de colaboración para desarrolladores. En su interior podemos encontrar:
 - **GitHub Actions:** Herramienta de automatización que permite crear flujos de trabajo, utilizada principalmente para implementar el flujo de *CI/CD*.
 - **GitHub Projects:** Herramienta de gestión de proyectos que permite planificar el trabajo mediante tableros visuales, facilitando el seguimiento de las tareas y el progreso del desarrollo. En este proyecto, se empleó la metodología *Kanban*.
- **SonarQube:** Plataforma enfocada en realizar análisis estático de código para potenciar la calidad y seguridad del software [21].
- **JaCoCo:** Es una biblioteca de cobertura de código Java que se utiliza para medir cuánto código se ejecuta durante las pruebas automatizadas [22].
- **Docker Desktop:** Es una aplicación que proporciona una interfaz fácil de usar para crear, ejecutar y administrar contenedores *Docker* en *Windows* o *Mac*. También incluye *Docker Engine*, *Docker Compose*, etc., lo que simplifica el desarrollo basado en contenedores [23].

4.4 Arquitectura

En esta sección se describe la composición del sistema. En el lado del servidor, se detallan los entornos de ejecución, el modelo de dominio con sus respectivas entidades y relaciones, y la interacción entre las diferentes capas de la arquitectura hexagonal. Por otro lado, en la parte del cliente, se analiza la estructura y relación entre los módulos que lo conforman. Finalmente, se explicará la estructura y organización del ecosistema de *Grafana* junto a la base de datos *Prometheus*.

La *API REST* de la aplicación expone un total de **79 endpoints**, los cuales se encuentran detallados en la documentación oficial disponible en el siguiente enlace: <https://diamond-insights.up.railway.app/swagger-ui/index.html>. Asimismo, dicha información se encuentra recopilada para su consulta en el Anexo E.

4.4.1 Entornos de Ejecución

Para facilitar el proceso de desarrollo y pruebas, se configuraron cuatro perfiles diferentes para tareas y escenarios específicos. Estos perfiles permiten que la aplicación tenga distintas configuraciones según el entorno de ejecución (pruebas, producción, desarrollo y *docker*.), lo que favorece su mantenimiento y portabilidad. Esto también se ajusta a las prácticas modernas de las aplicaciones de *software* como servicio (*SaaS*), como la metodología de los Doce Factores [24].

A continuación, se detallan los perfiles de ejecución configurados:

- **Test:** Perfil exclusivo para pruebas; esto desactiva toda la seguridad de la aplicación (*SSL*) para evitar problemas en las pruebas de integración y de sistema en el proceso de *CI*. Hace uso de la base de datos en memoria *H2* y se configura en el archivo *application-test.properties*.
- **Prod:** Perfil principal y estable de la aplicación que utiliza una base de datos *PostgreSQL* para la persistencia de datos. Esto se configura en el archivo *application-prod.properties*.
- **Dev:** Perfil utilizado durante el desarrollo de cualquier funcionalidad o corrección de errores (desarrollo en local), empleando una base de datos *PostgreSQL* para la persistencia de datos en un entorno local. Esto se configura en el archivo *application-dev.properties*.
- **Docker:** Este perfil se utiliza únicamente para ejecutar la aplicación en un entorno de contenedores. Utiliza una base de datos *PostgreSQL* con la imagen *Docker postgres:16*. Esta configuración se encuentra en el archivo *application-docker.properties*.

Todas las configuraciones comunes de la aplicación se encuentran en el archivo *application.properties*.

En la **Figura 4.1** se muestra un diagrama reflejando el motor de base de datos empleado en cada perfil o entorno de ejecución.

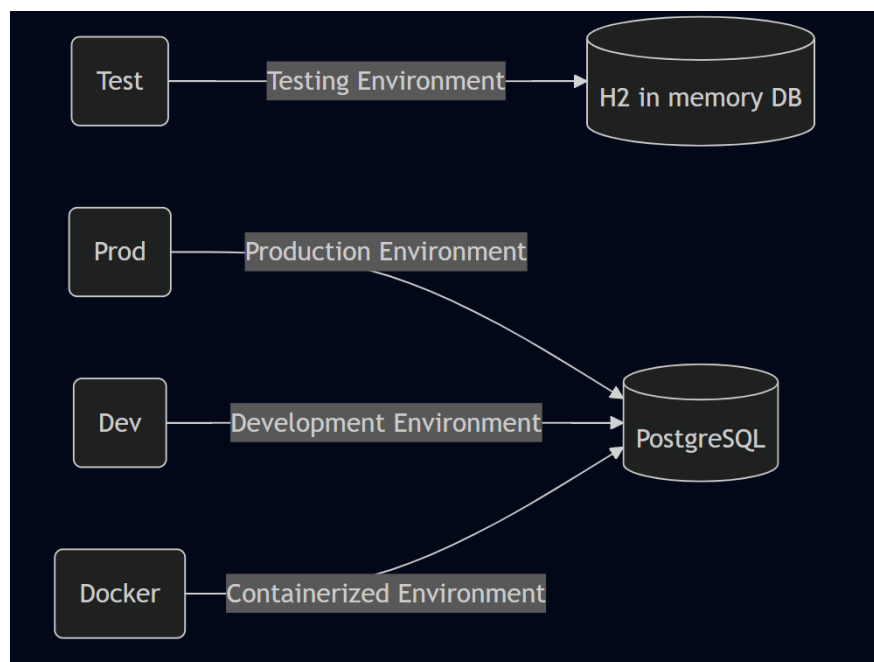


Figura 4.1: Entornos de ejecución de la aplicación

4.4.2 Arquitectura de Despliegue

En esta sección se describe la organización y comunicación de los procesos que componen el ecosistema de la aplicación en el entorno de producción (*Railway*). La arquitectura se divide en distintos módulos interconectados, estos son los siguientes:

- **Cliente (*Angular*):** Proceso encargado de la interfaz de usuario y se comunica de forma segura mediante *HTTPS* con el servidor.
- **Servidor (*Spring Boot*):** Constituye el núcleo del sistema. Centraliza la lógica de negocio, expone una *API REST* que se comunica con el cliente mediante objetos *JSON*. Además, orquesta la comunicación con la *API* externa (*statsAPI* de la *MLB*) y la base de datos.
- **Base de Datos (*PostgreSQL*):** Proceso independiente encargado de la persistencia de datos. El acceso está restringido al servidor para garantizar la integridad de la información.

- **Procesos asíncronos y API externa:** Para ofrecer en todo momento al usuario datos actualizados, se ejecutan tareas programadas (*Schedulers*) dentro del servidor diariamente. Estas obtienen datos a través de la API oficial de la MLB (*statsAPI*) mediante *RestTemplate*, y actualizan la base de datos local.
- **Monitorización:** Se desplegaron procesos específicos para la observabilidad. *Prometheus* recolecta métricas expuestas por *Spring Actuator* y *Grafana* las consume para ofrecer visualizaciones en paneles de control sobre el estado del servidor.

La **Figura 4.2** detalla el diagrama de secuencia que refleja el flujo de funcionamiento y la comunicación entre todos estos procesos.

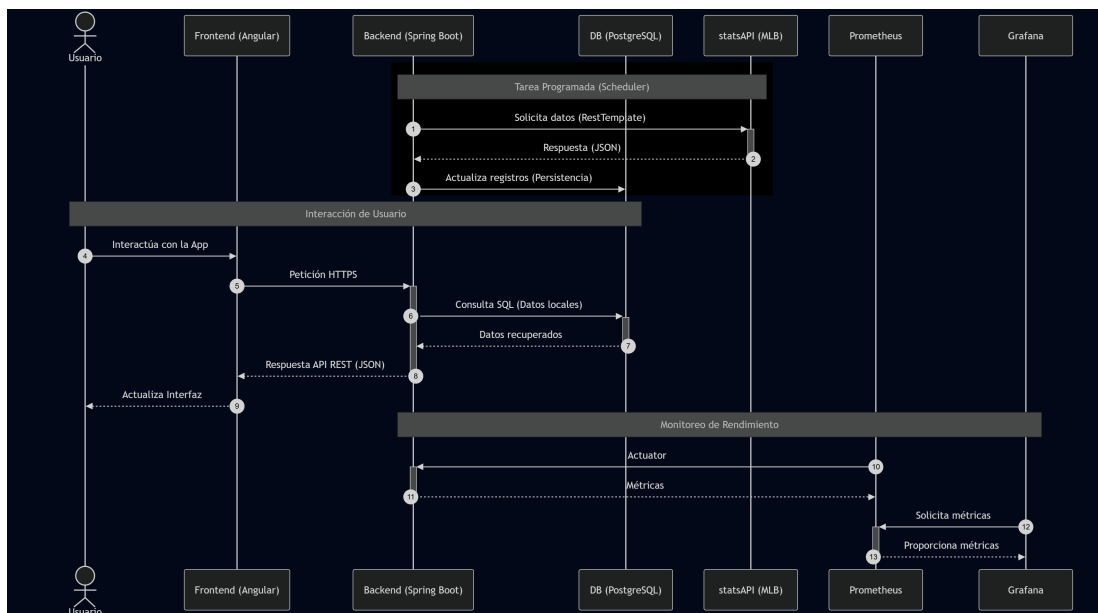


Figura 4.2: Diagrama de secuencia de la arquitectura de la aplicación

4.4.3 Modelo del dominio

En esta sección se describirán las entidades más importantes del modelo de dominio, junto a sus atributos más relevantes y relaciones con otras tablas. Adicionalmente, se mostrará el diagrama relacional de la aplicación, mostrando la interacción entre todas las entidades.

Entidad *Team*

Representa un equipo de la *MLB* conformado por 29 atributos de los cuales destacan los siguientes:

- ***statsApiId***: Identificador único en la *API* externa, fundamental para los procesos de sincronización de estadísticas oficiales.
- ***homeMatches* / *awayMatches***: Colecciones que registran los partidos como local y visitante, permitiendo la construcción del calendario de la temporada actual.
- ***stadium***: Referencia al estadio asociado. Se emplea tanto para las operaciones de modificación y creación de estadios y equipos, como en la visualización detallada del perfil del equipo.
- ***positionPlayers* / *pitchers***: Colecciones que registran los jugadores de posición y lanzadores del equipo. Usados para obtener la plantilla del equipo.
- ***favoriteByUsers***: Colección que registra los usuarios que seleccionaron dicho equipo como favorito. Usado para obtener las estadísticas de popularidad de equipos en la página *Analytics*.

Las relaciones lógicas de esta entidad se detallan en la **Tabla 4.2**.

Tabla 4.2: Relaciones de la entidad *Team*

Se relaciona con...	Cardinalidad
<i>UserEntity</i>	N..N
<i>Player</i>	1..N
<i>Stadium</i>	1..1
<i>Match</i>	1..N

Entidad *Player* y jerarquía de herencia

Esta entidad representa a un jugador de la *MLB*. Debido a la existencia de dos perfiles con métricas y posiciones claramente diferenciables (jugadores de posición y lanzadores), se ha optado por una estructura jerárquica. La entidad *Player* actúa como base (clase abstracta), de la cual heredan las entidades *PositionPlayer* y *Pitcher*.

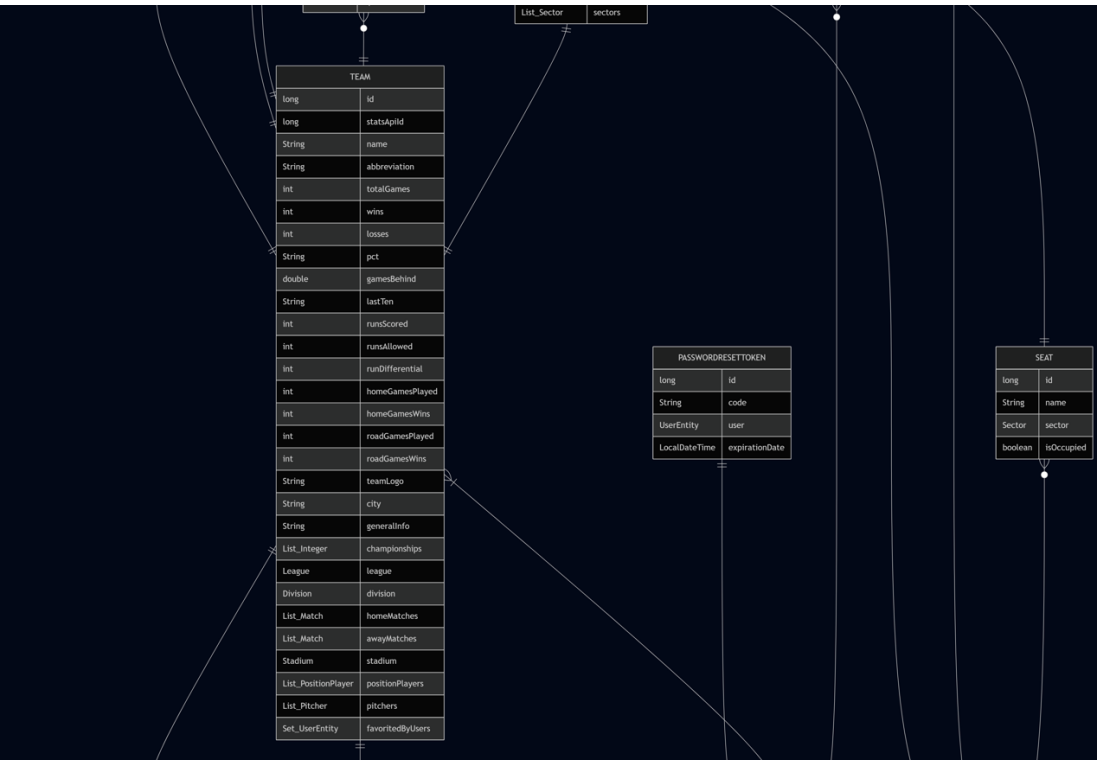


Figura 4.4: Diagrama relacional parte 2

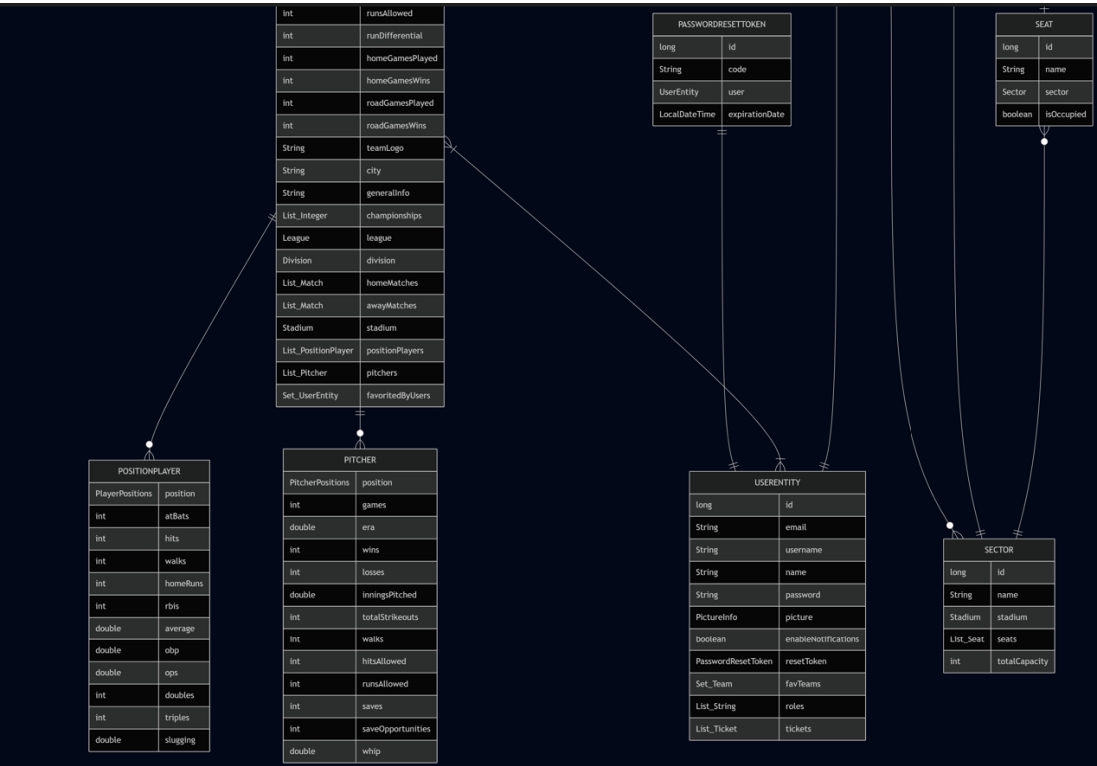


Figura 4.5: Diagrama relacional parte 3

4.4.4 Arquitectura del servidor

En esta sección se detallará la organización estructural del servidor basada en la arquitectura hexagonal. Dicha estructura se encuentra conformada por cuatro capas:

- **Capa de dominio (entidades):** Representa la capa más interna de la arquitectura, donde se definen los modelos de datos y las reglas de negocio.
- **Capa de aplicación (servicios):** Definen los servicios que coordinan el sistema, donde se centraliza toda la lógica de negocio y se desarrollan los casos de uso.
- **Capa de adaptadores de entrada (controladores y *schedulers*):** Definen los puntos de entrada del sistema. En esta aplicación, existen 2 puntos de entrada:
 - **Controladores *REST*:** Gestionan las peticiones *HTTPS* y las interacciones con el cliente.
 - ***Schedulers*:** Disparan procesos asíncronos que mantienen la información de la aplicación actualizada en todo momento.
- **Capa de adaptadores de salida (repositorios):** Gestionan la comunicación con la base de datos.

Debido a que el presente trabajo tiene una extensión limitada, se mostrarán dos diagramas que representan dos de los módulos más relevantes (**Figuras 4.6 y 4.7**), si desea consultar y visualizar el resto de diagramas del resto de módulos, los podrá encontrar en el Anexo F.

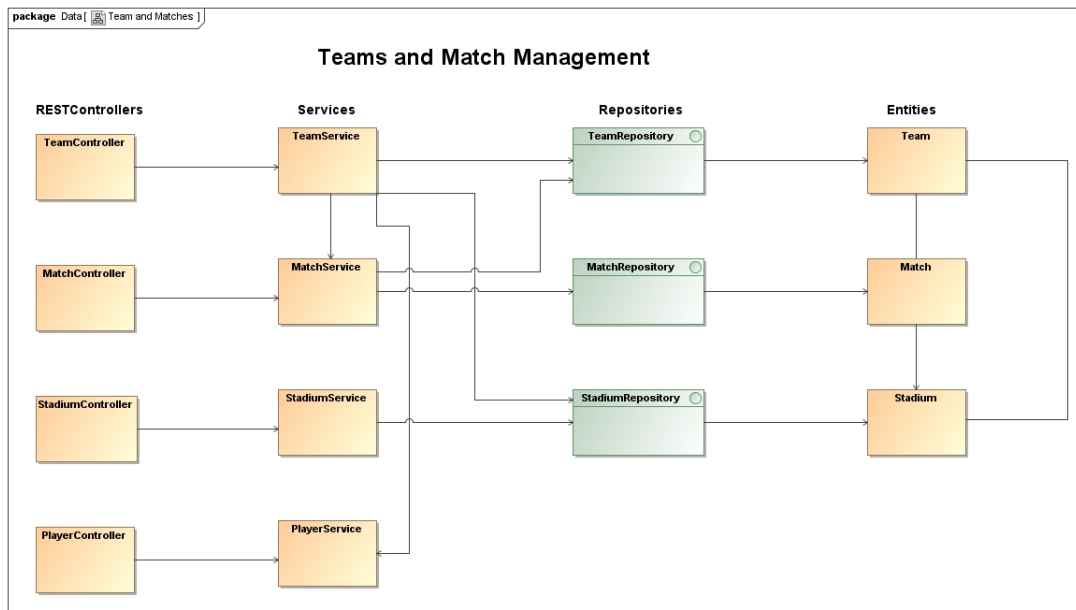


Figura 4.6: Diagrama de clases del módulo de equipos y partidos del servidor

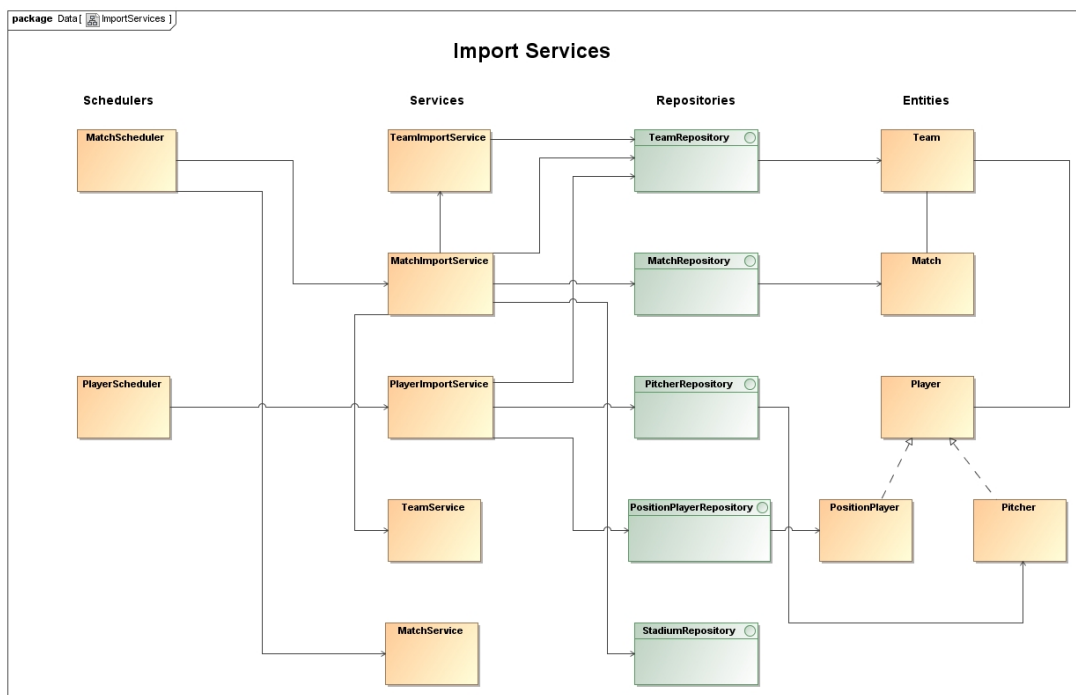


Figura 4.7: Diagrama de clases de la gestión de importación de datos

4.4.5 Arquitectura del cliente

En esta sección se detallará la organización estructural del cliente basada en un patrón de diseño orientado a componentes y servicios. Dicha estructura se encuentra conformada por tres capas:

- **Capa de interfaz (componentes):** Representan la parte visual de la aplicación, donde cada componente se encarga de gestionar fragmentos específicos de la interfaz de usuario.
- **Capa de lógica de negocio (servicios):** Centralizan la lógica de procesamiento y la comunicación con el servidor. Gracias a su naturaleza inyectable, estos servicios pueden ser reutilizados en cualquier parte del cliente.
- **Capa de datos (modelos):** Define la estructura de los objetos que se esperan recibir en el cliente a través de la *API REST*.

Debido a que el presente trabajo tiene una extensión limitada, se mostrarán dos diagramas que representan dos de los módulos más relevantes (**Figuras 4.8 y 4.9**), si desea consultar y visualizar el resto de diagramas del resto de módulos, los podrá encontrar en el Anexo G.

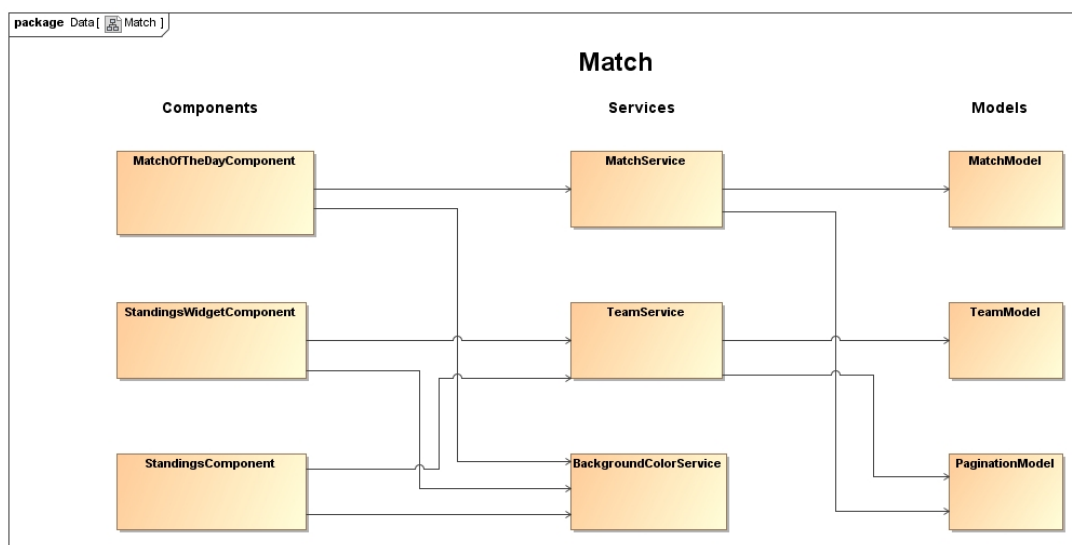


Figura 4.8: Diagrama de clases del módulo de partidos del cliente

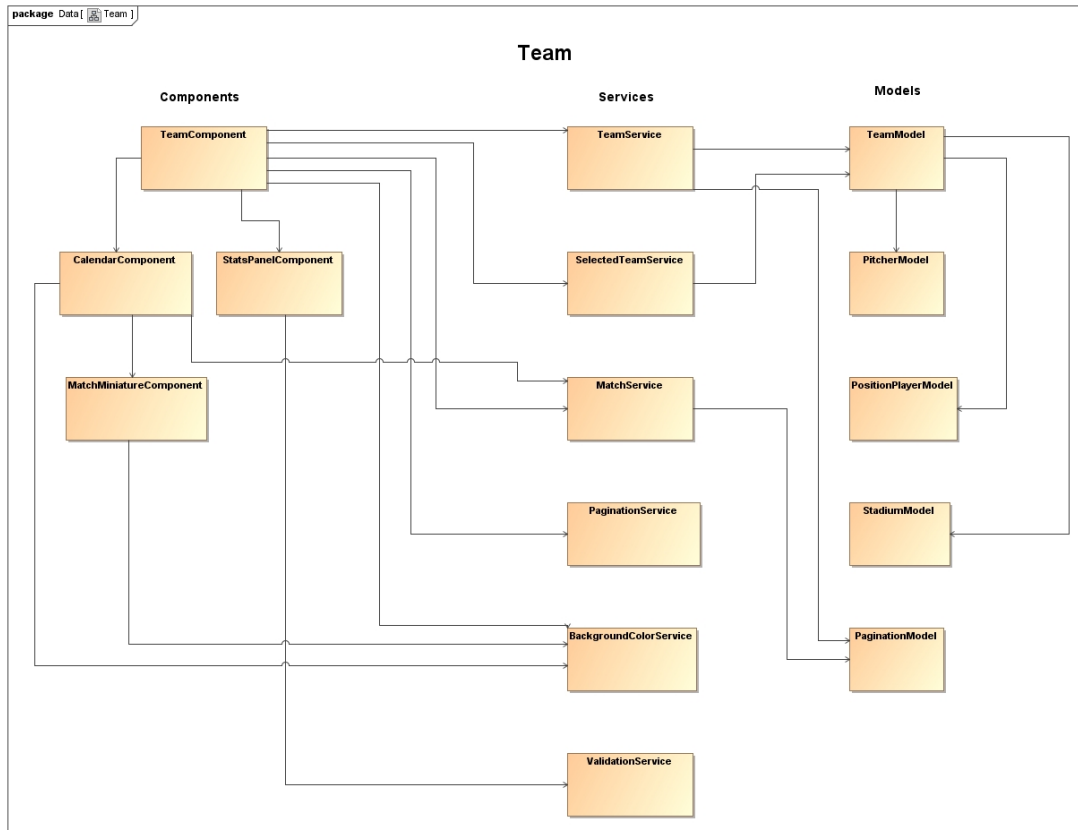


Figura 4.9: Diagrama de clases del módulo de equipos del cliente

4.4.6 Grafana

Es una plataforma dedicada al análisis de datos, mediante paneles interactivos que facilitan la visualización y el análisis [25]. Para ampliar el alcance analítico de la aplicación, se creó un panel para monitorizar el sistema del servidor, proporcionando diversos tipos de análisis, como la *JVM*, recolector de basura, grupos de búferes, carga de clases, entre otros.

Sin embargo, *Grafana* no puede funcionar de forma independiente; necesita una fuente de la que extraer todas estas métricas. Aquí es donde entra en juego **Prometheus** [26], una **base de datos de series temporales (TSDB)** [27]. A diferencia de una base de datos tradicional, una *TSDB* se centra en obtener datos estadísticos o métricas que cambian con el tiempo (datos históricos) [28].

Con esta combinación, *Grafana* dispone de todo lo necesario para funcionar, conformando así el **ecosistema de Grafana**, siendo este uno de los más populares para trabajar con esta plataforma [29]. Para su implementación, se creó una carpeta llamada **grafana-stack** en la raíz del repositorio, que contiene todos los archivos de configuración necesarios para su correcto funcionamiento. La fun-

cionalidad se divide en dos: configuración de *Prometheus* y de *Grafana*, dichas configuraciones se podrán consultar en el Anexo K.

4.5 Implementación

En esta sección se detallarán los aspectos más relevantes relacionados con la implementación de la aplicación.

4.5.1 Gestión de datos externos

Gran parte de los datos de la aplicación se obtienen directamente de la *API* oficial de la *Major League Baseball (MLB)*¹. El servidor se encarga de la comunicación con dicha *API* para extraer información clave en tiempo real, como el calendario de la temporada actual, las plantillas de cada equipo y las estadísticas detalladas de jugadores y equipos. A continuación, se detalla cómo se gestionan y procesan todos estos datos.

- ***MatchImportService***: En este servicio se centraliza toda la lógica de extracción de datos de los partidos de la temporada actual. Existen dos métodos principales ***getOfficialMatches*** que obtiene todos los partidos de las 162 jornadas de la temporada actual, y ***verifyMatchStatus*** que se ejecuta de manera regular y se encarga de comprobar el estado de los partidos del día actual, y verificar si han cambiado de estado (Programado, En Progreso, Finalizado). Para conocer su implementación, consulte el Anexo H.
- ***TeamImportService***: Servicio donde se centraliza la lógica de importación de los datos de los equipos. El método central del servicio es ***getTeamStats*** que se encarga de actualizar las estadísticas de la temporada actual de los equipos de forma regular. Para conocer su implementación, consulte el Anexo I.
- ***PlayerImportService***: Centraliza la lógica de importación de los datos de los jugadores. El servicio se basa en el método ***getTeamRoster*** que obtiene la plantilla de jugadores de cada equipo, y, además, internamente obtiene las estadísticas de los jugadores que se van actualizando periódicamente en la base de datos de la aplicación, mostrando siempre estadísticas que están en sincronía con las oficiales. Para conocer su implementación, consulte el Anexo J.

¹Documentación de *statsAPI*: <https://statsapi.mlb.com/docs/>

La lógica de extracción de datos es idéntica en los tres servicios: se utiliza **Rest-Template** para gestionar las peticiones, recuperar la respuesta y transformarla al formato requerido. Para asegurar un mapeo preciso, se diseñaron diversos **Data Transfer Objects (DTO)**, cada uno adaptado para representar fielmente la estructura de las respuestas específicas, que devuelve la *API* externa en cada caso. Cada uno de estos *DTOs* se encuentran en el paquete `/com.mlb.mlbportal/dto/mlbapi` del servidor.

Como bien se menciona anteriormente, existen datos de la *API* que necesitan ser actualizados regularmente, como pueden ser el estado de los partidos del día, estadísticas de jugadores, clasificaciones, entre otros. Para ello, se implementaron **Schedulers** cuya función es ejecutar dichos métodos todos los días en una hora concreta, de esta forma se garantiza que al finalizar la jornada del día, los datos de la aplicación hayan sido actualizados y preparados para el inicio la siguiente jornada.

Tabla 4.4: *Schedulers*

Scheduler	Propósito	Hora de ejecución (ES)
<i>MatchScheduler</i>	Verifica el estado de los partidos del día actual mediante <i>verifyMatchStatus</i> .	Cada 20 min (7:00 pm - 8:00 am)
<i>TeamScheduler</i>	Actualiza clasificaciones y estadísticas de todos los equipos al finalizar la jornada.	Diariamente a las 5:00 am
<i>PlayerScheduler</i>	Actualiza plantillas y estadísticas de todos los jugadores tras la jornada.	Diariamente 6:00 am

4.5.2 Resiliencia

En la subsección anterior se habló sobre la gestión de los datos provenientes de la *API* externa, sin embargo, esto conlleva a un alto grado de dependencia al estado de actividad de un servicio externo, lo que significaría que en caso de una degradación de dicho servicio, la aplicación fallaría o se encontraría en un estado no deseado por un suceso ajeno a su funcionamiento. Para evitar estos escenarios, se implementó un mecanismo de **resiliencia** que protege a la aplicación de fallos o inactividad de la *API* externa.

Dicho mecanismo de resiliencia se implementó utilizando la librería **Resilience4J** [12], una librería liviana dedicada a manejar la tolerancia a fallos en *Java*.

Esto se consigue empleando el patrón **Circuit Breaker** mediante la anotación `@CircuitBreaker`, que funciona mediante una **máquina finita de estados** conformada por 3 estados principales: *CLOSED*, *OPEN* y *HALF-OPEN*, que funcionan de la siguiente manera:

- **CLOSED:** Es el estado por defecto donde las peticiones se realizan sin problemas al servicio externo.
- **OPEN:** Se activa cuando los errores superan un determinado umbral (por ejemplo el 50%).
- **HALF_OPEN:** Tras un tiempo de espera, se permite que un determinado número de llamadas se vuelvan a realizar para verificar el estado del servicio externo. Si vuelven a fallar, se vuelve a *OPEN*, en caso contrario *CLOSED* [12].

En la **Figura 4.10** se puede visualizar el flujo de trabajo de dicha máquina estados ².

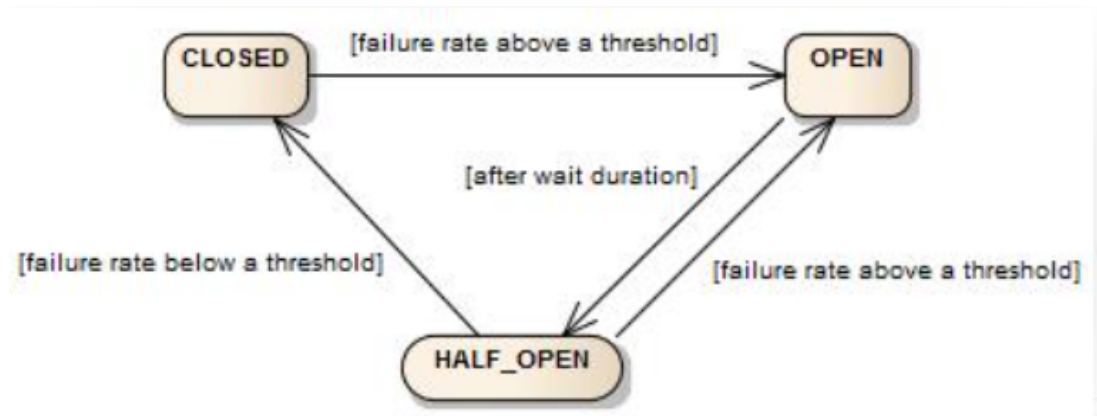


Figura 4.10: Máquina finita de estados del *@CircuitBreaker*

Su función principal consiste en interrumpir las llamadas a un servicio externo degradado para permitir su recuperación y proteger los recursos de la aplicación. Adicionalmente, este comportamiento se combinó con el módulo *@Retry*, el cual permite realizar un número definido de re-intentos tras detectar un fallo. Si no se obtiene una respuesta exitosa dentro del número de intentos configurados, el *@CircuitBreaker* registra la incidencia y se ejecuta automáticamente el *fallback-Method* correspondiente [13].

Dichas funciones suelen tener como propósito realizar una acción en caso de que el sistema externo se degrade, en el caso de esta aplicación, se optó por mostrar **logs** informando del fallo, de esta forma la ejecución no se detiene y los usuarios no se ven perjudicados.

²Disponible en: <https://resilience4j.readme.io/docs/circuitbreaker> (Accedido el 14 de mayo de 2026).

4.5.3 Asincronía

En ciertos escenarios de la aplicación, algunas tareas (como el envío de correos electrónicos o *schedulers*) pueden llegar a demorar más de lo usual. Para evitar bloqueos en el hilo de ejecución principal y mejorar la experiencia de usuario, dichas tareas se ejecutan de manera asíncrona, de esta forma, cuando un usuario solicite un código para reiniciar su contraseña (por ejemplo), pueda navegar libremente por la interfaz sin interrupciones, mientras el sistema gestiona el envío del correo en segundo plano. Para lograr esto, se habilitó la asincronía en la aplicación mediante la anotación **@EnableAsync** en el *main*, y, finalmente, se anotaron los métodos involucrados con **@Async**.

4.5.4 Patrones de Diseño

La aplicación sigue múltiples patrones de diseño que se utilizaron durante el desarrollo de la misma, a continuación, se hará una mención detallada de cada uno de ellos.

Servidor

En la [Tabla 4.5](#) se describen los patrones de diseño utilizados en el servidor.

Tabla 4.5: Patrones de diseño del servidor

Elemento	Patrón	Categoría	Uso
@Transactional	Proxy (AOP) [30]	Estructural	Extraer código repetitivo ajeno a la lógica de negocio.
@Service, @Component	<i>Singleton</i> [4]	Creacional	Garantizar una única instancia de los <i>beans</i> en el contexto.
Records	DTO [31]	Estructural	Inmutabilidad y abstracción en el transporte de datos.
@CircuitBreaker	<i>Circuit Breaker</i> [32]	Resiliencia	Evitar fallos en cascada ante la degradación de la <i>API</i> .
@Repository	Repositorio [33]	Persistencia	Abstraer la lógica de acceso a datos.
@RestController	Adaptador [34]	Estructural	Punto de entrada para interactuar con el cliente.
@Scheduler	Observable [35]	Comportamiento	Ejecución de tareas automáticas diariamente.
Mappers	Mappers [36]	Estructural	Conversión entre entidades de dominio y objetos DTO.
Factory	<i>Factory</i> [37]	Creacional	Encapsular la creación de objetos complejos.

Cuando en la **Tabla 4.5** se hace referencia a *AOP* como un patrón de diseño, se alude a la **programación orientada a aspectos**. *AOP* es un paradigma que sirve para separar las preocupaciones transversales de la lógica principal de la aplicación. En otras palabras, permite extraer código repetitivo que no forma parte de la lógica de negocio de la aplicación, y ejecutarlo automáticamente en cualquier punto donde sea necesario. Esto se puede observar en los métodos de la capa de servicio bajo la anotación *@Transactional*, que realizan las siguientes funciones:

- **Before:** El **aspecto transaccional** abre la conexión.
- **Target:** Ejecuta el método.
- **After:** El **aspecto** ejecuta el *commit* de la transacción de forma automática.

Cliente

En la **Tabla 4.6** se describen los patrones de diseño utilizados en el cliente.

Tabla 4.6: Patrones de diseño del cliente

Elemento	Patrón	Categoría	Uso
Interceptores	Cadena Responsabilidad [38]	Comportamiento	Procesar peticiones <i>HTTPS</i> de forma secuencial.
Observables (RxJS)	Observable [39]	Comportamiento	Gestión de flujos de datos asíncronos y reactividad en la interfaz.
Guards	Proxy [40]	Estructural	Controlar el acceso a las rutas según el estado de autenticación.
Servicios	<i>Singleton</i> [41]	Creacional	Mantener una única instancia para la gestión de datos y lógica compartida.
Modelos / Tipos	DTO [42]	Estructural	Definir la estructura de los objetos intercambiados con el servidor.

4.5.5 Gestión de lotes en la creación de entradas

Al configurar los sectores de un evento, el administrador define la capacidad de asientos de cada uno de ellos. Dado que el volumen por sector suele superar las 50 unidades, realizar inserciones individuales (registro a registro) generaría un cuello de botella crítico, afectando gravemente al rendimiento del sistema.

Para optimizar este proceso, se implementó una estrategia de inserción por lotes o *batches*, minimizando drásticamente la cantidad de conexiones a la base de datos. Se ha configurado un tamaño de lote de 50 registros, y se ha utilizado *JdbcTemplate* para gestionar las sentencias *SQL* de forma masiva. Gracias a esta arquitectura, si un sector requiere de la creación de 100 asientos, el sistema solo necesita realizar 2 accesos a la base de datos en lugar de 100.

En el **Código 4.1** se muestra la implementación del servicio encargado de la gestión de lotes.

```
1  @Service
2  @RequiredArgsConstructor
3  public class SeatBatchGenerationService {
4      private final JdbcTemplate jdbcTemplate;
5
6      @Transactional
7      public void batchGeneration(List<Seat> seats, Long sectorId) {
8          String query = "INSERT INTO T_SEAT (name, is_occupied, sector_id)
9              VALUES (?, ?, ?)";
10         this.jdbcTemplate.batchUpdate(query, new BatchPreparedStatementSetter()
11         {
12             @Override
13             public void setValues(PreparedStatement ps, int i) throws
14                 SQLException {
15                 Seat seat = seats.get(i);
16                 ps.setString(1, seat.getName());
17                 ps.setBoolean(2, seat.isOccupied());
18                 ps.setLong(3, sectorId);
19             }
20
21             @Override
22             public int getBatchSize() {
23                 return seats.size();
24             }
25         });
26     }
27 }
```

Código 4.1: Implementación del *SeatBatchService*

4.5.6 Sistema de mensajería interno

La aplicación cuenta con un sistema de mensajería interno entre usuarios y administradores para reportar cualquier problema. El flujo de trabajo es sencillo: un usuario crea un *ticket* de soporte que se envía a los administradores, estos acceden a ellos a través de una bandeja de entrada personal, donde pueden responderlos o cerrarlos.

Para completar esta operación, el sistema integra *Java Mail Sender* para las notificaciones, un mecanismo de **sondeo** que proporciona a los administradores actualizaciones en tiempo real sobre la cantidad de *tickets* abiertos, y un sistema de control de concurrencia. Este último componente impide que varios administradores editen o cierren el mismo *ticket* simultáneamente. Esta medida de seguridad se implementa mediante el bloqueo optimista, que utiliza el control de versiones de la base de datos para gestionar automáticamente el acceso concurrente, y bloquear los *tickets* que están siendo revisados por otro administrador.

4.5.7 Algoritmos de consulta avanzada

Gestión de eventos

Los usuarios pueden comprar entradas de forma flexible, adaptándose a la realidad de cada recinto. Como cada estadio tiene una distribución de secciones y asientos única, los precios varían según la ubicación y el tipo de evento. Para conseguir ofrecer una venta de entradas lo más realista posible, es decir, con características cambiantes, se utiliza la entidad *EventManager* que se encarga de centralizar y actualizar la configuración específica de cada partido. De esta forma se consigue que cada entrada pertenezca a una sección concreta del estadio con su respectivo precio, logrando que sean diferenciables entre sí y por tanto más realista.

Los problemas de concurrencia, como la selección de asientos mientras se está procesando un pago, se gestionan mediante un estricto control de acceso a la base de datos. El sistema gestiona las consultas simultáneas para evitar inconsistencias en los datos, y garantiza que los usuarios sean notificados si su selección ya no se encuentra disponible.

Clasificación de jugadores

La aplicación permite a los usuarios consultar la clasificación de los jugadores según cada estadística de la temporada actual (tanto para lanzadores como para jugadores de posición). Esta funcionalidad incluye una amplia variedad de filtros que permiten a los usuarios refinar su búsqueda.

Dado que la estadística seleccionada por el usuario cambia dinámicamente, y las tablas utilizadas para extraer los datos también cambian en función del tipo de jugador seleccionado, no es posible ejecutar una consulta tradicional (estática) a la base de datos debido a su comportamiento dinámico. Por lo tanto, la consulta se construyó manualmente insertando los datos correspondientes para cada caso, la estadística deseada y los filtros activos. Una vez completada la consulta, se ejecutó mediante el **EntityManager**. El *EntityManager* es la interfaz principal de *JPA* que actúa como intermediario entre la aplicación *Spring (Java)* y la base de datos.

En el **Código 4.2** se muestra la implementación del método encargado de obtener la clasificación para una métrica concreta.

```

1  @Transactional(readOnly = true)
2  @Cacheable(value = "single-stat-player-rankings", key = "{#stat, #playerType,
   #teamNames, #league, #division, #page, #size}")
3  public Page<PlayerRankingsDTO> getTopPlayersRanking(int page, int size, String
   playerType, String stat, List<String> teamNames, League league, Division
   division) {
4      String tableName = QueryBuilder.getTableName(playerType);
5      if (!QueryBuilder.isValidStat(stat, playerType)) {
6          throw new InvalidTypeException("The provided stat is not valid");
7      }
8      Pageable pageable = PageRequest.of(page, size);
9      boolean hasTeamFilter = (teamNames != null && !teamNames.isEmpty());
10     boolean hasLeagueFilter = league != null;
11     boolean hasDivisionFilter = division != null;
12
13     // Data Query Builder
14     String dataQueryString = QueryBuilder.buildDataQuery(stat, tableName,
   hasTeamFilter, hasLeagueFilter, hasDivisionFilter);
15     TypedQuery<PlayerRankingsDTO> dataQuery =
   this.entityManager.createQuery(dataQueryString,
   PlayerRankingsDTO.class);
16
17     // Count Query Builder (For pagination)
18     String countQueryString = QueryBuilder.buildCountQuery(stat, tableName,
   hasTeamFilter, hasLeagueFilter, hasDivisionFilter);
19     TypedQuery<Long> countQuery =
   this.entityManager.createQuery(countQueryString, Long.class);
20     QueryBuilder.setQueryParams(dataQuery, countQuery, teamNames, league,
   division, hasTeamFilter, hasLeagueFilter, hasDivisionFilter);
21
22     // Pagination
23     dataQuery.setFirstResult((int) pageable.getOffset());
24     dataQuery.setMaxResults(pageable.getPageSize());
25     List<PlayerRankingsDTO> content = dataQuery.getResultList();
26     Long totalElements = countQuery.getSingleResult();
27
28     return new PageImpl<>(content, pageable, totalElements);
29 }

```

Código 4.2: Obtención de la clasificación de una métrica concreta

4.5.8 Optimizaciones

Con el fin de mejorar la eficiencia de la aplicación y sus tiempos de respuesta, se aplicaron diversas herramientas y técnicas para su optimización, las más destacadas son las siguientes:

Sistema de caché

Para optimizar los tiempos de respuesta y minimizar la carga en la base de datos, se integró un sistema de caché local en memoria mediante la librería *Caffeine*. Bajo este esquema, ciertos métodos de lectura cuentan con un almacenamiento temporal que expira automáticamente tras un periodo definido. El flujo es directo: al invocar un servicio, el sistema verifica primero la caché; si los datos están presentes, se retornan de inmediato. De lo contrario, el método se ejecuta con normalidad y los resultados se indexan en la caché para futuras consultas.

Para implementar dichas cachés a cada uno de sus correspondientes métodos, se añadió la anotación `@Cacheable(name = "name")` a cada uno de ellos.

En la **Figura 4.11** se muestra un diagrama de actividad reflejando el funcionamiento del sistema de caché explicado previamente.

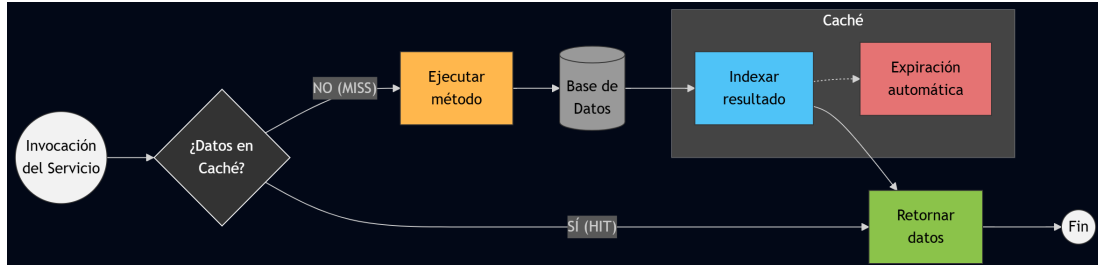


Figura 4.11: Diagrama de actividad del funcionamiento del sistema de caché local

Índices

Debido a que en ciertas entidades es habitual realizar búsquedas por atributos distintos al identificador (como el nombre), se añadieron índices para optimizar las consultas.

Dado que comparar campos de texto suele ser más costoso que los numéricos, la creación de un índice proporciona una referencia directa en la base de datos. Esto evita que a la hora de hacer la consulta, se tenga que recorrer la tabla completa fila por fila, logrando una mejora drástica en la velocidad de acceso y en la eficiencia general del sistema.

En la **Tabla 4.7** se muestran los atributos sobre los que se han definido los índices, junto a sus tablas correspondientes.

Tabla 4.7: Atributos indexados en la base de datos

Tabla	Atributo(s)
<i>Team</i>	<i>name</i>
<i>Stadium</i>	<i>name</i>
<i>Player</i>	<i>name</i>
<i>UserEntity</i>	<i>username, email</i>

EntityGraph

El *EntityGraph* es una funcionalidad de *JPA* que permite definir de manera dinámica qué asociaciones de una entidad deben cargarse en una única consulta a la base de datos a través de un ***JOIN*** [43].

Su principal utilidad es eliminar el problema de las **$N + 1$ consultas**. Este escenario ocurre al recuperar un listado de registros que tienen asociaciones con otras entidades [44]. Normalmente, el sistema ejecuta una consulta inicial para la entidad principal, y N consultas adicionales (una por cada registro) para cargar sus relaciones. El ***EntityGraph*** resuelve esto forzando un ***LEFT JOIN*** en la base de datos, permitiendo recuperar la entidad y sus asociaciones en una única transacción [45]. En pocas palabras, esta optimización transforma todas las relaciones al tipo ***EAGER***, incluyendo aquellas que sean del tipo ***LAZY***. Esto es una estrategia que debe de aplicarse de forma selectiva para evitar una sobrecarga innecesaria de datos en memoria.

En este proyecto, dicha optimización se utilizó en diversas consultas donde se pretendía obtener información relevante de tablas externas, y de aquellas donde existe un gran volumen de registros en la respuesta *SQL*.

El ejemplo más significativo fue para recuperar las métricas de rendimiento de la *API* de la aplicación, para intervalos de tiempo superiores a 1 día como bien se puede apreciar en la **Tabla 4.8**.

Tabla 4.8: Impacto de la optimización con *EntityGraph* en la latencia de consultas

Filtro	Tiempo sin <i>EntityGraph</i> (ms)	Tiempo con <i>EntityGraph</i> (ms)
1 hora	384	250
1 día	567	377
1 semana	724	243
1 mes	10190	1680

4.5.9 Mecanismo de sondeo

Su propósito es verificar el número de *tickets* de soporte abiertos en el buzón de mensajes de los administradores, realizando un sondeo cada 3 segundos únicamente si el usuario activo es un administrador. De esta forma, los administradores cuando consulten el número de *tickets* abiertos, siempre tendrán la cifra actual y correcta.

4.5.10 Problemas más destacados

Durante el proceso de desarrollo, surgieron diversos inconvenientes. Si bien la mayoría no fueron nada fuera de lo habitual en cualquier desarrollo *software*, algunos presentaron una mayor dificultad debido a su complejidad técnica o al uso de nuevas herramientas. A continuación, se mencionan dichos desafíos y las soluciones adoptadas:

- **Gestión de seguridad (*SSL*) en entorno de prueba:** Al configurar la aplicación en local con *HTTPS* y *SSL* en local, y no tener el *keystore.p12* publicado en el repositorio por motivos de seguridad, las pruebas de integración y de sistema fallaban en el flujo de integración continua. Para solventar esto, se definió un perfil de *test* donde se desactiva todo lo relacionado con la seguridad, puesto que no aportan valor crítico para su ejecución.
- **Gestión del usuario activo en pruebas de sistema:** Múltiples métodos dependen del usuario autenticado para operar. Esto generaba errores de tipo *NullPointerException* en las pruebas, ya que el objeto *Principal* resultaba nulo al no existir una sesión real. La solución consistió en crear un componente que simula la autenticación de un usuario activo, mediante las cabeceras de la petición, de esta forma, en las respectivas pruebas solo bastaba con inyectar dicha cabecera para así emular la autenticación.

- **Gestión del servicio de correos y de *Cloudinary* en las pruebas:** Al igual que sucedía con la seguridad en las pruebas de integración y de sistema, el problema se presentaba también en las pruebas que involucraban el servicio de correos *JavaMailSender*, y la base de datos externa que se usó para almacenar el servicio multimedia, *Cloudinary*. Para solventar esto, se crearon dos archivos de configuración dedicados al entorno de pruebas para simular dichos servicios externos, y evitar errores en el flujo de integración continua.
- **Gestión del tiempo en el entorno de pruebas:** El funcionamiento de los partidos está estrechamente ligado a la fecha y hora. A diferencia del entorno de ejecución local, no se puede utilizar *LocalDate.now()* puesto que comprometería la validez de las pruebas. En consecuencia, se utilizó la clase *Clock* que permite configurar una fecha y hora inmutable para todas las pruebas, asegurando que los resultados sean siempre consistentes independientemente del día de su ejecución.

4.6 Control de calidad

En esta sección se detallarán los controles de calidad realizados tanto en el cliente como en el servidor.

Para conocer las instrucciones de ejecución de las pruebas de la aplicación, consulte el Anexo D.

4.6.1 Servidor

El servidor consta de tres tipos de pruebas:

- **Unitarias:** Es el tipo de prueba más abundante, se dedica a probar la funcionalidad de cada servicio de la aplicación de manera independiente, abstrayendo cada uno de sus dependencias. Para ello, se utilizaron *mocks* que simulan el comportamiento de las dependencias de cada método del servicio. En estos *tests* se prueba no solo los casos positivos sino también si el método se comporta de manera adecuada ante situaciones erróneas.
- **Integración:** Se encarga de probar la aplicación en el entorno real de la aplicación, donde las dependencias ya no están simuladas. Su principal función es probar la integración de la capa de servicios con la de repositorios, evaluando la integración con la base de datos y la persistencia de los datos en ella. Se focaliza principalmente en métodos de escritura, puesto que son estos los que suponen una modificación en la base de datos. Dado que este

tipo de pruebas es más denso que las unitarias, y que además, en ellas ya se evalúa el comportamiento frente a errores, este tipo de prueba únicamente evalúa casos positivos.

- **Sistema o extremo a extremo:** Similares a las pruebas de integración ya que utiliza las dependencias reales y se ejecutan en el contexto real de la aplicación, se encargan de probar el funcionamiento de la *API REST* a nivel de formatos de las respuestas y códigos de respuestas adecuados.

La **Tabla 4.9** resume las estadísticas del control de calidad realizado sobre el servidor.

Tabla 4.9: Resumen estadístico del conjunto de pruebas y cobertura del servidor

Categoría	Cantidad	Porcentaje (%)
Pruebas unitarias	243	61,5
Pruebas de integración	62	15,6
Pruebas de sistema	90	22,7
Total pruebas	395	99.8 $\approx$ 100
Cobertura total (%)	81	
Líneas de código	8400	

En la **Figura 4.12** se muestra el resultado de la ejecución de las 395 pruebas en la capa del servidor.

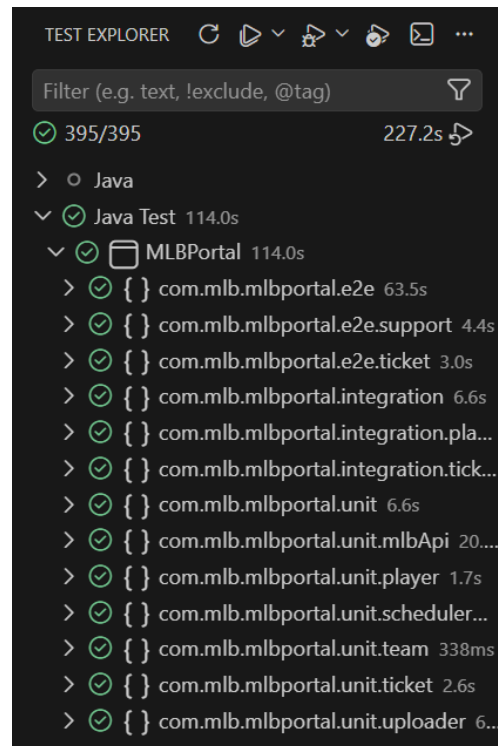


Figura 4.12: Resultado de la ejecución de las pruebas del servidor

Adicionalmente, se utilizó *Sonar* como herramienta de análisis estático de código durante todo el proceso de desarrollo. En la **Figura 4.13** se muestran los resultados finales sobre dicho análisis.

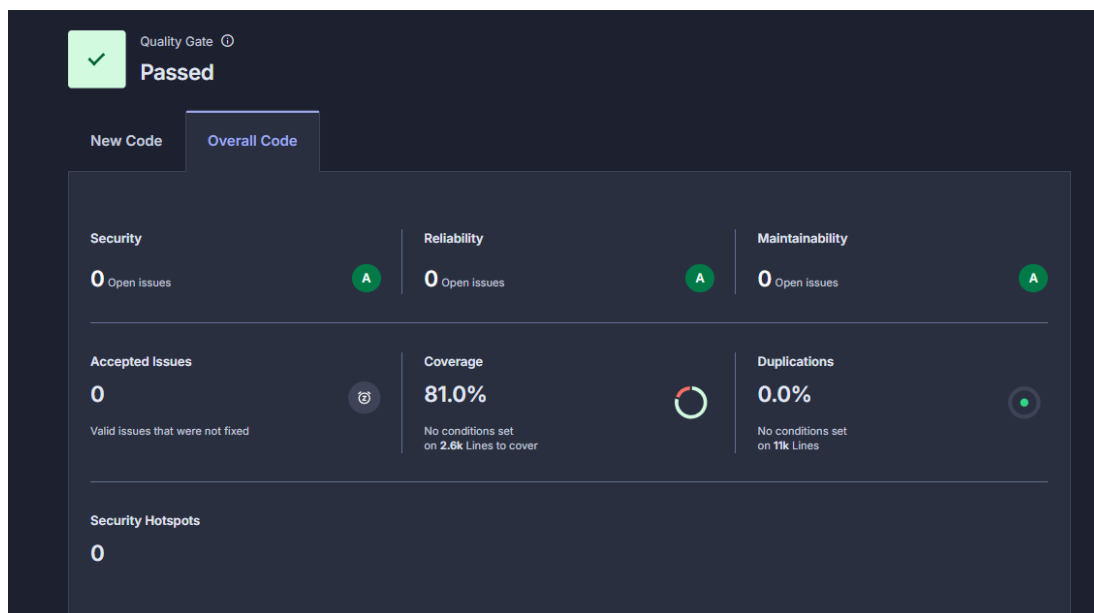


Figura 4.13: Análisis estático de código con Sonar

4.6.2 Cliente

El cliente consta de tres tipos de pruebas:

- **Unitarias:** Se encargan de probar la lógica de cada servicio y cada componente de manera independiente, es decir, todas las dependencias se simulan. Debido a su rapidez, se prueban no solo los escenarios positivos sino también los negativos.
- **Integración:** Se encargan de probar la integración del componente con el servicio. El objetivo es verificar que las llamadas a los servicios y los datos obtenidos de ellos mediante los *Observables* se realizan correctamente. Dado que este tipo de pruebas son más lentas que las unitarias, solo se prueban los escenarios positivos.
- **De interfaz de usuario aisladas:** Se encargan de evaluar la interfaz de usuario, y al igual que las pruebas de integración, solo se prueban escenarios positivos debido a lo densas que este tipo de pruebas pueden llegar a ser.

La **Tabla 4.10** resume las estadísticas del control de calidad realizado sobre el cliente.

Tabla 4.10: Resumen estadístico del conjunto de pruebas y cobertura del cliente

Categoría	Cantidad	Porcentaje (%)
Pruebas unitarias	311	62,9
Pruebas de integración	78	15,7
Pruebas de interfaz (UI)	105	21,2
Total pruebas	494	99.8 $\approx$ 100
Cobertura total (%)	80	
Líneas de código	2395	

A continuación, la **Figura 4.14** muestra un fragmento del reporte de cobertura del cliente:

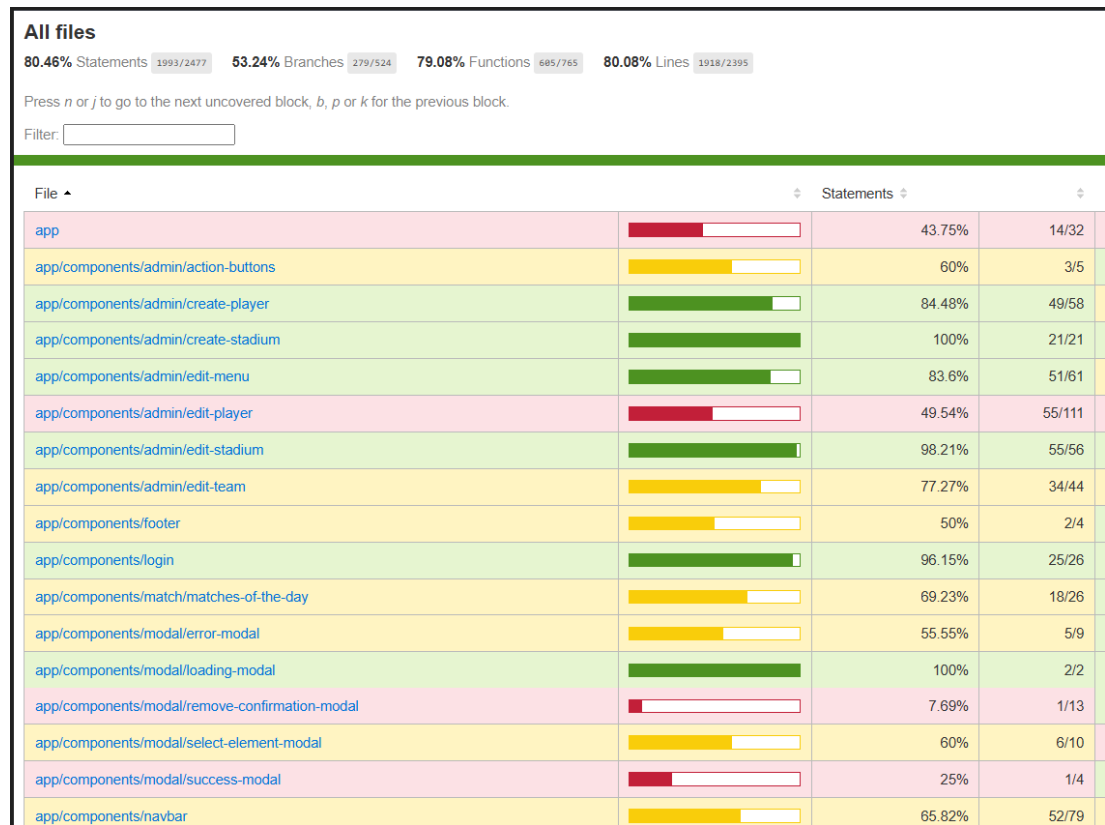


Figura 4.14: Reporte de cobertura de las pruebas en el cliente

4.7 Despliegue

En esta sección se describirá el proceso de despliegue que siguió la aplicación, comenzando por el empaquetado del cliente y servidor, y culminando con su puesta en producción en la plataforma de *Railway*.

4.7.1 Empaquetado con *Docker*

La aplicación se empaqueta en una única imagen *Docker* tanto para cliente como para servidor, dicha imagen se construye de un *Dockerfile* y es orquestado por un *docker-compose*. Para su correcto funcionamiento, utiliza la imagen **postgres:16** como base de datos, junto a un volumen de datos para lograr la persistencia de datos.

A la hora de generar las imágenes, el proyecto posee dos *Dockerfiles* para ello, uno para el entorno de desarrollo (*Dockerfile.dev*) y otro para el entorno de producción (*Dockerfile.prod*).

Esta separación se realizó debido a las diferentes condiciones de estos entornos; por ejemplo, en el entorno local, la *URL* de la fuente de datos utilizada es la del contenedor de la base de datos en la máquina local, mientras que en *Railway*, este parámetro corresponde al contenedor de la base de datos que este proporciona.

Para generar la imagen de la aplicación solo bastaría con ejecutar el siguiente comando tras estar situado en la carpeta *docker*:

```
docker build -f Dockerfile.dev -t docker.io/fonssi29/mlb-portal:tag .
```

Finalmente, se podrá ejecutar la aplicación mediante el siguiente comando:

```
docker compose up
```

Las diferentes versiones de la imagen se pueden encontrar en el repositorio de *DockerHub* que encontrará en el pie de página³.

De igual forma, el archivo *docker-compose* se puede encontrar como **Artefacto OCI** en el repositorio de *DockerHub* que encontrará en el pie de página⁴.

El despliegue de la aplicación en *Railway* se realiza mediante el archivo *Dockerfile.prod*. A partir de esta configuración, la plataforma construye automáticamente

³Repositorio con las imágenes de la aplicación: <https://hub.docker.com/r/fonssi29/mlb-portal>

⁴Repositorio con el artefacto OCI: <https://hub.docker.com/r/fonssi29/mlb-portal-compose>

el contenedor de la aplicación.

Para conocer las instrucciones de ejecución de la imagen de la aplicación en el entorno local, consulte el Anexo B.

4.7.2 Despliegue con *Railway*

La aplicación cuenta con dos ecosistemas desplegados en los servidores de *Railway*, uno de ellos perteneciente a la aplicación, que se encuentra conformado por el servicio a la base de datos de *PostgreSQL* junto a su volumen (para así garantizar la persistencia de datos), y el servicio de la aplicación que se construye a partir de su *Dockerfile*. Por otra parte, se encuentra el ecosistema de *Grafana* conformado por la base de datos *Prometheus* y el servicio de *Grafana*, ambos contruidos a partir de sus respectivos *Dockerfiles*.

La forma en la que estos servicios se comunican es a través de la conexión a la red interna que ofrece *Railway*, de esta forma, se configura la comunicación entre la base de datos y la aplicación conectando el servicio de *PostgreSQL* a la red interna del servicio de la aplicación. Análogamente sucede lo mismo entre *Prometheus* y *Grafana*, este último se conecta a la red interna de *Prometheus* para así poder obtener las métricas, mientras que *Prometheus* se conecta a la red de la aplicación para así poder recolectar los datos.

Cada vez que se incorpora un nuevo cambio en la rama *main* se realiza un despliegue, provocando así que los servicios no se encuentren disponibles por un breve periodo de tiempo. Dicha inactividad suele tener una duración aproximada de entre **20 a 30 segundos**.

La **Figura 4.15** muestra las conexiones entre los servicios dentro de la red interna de *Railway*.

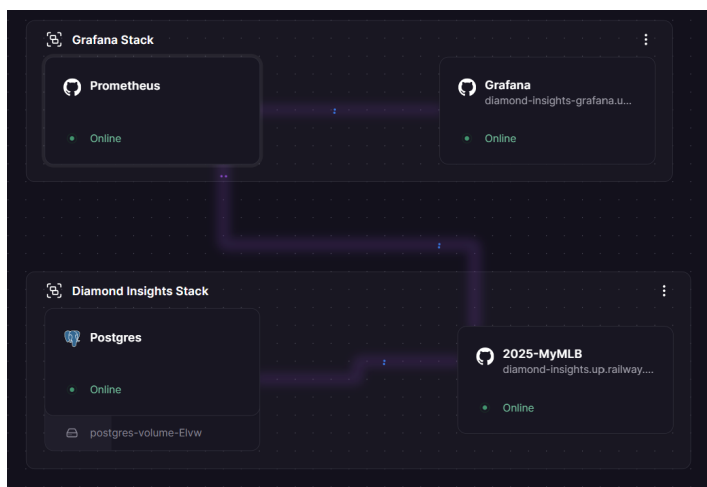


Figura 4.15: Panel de despliegue de *Railway*

4.8 Proceso de desarrollo

El proceso de desarrollo de la aplicación sigue un método **iterativo e incremental** que se rige por los principios del **Manifiesto Ágil** [46] e incorpora algunas de las mejores prácticas de la **Programación Extrema (XP)** [46] y **Kanban** [47], como iteraciones de desarrollo cortas, pruebas automatizadas e integración continua (*CI*).

4.8.1 Gestión de tareas

Para gestionar las tareas a realizar durante el proceso de desarrollo, se utilizaron *GitHub Issues* y *GitHub Projects*.

- **GitHub Issues:** Representan las tareas de cada fase de desarrollo, agrupadas por hitos que representan cada fase.
- **GitHub Project:** Proporciona un tablero *Kanban* para organizar todas las tareas.

4.8.2 Git y GitHub

Se empleó un repositorio *Git* para gestionar el control de versiones de la aplicación durante el proceso de desarrollo, utilizando **GitHub Flow** como estrategia de ramas. Dicha estrategia consiste en tener la rama principal (*main*) con una versión estable de la aplicación, mientras que todos los nuevos cambios que se van a incluir se desarrollen en ramas independientes, para finalmente, incorporarlas en la rama estable mediante *Pull Requests*, de esta forma se garantiza una completa estabilidad en la rama *main*. Las ramas utilizadas siguen la siguiente semántica:

- **main:** Rama estable de la aplicación que siempre está lista para ser desplegada.
- **feature:** En estas ramas se implementan las funcionalidades de manera aislada, incluyendo sus pruebas correspondientes para asegurar la integridad del código.
- **fix:** En estas ramas se corrigen todos los errores detectados en la versión estable de la aplicación.
- **config:** En estas ramas se realizan configuraciones a la aplicación. Se empleó sobretodo al inicio del proyecto a la hora de realizar las configuraciones iniciales.

- **documentation:** Ramas usadas para añadir documentación del proyecto durante cada fase del proceso de desarrollo.

La **Tabla 4.11** resume las estadísticas de actividad del repositorio.

Tabla 4.11: Métricas de actividad del repositorio

Categoría	Valor
Número de ramas	≈ 49
Número de <i>commits</i>	979
Ejecuciones de <i>workflows</i>	1478

Las **Figuras 4.16 y 4.17** reflejan de manera gráfica las estadísticas del proyecto a lo largo del tiempo.

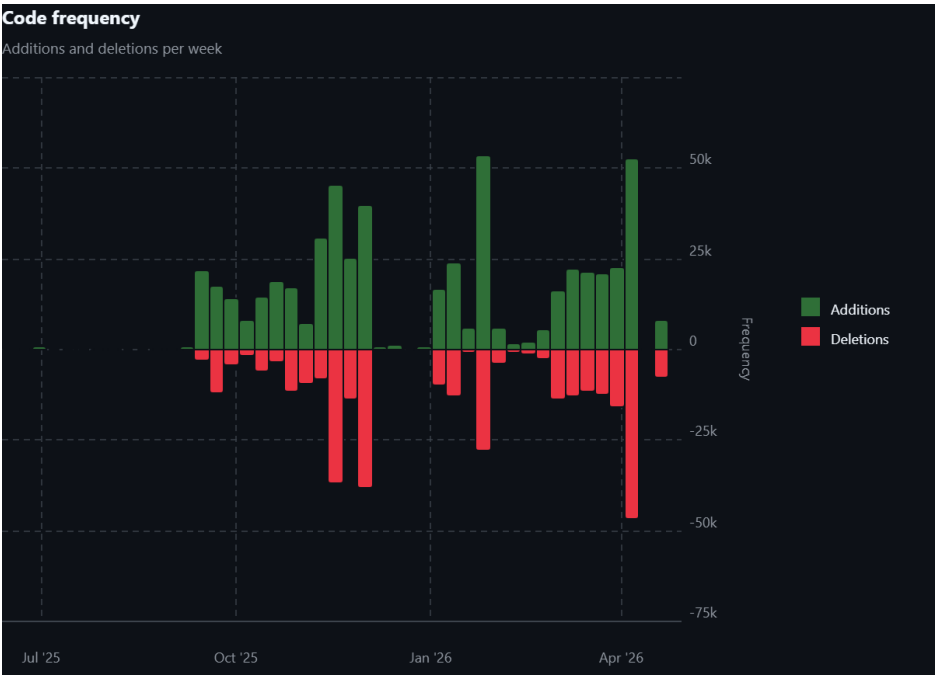


Figura 4.16: Métricas de código por *GitHub*

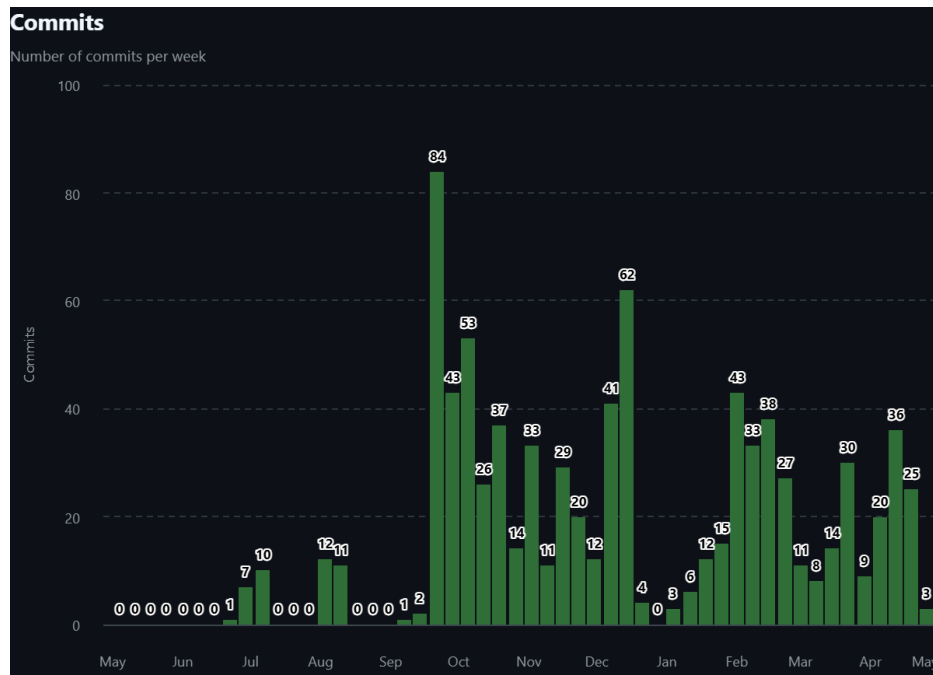


Figura 4.17: Cantidad de *commits* semanales por *GitHub*

4.8.3 Integración y entrega continua

Como primeras fases del flujo de integración continua, se tiene la fase de integración y entrega continua ambas realizadas a través de *workflows* de *GitHub Actions*, a continuación se detallará cada uno de ellos:

Basic Quality Control

Este *workflow* se ejecuta siempre que se hace un *commit* en las ramas *feature/**, *fix/** y *config/**. En el se realiza los siguiente:

- ***build-backend***: Configura y compila el servidor.
- ***backend-unit-tests***: Ejecuta las pruebas unitarias del servidor.
- ***build-frontend***: Configura y compila el cliente.
- ***frontend-unit-tests***: Ejecuta las pruebas unitarias del cliente.

Complete Quality Control

Se activa con cada incorporación a la rama *main*. Se realiza lo siguiente:

- ***build-backend***: Configura y compila el servidor.
- ***backend-unit-tests***: Ejecuta las pruebas unitarias del servidor.
- ***backend-integration-tests***: Ejecuta las pruebas de integración del servidor.
- ***backend-e2e-tests***: Ejecuta las pruebas de sistema del servidor.
- ***build-frontend***: Configura y compila el cliente.
- ***frontend-unit-tests***: Ejecuta las pruebas unitarias del cliente.
- ***frontend-integration-tests***: Ejecuta las pruebas de integración del cliente.
- ***frontend-e2e-tests***: Ejecuta las pruebas de interfaz de usuario del cliente.

Sonar Build

Permite la integración de ***SonarCloud*** con el repositorio, lo que le permite ejecutar el análisis estático de código correspondiente. Mediante este proceso, ***SonarCloud*** realiza una serie de comprobaciones de calidad, como la detección de problemas de código, errores, vulnerabilidades y métricas de cobertura, y luego vincula los resultados directamente al repositorio para facilitar el seguimiento y la mejora continua. Se activa cada vez que se abre una *Pull Request* y por cada incorporación a *main*.

Para la parte de entrega continua, se tienen los siguientes *workflows*:

CD Main

Se activa cada vez que se abre una *Pull Request* y se encarga de generar y publicar una imagen *docker* de la aplicación junto a su artefacto OCI:

- ***build-backend***: Configura y compila el servidor.
- ***build-frontend***: Configura y compila el cliente.
- ***docker-build-push***: Crea la imagen de *Docker* bajo la etiqueta ***dev*** y la publica en el repositorio de *DockerHub*.

- ***publish-compose***: Publica el *docker-compose* como un artefacto OCI bajo la etiqueta ***dev***.

CD Dispatch

Se activa manualmente desde cada rama. Las tareas son exactamente las mismas que en el flujo de trabajo anterior; la diferencia radica en que la etiqueta será `<nombre-de-rama>-<fecha-hora>-<commit>`.

CD Release

Se activa cada vez que se produce una *release*. Las tareas son las mismas que en los flujos de trabajo anteriores; sin embargo, publica tanto imágenes *Docker* como artefactos OCI utilizando la versión de lanzamiento y ***latest*** como etiquetas.

Nightly

Para complementar el flujo de *CI*, se establecieron dos flujos de trabajo programados para ejecutarse todas las noches (uno para el servidor y otro para el cliente). Estos realizan un conjunto completo de pruebas automatizadas, lo que garantiza:

- **Integridad**: Verificaciones continuas para asegurar que los cambios diarios no generen fallos en la versión estable.
- **Fiabilidad**: Garantizar la detección temprana de regresiones.

Con esto se garantiza que la aplicación tenga una versión estable en producción y que, si se produce algún error, este sea notificado y gestionado rápidamente durante el desarrollo.

4.8.4 Despliegue continuo y versionado

Con respecto al despliegue continuo, el proyecto consta con un flujo de trabajo gestionado por *Railway* que se activa con cada incorporación a la rama *main*, en el que se despliega una nueva versión de la aplicación. Durante este proceso, la aplicación se encuentra inactiva durante **20 - 30 segundos aproximadamente**.

Al finalizar cada fase del proceso de desarrollo, se publicó una nueva *release* con la versión actualizada del proyecto. En la **Tabla 4.12** se detallan las funcionalidades incluidas en cada una de ellas.

Tabla 4.12: Historial de versiones y funcionalidades desarrolladas

Versión	Fecha	Funcionalidades desarrolladas
0.1	19/12/2025	• Autenticación (inicio de sesión, registro y recuperación de contraseña). • Cierre de sesión. • Partidos del día. • Clasificaciones. • Añadir y eliminar equipos favoritos. • Modificar equipo, estadio y jugador. • Crear jugador y estadio. • Borrar jugador.
0.2	24/02/2026	• Página de perfil. • Soporte de usuarios. • Compra de entradas a los partidos. • Crear, modificar y eliminar un evento de un partido concreto. • Despliegue de la aplicación en los servidores de <i>Railway</i>.
1.0	20/04/2026	• Sección <i>Analytics</i>. • Clasificación de jugadores por cada estadística de la temporada actual. • Rediseño de la página del equipo. • Estadísticas más detalladas sobre cada equipo. • Integración y configuración de <i>Grafana</i>.

A continuación, en la **Figura 4.18** se presenta el flujo de integración continua seguido durante el proceso de desarrollo de forma visual.

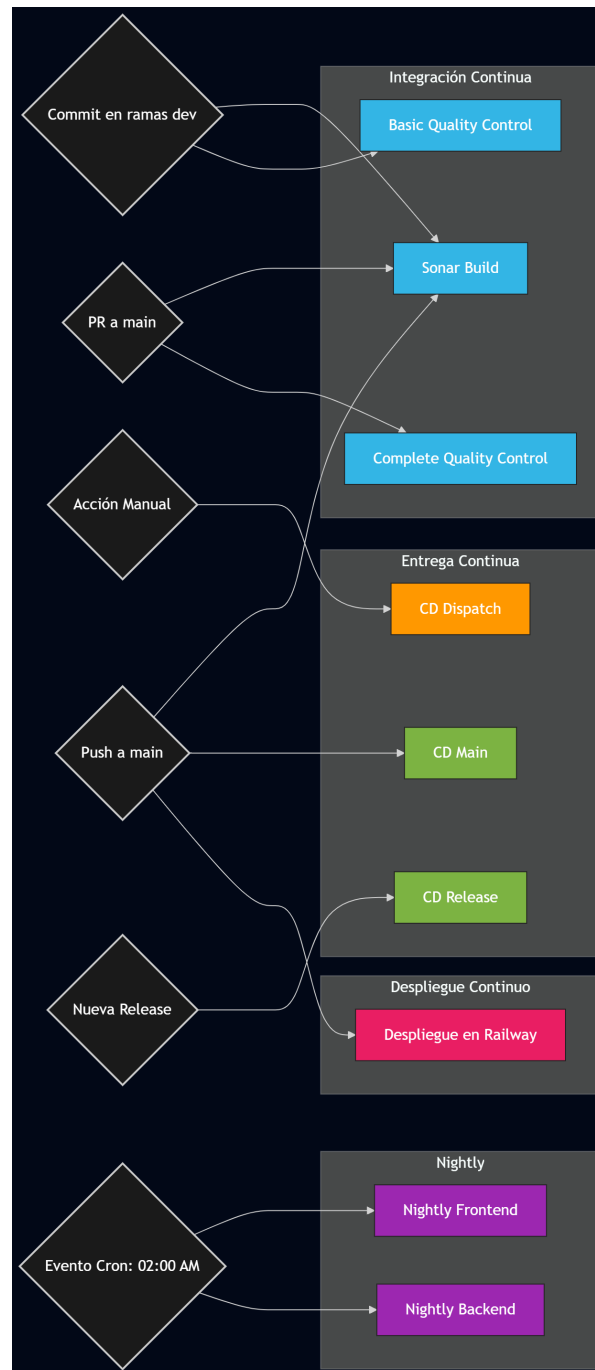


Figura 4.18: Diagrama del flujo de integración continua

5

Conclusiones y trabajos futuros

5.1 Conclusiones

Tras finalizar el desarrollo de *Diamond Insights*, se logró obtener un producto final bastante completo y funcional que ofrece desde los aspectos más básicos sobre la *Major League Baseball*, hasta aquellos más avanzados que cautivan a cualquier tipo de aficionado.

En relación con los **objetivos funcionales**, se logró crear un portal que va más allá de lo inicialmente planteado y de lo convencional. *Diamond Insights* ofrece al aficionado una verdadera personalización de los datos disponibles, logrando que sienta que el contenido ofrecido gire en torno a sus preferencias. La venta de entradas y el sistema de notificaciones mediante correo electrónico, ha transformado la aplicación de una simple web informativa, a una que ofrece servicios. Finalmente, las métricas y paneles estadísticos modelados mediante gráficos interactivos, aportan ese valor añadido que diferencia la aplicación de un simple portal informativo a uno más analítico, logrando escalar el tipo de aficionado al que va dirigido.

Desde la parte más **técnica**, la arquitectura sobre la que se construye *Diamond Insights* demuestra ser sólida, escalable y profesional, además de cumplir con las buenas prácticas de desarrollo *software*. Con *Spring Boot* y *PostgreSQL* encargados de la gestión del lado del servidor, se consiguió una gestión eficiente de los datos, flujo de ejecución, disponibilidad de la *API* y sobretodo, cubrir el sistema con una capa de seguridad para mantener su integridad. Mientras que

el uso de *Angular* y *TailwindCSS* en el lado del cliente, facilitó la creación de una interfaz de usuario moderna y amigable para todo tipo de usuario. Un aspecto clave de todo el proceso de desarrollo, fue el flujo de integración continua, qué, junto con las múltiples pruebas implementadas tanto en el cliente como en el servidor, permitió en todo momento garantizar una correcta integración, distribución y despliegue de la aplicación de manera estable y segura.

A nivel personal, este proyecto supuso un reto personal que he disfrutado en todo momento, sirviéndome para profundizar en el mundo del desarrollo de aplicaciones web, y descubrir herramientas modernas y de gran interés personal, como la implementación de un sistema de caché local para mejorar el rendimiento, o el diseño de una arquitectura resiliente a fallos externos.

Al finalizar este proyecto, siento que tengo un mayor dominio sobre el ecosistema de *Spring*, *Angular*, *DevOps*, y en general, sobre el desarrollo de *software* de calidad. Cada funcionalidad añadida nació de ideas ambiciosas de querer explorar nuevos aspectos y profundizar en los ya conocidos. Me fui estableciendo retos personales a lo largo del desarrollo, con el objetivo de fortalecer mis conocimientos, habilidades y formación, para estar realmente preparado para formar parte de proyectos profesionales y desarrollar aplicaciones de mayor calibre.

5.2 Trabajos futuros

A pesar de que el proceso de desarrollo de la aplicación ha finalizado para este trabajo, la funcionalidad de la misma está lejos de finalizar ya que existen múltiples puntos de mejora como puede ser la interfaz de usuario, y aspectos más ambiciosos con los que escalar el proyecto, tales como implementar un sistema de predicciones de partidos y desempeño de equipos y jugadores, proporcionar muchas más métricas sobre jugadores y equipos, añadir una segunda base de datos para distribuir las operaciones de lectura y escritura, entre muchas otras más. Sin embargo, la más prioritaria, sería escalar la cobertura para partidos de *playoff*, ofreciendo un seguimiento no solo de la temporada regular, sino de la última fase de la temporada donde se define el campeón de la liga.

Bibliografía

- [1] A. W. y Steve Schoger, “Tailwindcss documentation,” <https://tailwindcss.com/>, 2026.
- [2] GoalKicker.com, *AngularJS Notes for Professionals*. GoalKicker.com, 2017.
- [3] —, *Angular2+ Notes for Professionals*. GoalKicker.com, 2018.
- [4] —, *Spring Framework Notes for Professionals*. GoalKicker.com, 2017.
- [5] —, *PostgreSQL Notes for Professionals*. GoalKicker.com, 2017.
- [6] —, *TypeScript Notes for Professionals*. GoalKicker.com, 2018.
- [7] O. Foundation, “Node.js runtime,” <https://nodejs.org/>, 2026.
- [8] I. npm, “npm documentation,” <https://www.npmjs.com/>, 2026.
- [9] Google, “Angular documentation,” <https://angular.dev>, 2026.
- [10] A. S. Foundation, “Apache maven,” <https://maven.apache.org/>, 2026.
- [11] C. Ltd, “Cloudinary - image and video api platform,” <https://cloudinary.com>, 2026.
- [12] R. W. y Bohdan Mushkevych, “Resilience4j - fault tolerance library for java,” <https://resilience4j.readme.io/docs/getting-started-3>, 2026.
- [13] A. Stalin, “Resilience4j en spring boot: Resiliencia como valor estratégico en tus microservicios,” *Medium*, 2025.
- [14] E. equipo de desarrollo de PostgreSQL, “Postgresql: The world’s most advanced open source relational database,” <https://www.postgresql.org>, 2026.
- [15] I. Docker, “Docker documentation,” <https://www.docker.com/>, 2026.
- [16] R. Corp, “Railway - ship software peacefully,” <https://railway.com>, 2026.
- [17] Microsoft, “Visual studio code,” <https://code.visualstudio.com/>, 2026.
- [18] JetBrains, “Intellij idea,” <https://www.jetbrains.com/es-es/idea/>, 2026.
- [19] D. Corporation, “Dbeaver community free universal database tool,” <https://dbeaver.io/>, 2026.
- [20] S. Chacon and B. Straub, “Git - reference manual,” <https://git-scm.com/>, 2026.
- [21] S. S.A., “Sonarqube / sonarcloud documentation,” <https://www.sonarsource.com/>, 2026.
- [22] M. G. . C. KG, “Jacoco - java code coverage library,” <https://www.eclemma.org/jacoco/>, 2026.
- [23] Docker, Inc., “Docker desktop,” <https://www.docker.com/products/docker-desktop/>, 2026.
- [24] A. Wiggins, “The twelve-factor app,” <https://12factor.net/es>, 2017.

-
- [25] G. Labs, “Grafana documentation - the open observability platform,” <https://grafana.com/docs/>, 2026.
 - [26] P. Authors, “Prometheus - monitoring system and time series database,” <https://prometheus.io/docs/introduction/overview/>, 2026.
 - [27] InfluxData, “What is a time series database (tsdb)?” <https://www.influxdata.com/time-series-database/>, 2026.
 - [28] A. B. y O. Kopp y M. Falkenthal, “Survey and comparison of open source time series databases,” *University of Stuttgart*, 2017.
 - [29] B. Brazil, *Prometheus: Up Running: Infrastructure and Application Performance Monitoring*. O'Reilly Media, 2018.
 - [30] P. Software, “Spring framework - aspect oriented programming (proxy pattern),” <https://docs.spring.io/spring-framework/reference/core/aop.html>, 2026.
 - [31] M. Fowler, “Data transfer object,” <https://martinfowler.com/eaCatalog/dataTransferObject.html>, 2024.
 - [32] C. Richardson, “Pattern: Circuit breaker,” <https://microservices.io/patterns/reliability/circuit-breaker.html>, 2026.
 - [33] E. H. y Rob Mee, “Repository pattern,” <https://martinfowler.com/eaCatalog/repository.html>, 2024.
 - [34] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
 - [35] B. . Spring, “Task execution and scheduling in spring,” <https://docs.spring.io/spring-framework/reference/integration/scheduling.html>, 2026.
 - [36] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.
 - [37] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
 - [38] Google, “Angular - cloud interceptors (chain of responsibility pattern),” <https://angular.dev/guide/http/interceptors>, 2026.
 - [39] ReactiveX, “Rxjs - the observable pattern for javascript,” <https://rxjs.dev/guide/observable>, 2026.
 - [40] Google, “Angular - route guards (proxy pattern),” <https://angular.dev/guide/routing/common-tasks#guarding-routes>, 2026.
 - [41] L. G. B. Nielsen, “Spectacular angular dependency injection (singleton pattern),” *Medium - Angular in Depth*, 2023. [Online]. Available: <https://blog.angularindepth.com/>
 - [42] Microsoft, “Typescript handbook - interfaces and dtos,” <https://www.typescriptlang.org/docs/handbook/interfaces.html>, 2026.
 - [43] O. . E. Foundation, “Jakarta persistence specification - entity graphs,” <https://jakarta.ee/specifications/persistence/3.1/jakarta-persistence-spec-3.1.html#a12513>, 2026.
 - [44] E. Paraschiv, “The n+1 query problem in hibernate,” <https://www.baeldung.com/hibernate-nplusone>, 2024.
 - [45] V. Mihalcea, *High-Performance Java Persistence*. Hypersistence, 2016.
 - [46] R. S. Pressman, *Ingeniería del Software. Un Enfoque Práctico*. McGraw-Hill, 2005.
 - [47] S. McConnell, *Desarrollo y Gestión de Proyectos Informáticos*. McGraw-Hill, 1997.

BIBLIOGRAFÍA

- [48] B. Enis, “Flowbite - tailwindcss components,” <https://flowbite.com/docs/getting-started/angular/>, 2026.
- [49] L. H. y Addy Osmani, *Learning Patterns: Patterns for building powerful web apps with JavaScript and React/Angular*. Patterns.dev, 2021.
- [50] J. A. Calvo-Manzano, J. Cervera, L. Fernández, and M. Piattini, *Aplicaciones Informáticas de Gestión. Una perspectiva de Ingeniería del Software*. Ra-Ma, 2004.

Anexos



Funcionalidades detalladas

En este anexo se describirán más en detalle las funcionalidades que ofrece la aplicación, presentadas en formato de historias de usuario. Se dividen en : básicas, intermedias y avanzadas.

A.1 Funcionalidades básicas

A continuación, se detallarán las funcionalidades que fueron incluidas en la versión 0.1.

A.1.1 Registro de usuario

Como: Usuario anónimo.

Quiero: Crear una cuenta en la aplicación.

Para que: Pueda tener acceso a funcionalidades exclusivas.

Criterios

- El formulario de registro deberá solicitar:
 - Nombre de usuario.
 - Correo electrónico.

- Contraseña.
- El nombre de usuario deberá ser único.

Dependencias

- El usuario no deberá existir previamente en la base de datos de la aplicación.

Pruebas

- Verificar que el nombre de usuario introducido no exista previamente en el sistema.
- El intento de registro de un usuario ya existente deberá lanzar un mensaje de error.

A.1.2 Iniciar sesión

Como: Usuario registrado.

Quiero: Acceder a mi cuenta.

Para que: Pueda acceder a mi perfil y mis personalizaciones.

Criterios

- El formulario de *login* deberá solicitar:
 - Nombre de usuario.
 - Contraseña.
- Si las credenciales son válidas, el usuario será redirigido a la página principal.
- Si las credenciales son inválidas, se mostrará un mensaje de error.
- Una opción para recuperar la contraseña deberá ser visible.

Dependencias

- El usuario deberá existir previamente en el sistema.

Pruebas

- Verificar que las credenciales introducidas son válidas.
- El intento de inicio de sesión con un usuario que no existe o inválido deberá lanzar un mensaje de error.
- Si el usuario selecciona la opción de recuperar su contraseña, verificar que el correo electrónico que permite reiniciarla se envía correctamente.

A.1.3 Cerrar sesión

Como: Usuario registrado.

Quiero: Cerrar la sesión de mi cuenta.

Para que: Pueda finalizar mi sesión actual y cambiar a otra cuenta.

Criterios

- Los datos del usuario no deben ser eliminados durante el proceso de cierre de sesión.
- La sesión actual debe ser finalizada correctamente.
- El usuario debe ser redirigido a la página de inicio de sesión o registro.

Dependencias

- El sistema debe tener implementada la gestión de sesiones o tokens.

Pruebas

- Verificar que la cuenta del usuario aún existe en la base de datos después de cerrar la sesión.
- Verificar que el usuario es redirigido a la página de inicio de sesión o registro una vez cerrada la sesión.
- Verificar que al intentar acceder a una página restringida una vez cerrada la sesión, se redirige instantáneamente a la página de inicio de sesión o registro.

A.1.4 Añadir equipo favorito

Como: Usuario registrado.

Quiero: Seleccionar un equipo como mi "favorito".

Para que: Pueda personalizar la información que me proporciona la aplicación.

Criterios

- El equipo seleccionado no debe estar ya en la lista de favoritos.

Dependencias

- El sistema debe contar con equipos en la base de datos.
- El usuario debe haber iniciado sesión.

Pruebas

- Verificar que un usuario no puede añadir el mismo equipo más de una vez a sus favoritos.
- Verificar que los equipos favoritos se guardan correctamente y pueden ser recuperados o visualizados.
- Verificar que los partidos del día y las clasificaciones se muestran priorizando los equipos marcados como favoritos por el usuario.

A.1.5 Eliminar equipo favorito

Como: Usuario registrado.

Quiero: Eliminar un equipo de mis "favoritos".

Para que: Pueda actualizar mi lista de preferencias.

Criterios

- La cantidad actual de equipos favoritos en la lista debe ser actualizada.
- Se puede mostrar un mensaje de confirmación antes de proceder con la eliminación.

Dependencias

- El equipo debe estar presente actualmente en la lista del usuario.

Pruebas

- Verificar que un equipo puede ser eliminado de la lista de favoritos.
- Verificar que la lista se actualiza correctamente tras la eliminación.

A.1.6 Editar ajustes del perfil

Como: Usuario registrado.

Quiero: Editar los ajustes de mi cuenta.

Para que: Pueda personalizar mi configuración personal.

Criterios

- El usuario debe poder editar los siguientes campos:
 - Correo electrónico.
 - Contraseña.
 - Foto de perfil.
 - Activar/Desactivar notificaciones.
- Debe aparecer un mensaje de confirmación antes de completar la operación.
- Se debe mostrar un mensaje de éxito tras completar la operación correctamente.

Dependencias

- La nueva contraseña debe ser diferente a la contraseña actual.

Pruebas

- Verificar que los cambios en el perfil se guardan correctamente en la base de datos.
- Verificar que cualquier entrada de datos inválida lanza un mensaje de error.

- Verificar que, tras guardar, la nueva configuración se muestra correctamente al usuario.

A.1.7 Actualizar información del equipo

Como: Administrador.

Quiero: Actualizar la información editable de los equipos.

Para que: Los datos de los equipos se mantengan al día.

Criterios

- Debe existir una sección exclusiva para administradores para realizar esta operación.
- El administrador debe poder editar los siguientes campos:
 - Ciudad.
 - Información general del equipo.
 - Campeonatos.
 - Estadio.
- Una vez actualizada la información, se debe mostrar un mensaje de éxito.

Dependencias

- El equipo debe existir previamente en la base de datos.
- Si se modifica el estadio, el nuevo estadio no debe estar asociado a ningún otro equipo.

Pruebas

- Verificar que esta sección solo es visible para los administradores.
- Verificar que los campos editables de un equipo se guardan correctamente en la base de datos.
- Verificar que el mensaje de éxito se muestra correctamente.
- Verificar que la introducción de valores inválidos lanza un mensaje de error.

A.1.8 Editar información del jugador

Como: Administrador.

Quiero: Editar la información del jugador.

Para que: Los datos del jugador puedan actualizarse siempre que sea necesario.

Criterios

- Debe existir una sección exclusiva para administradores para esta operación.
- El administrador debe poder editar los siguientes campos:
 - Foto del jugador.
 - Equipo en el que juega.
 - Posición en la que juega.
 - Turnos al bate totales (AB).
 - Número de hits (H).
 - Número de carreras impulsadas (RBI).
 - Número de cuadrangulares (HR).
- El promedio de bateo (Avg) debe calcularse automáticamente a partir de AB y H.
- Una vez finalizado, se debe mostrar un mensaje de éxito.
- Cada valor numérico debe ser positivo.
- En caso de que el jugador cambie de equipo, el nuevo equipo debe ser válido (el *roster* del equipo debe tener menos de 24 jugadores).
- En caso de que el jugador cambie de posición, la nueva posición debe ser válida.

Dependencias

- El jugador debe existir previamente en la base de datos.
- El nuevo equipo debe existir en la base de datos y no tener el *roster* completo.
- La nueva posición debe ser una posición válida.

Pruebas

- Verificar que esta sección solo es visible para administradores.
- Verificar que la nueva foto del jugador se guarda correctamente en la base de datos.
- Verificar que el nuevo equipo del jugador se refleja correctamente en su perfil.
- Verificar que el nuevo equipo registra correctamente al nuevo jugador en su *roster*.
- Verificar que el total de turnos al bate (AB) se actualiza correctamente.
- Verificar que la cantidad de *hits* (H) se actualiza correctamente.
- Verificar que la cantidad de carreras impulsadas (RBI) se actualiza correctamente.
- Verificar que la cantidad de cuadrangulares (HR) se actualiza correctamente.
- Verificar que el promedio de bateo (Avg) se calcula correctamente.
- Verificar que la introducción de valores inválidos lanza un mensaje de error.
- Verificar que introducir un equipo no válido lanza un mensaje de error.
- Verificar que introducir una posición no válida lanza un mensaje de error.
- Verificar que se muestra un mensaje de éxito tras una operación correcta.

A.1.9 Editar información del estadio

Como: Administrador.

Quiero: Editar la información del estadio.

Para que: Los datos del estadio puedan actualizarse siempre que sea necesario.

Criterios

- Debe existir una sección exclusiva para administradores para esta operación.
- El administrador debe poder editar los siguientes campos:
 - Foto del estadio.
- Una vez finalizado, se debe mostrar un mensaje de éxito.

Dependencias

- El estadio debe existir previamente en la base de datos.
- El número de fotos no debe exceder de 5.

Pruebas

- Verificar que esta sección solo es visible para administradores.
- Verificar que la nueva foto del estadio se guarda correctamente en la base de datos.
- Verificar que el mensaje de éxito se muestra correctamente una vez completada la operación.

A.1.10 Crear estadio

Como: Administrador.

Quiero: Crear un nuevo estadio.

Para que: Se puedan añadir nuevos estadios al sistema.

Criterios

- Debe existir una sección exclusiva para administradores para esta operación.
- El administrador debe rellenar los siguientes campos para crear el estadio:
 - Nombre del estadio.
 - Fecha de inauguración.
- Una vez finalizado, se debe mostrar un mensaje de éxito.

Dependencias

- El estadio no debe existir previamente en la base de datos.

Pruebas

- Verificar que esta sección solo es visible para administradores.
- Verificar que las entradas de datos (*inputs*) son válidas.
- Verificar que el mensaje de éxito se muestra correctamente una vez completada la operación.

A.1.11 Crear jugador

Como: Administrador.

Quiero: Añadir un nuevo jugador al *roster* de la *MLB*.

Para que: Los nuevos jugadores que debutan en la liga sean elegibles para ser incluidos en la aplicación.

Criterios

- Debe existir una sección exclusiva para administradores para esta operación.
- El administrador debe rellenar los siguientes campos para crear al jugador:
 - Nombre del jugador.
 - Posición del jugador.
 - Equipo en el que jugará.
 - Número del jugador.
- Una vez finalizado, se debe mostrar un mensaje de éxito.

Dependencias

- El jugador no debe existir en la base de datos.
- El equipo no debe tener el *roster* completo.
- La posición debe ser una posición válida.

Pruebas

- Verificar que esta sección solo es visible para administradores.
- Verificar que las entradas de datos son válidas.
- Verificar que el equipo introducido no tiene el *roster* completo.
- Verificar que la posición introducida es válida.
- Verificar que el mensaje de éxito se muestra correctamente una vez completada la operación.

A.1.12 Eliminar jugador

Como: Administrador.

Quiero: Eliminar un jugador del *roster* de la *MLB*.

Para que: Los jugadores que se retiren o dejen la *MLB* puedan ser eliminados de la aplicación.

Criterios

- Debe existir una sección exclusiva para administradores para esta operación.
- Debe aparecer un modal de confirmación antes de eliminar al jugador.
- Una vez confirmada la operación, se debe mostrar un mensaje de éxito.

Dependencias

- El jugador debe existir en la base de datos.

Pruebas

- Verificar que esta sección solo es visible para administradores.
- Verificar que el jugador ya no existe en la base de datos.

A.2 Funcionalidades intermedias

A continuación, se detallarán las funcionalidades que fueron incluidas en la versión 0.2.

A.2.1 Eliminar cuenta

Como: Usuario registrado.

Quiero: Eliminar mi cuenta de la aplicación.

Para que: Pueda eliminar todos mis datos personales y dejar de usar los servicios de la aplicación.

Criterios

- La opción de "eliminar cuenta" está disponible desde los ajustes de la cuenta del usuario.
- El usuario debe confirmar la acción.
- Una vez confirmada, todos los datos de la cuenta se eliminarán permanentemente de la aplicación.

Dependencias

- El usuario debe haber iniciado sesión para acceder a los ajustes de su cuenta.
- Todos los datos asociados (por ejemplo, *tickets*) deben ser desvinculados de esa cuenta.

Pruebas

- Verificar que la opción de eliminar solo es visible para el usuario registrado.
- Verificar que el proceso de eliminación borra todos los datos relacionados con el usuario de la base de datos.
- Verificar que intentar iniciar sesión con credenciales eliminadas devuelve un mensaje de error.

A.2.2 Compra de entradas

Como: Usuario registrado.

Quiero: Comprar entradas.

Para que: Pueda asistir a los partidos de mis equipos favoritos.

Criterios

- El usuario debe completar el formulario de compra con entradas válidas.
- Se debe mostrar un mensaje de éxito una vez que la operación se complete satisfactoriamente.
- Si ocurre algún error, un mensaje de error debe informar al usuario sobre el problema.
- Tras la compra, de las entradas deben quedar registrados bajo su respectivo usuario (lista de entradas).
- Tras la compra, las entradas deben aparecer en *My Tickets*.
- El sistema debe gestionar cualquier problema de concurrencia que pueda ocurrir durante el proceso de compra.

Dependencias

- El número de entradas solicitados debe ser menor o igual a las disponibles.
- El número de asientos seleccionados debe ser igual al número de entradas seleccionados.

Pruebas

- Verificar que el usuario puede comprar una entrada válida.
- Verificar que intentar comprar más entradas de las disponibles resulta en un error.
- Verificar que el número de asientos seleccionados es igual al número de entradas seleccionados.
- Verificar que aparece un mensaje de éxito tras la compra.
- Verificar que las entradas se guardan en la lista de entradas del usuario tras la compra.

- Verificar que las entradas comprados aparecen en la sección ***My Tickets*** tras la compra.
- Verificar que el número total de entradas disponibles para el tipo respectivo se actualiza correctamente en la base de datos tras la compra.

A.2.3 Notificaciones por correo electrónico

Como: Usuario registrado.

Quiero: Recibir notificaciones de la aplicación a través de mi correo electrónico.

Para que: Pueda estar informado sobre eventos importantes como próximas fechas de partidos, cambios en el perfil, etc.

Criterios

- El usuario debe poder activar o desactivar las notificaciones por correo electrónico desde sus ajustes.
- Las notificaciones se envían únicamente si el usuario las ha activado.
- Las notificaciones deben cubrir eventos/acciones importantes como: confirmación de compra, cambios en cualquiera de los ajustes personales, etc.

Dependencias

- Las preferencias de notificación del usuario deben estar almacenadas y ser accesibles.

Pruebas

- Verificar que los correos electrónicos se envían correctamente cuando las notificaciones están activadas.
- Verificar que no se envían correos electrónicos cuando las notificaciones están desactivadas.
- Verificar que los cambios en las preferencias de notificación se actualizan y almacenan correctamente.

A.2.4 Contactar con soporte por correo electrónico

Como: Invitado.

Quiero: Enviar un mensaje para reportar un problema o solicitar soporte.

Para que: Pueda obtener ayuda de los administradores de la aplicación con respecto al problema.

Criterios

- El usuario puede rellenar un formulario de contacto con los siguientes campos:
 - Correo electrónico (si es un usuario invitado).
 - Asunto.
 - Cuerpo del mensaje.
- El mensaje debe enviarse a los administradores a través de correo electrónico.
- Se debe mostrar un mensaje de confirmación al usuario tras un envío exitoso.
- Los mensajes deben mostrarse en la bandeja de entrada del administrador dentro de la aplicación.

Pruebas

- Verificar que el formulario de contacto envía el mensaje correctamente.
- Verificar que el mensaje de confirmación se muestra correctamente.
- Verificar que aparecen mensajes de error cuando los campos obligatorios están vacíos o son inválidos.
- Verificar que el administrador puede responder al usuario con éxito.
- Verificar que el administrador puede cerrar cualquier *ticket* de soporte con éxito.

A.3 Funcionalidades avanzadas

A continuación, se detallarán las funcionalidades que fueron incluidas en la versión 1.0.

A.3.1 Analíticas de visibilidad de la aplicación

Como: Administrador.

Quiero: Ver las analíticas relacionadas con la tasa de crecimiento de la aplicación.

Para que: Pueda identificar el nivel de visibilidad de la aplicación, así como los nuevos usuarios y aquellos que se han perdido.

Criterios

- Debe existir una sección exclusiva para administradores para visualizar estos datos.
- Los datos se mostrarán a través de un *line chart* (datos históricos).
- El gráfico debe actualizarse dinámicamente con cada visita a la aplicación, registro y cuenta eliminada.

Pruebas

- Verificar que esta sección solo es visible para los administradores.
- Verificar que los datos se actualizan en consecuencia.

A.3.2 Analíticas de rendimiento de la *API*

Como: Administrador.

Quiero: Ver el rendimiento de la *API*.

Para que: Pueda identificar el total de solicitudes realizadas, la tasa de éxito y los *endpoints* más demandados por los usuarios.

Criterios

- Debe existir una sección exclusiva para administradores para visualizar estos datos.

- Los datos se mostrarán a través de varios gráficos:
 - Gráfico de líneas (datos históricos) para el total de solicitudes y latencia por día.
 - Gráfico de sectores (*pie chart*) para solicitudes exitosas vs. erróneas.
 - Gráfico de barras horizontales para los *endpoints* más demandados (top 5).
- Los datos deben obtenerse de ***Spring Actuator***.

Pruebas

- Verificar que esta sección solo es visible para los administradores.
- Verificar que los datos se visualizan correctamente.
- Verificar que los datos se muestran correctamente según el rango de fechas especificado.

A.3.3 Gestión de la caché

Como: Administrador.

Quiero: Implementar un sistema de caché en el servidor. **Para que:** Los tiempos de respuesta y los accesos a la base de datos se reduzcan. Además, que permita la restauración manual del estado de la caché si hay datos desactualizados.

Criterios

- Debe existir una sección exclusiva para administradores para visualizar estos datos.
- El sistema de caché se implementará con la librería *Caffeine*.

Pruebas

- Verificar que esta sección solo es visible para administradores.
- Verificar que todas las *cachés* del sistema se muestran correctamente.
- Verificar que el vaciado de una caché se realiza correctamente.

A.3.4 Estadísticas de los equipos favoritos de los usuarios

Como: Administrador.

Quiero: Ver las estadísticas sobre qué equipos son seleccionados con mayor frecuencia como "favoritos" por los usuarios.

Para que: Pueda identificar qué equipos son los más queridos según los usuarios.

Criterios

- Debe existir una sección exclusiva para administradores para visualizar estos datos.
- Los datos se mostrarán a través de un gráfico de barras horizontales.
- El gráfico debe actualizarse dinámicamente en función de las preferencias de los usuarios.

Pruebas

- Verificar que esta sección solo es visible para administradores.
- Verificar que los datos se actualizan cuando los usuarios modifican sus equipos favoritos.

A.3.5 Clasificaciones de jugadores

Como: Usuario anónimo.

Quiero: Ver las clasificaciones de jugadores para cada estadística de la temporada actual.

Para que: Pueda comparar el rendimiento de algunos jugadores con el resto de la liga o basándome en los filtros aplicados.

Criterios

- Las clasificaciones deben permitir filtros, tales como:
 - Estadística.
 - Equipos.
 - Liga.

- División.
- Las clasificaciones también pueden mostrarse mediante un gráfico de barras.

Pruebas

- Verificar que los filtros funcionan correctamente.
- Verificar que las estadísticas corresponden al tipo correcto de jugador (jugador de posición o lanzador).
- Verificar que los datos mostrados en el gráfico de barras son correctos.

A.3.6 Estadísticas de equipos

Como: Usuario anónimo.

Quiero: Ver estadísticas detalladas de un equipo determinado y compararlo con otros.

Para que: Pueda comparar el rendimiento del equipo con el resto de la liga o basándome en los filtros aplicados.

Criterios

- Debe permitir filtros, tales como:
 - Equipos.
 - Liga.
 - División.
 - Equipos con un porcentaje de victorias superior a .500 (50%).
- Los datos se mostrarán a través de varios gráficos:
 - *Line chart* (datos históricos) para la clasificación de división del equipo a lo largo de la temporada.
 - *Pie chart* para la distribución de victorias y victorias contra ciertos rivales.
 - Gráfico de barras horizontales para carreras permitidas y anotadas.

Pruebas

- Verificar que los filtros funcionan correctamente.
- Verificar que los gráficos muestran los datos correctamente.
- Verificar que los equipos rivales se muestran correctamente.
- Verificar que en los equipos rivales no haya equipos duplicados.
- Verificar que los equipos rivales no incluyan al equipo base.



Instrucciones para ejecutar la aplicación

En este anexo se describirán las instrucciones a seguir para poder ejecutar la imagen de la aplicación.

B.1 Requisitos para *Windows* y *MacOS*

Si eres usuario de *Windows* o *Mac*, necesitarás tener ***Docker Desktop*** instalado en tu sistema. En caso de que no lo tengas, puedes consultar el sitio web oficial ¹ para obtener instrucciones sobre el proceso de instalación.

Es importante que tenga en cuenta, que para poder ejecutar cualquier comando de *Docker*, necesitará tener abierto *Docker Desktop* ya que esto permite que *Docker Daemon* se encuentre activo.

¹Web oficial de *Docker*: <https://www.docker.com/products/docker-desktop/>

B.2 Requisitos para *Linux*

Si eres usuario de *Linux*, necesitarás tener **Docker** y **Docker Compose** instalados en tu sistema. En caso de que no los tengas, puedes consultar estos sitios web oficiales para obtener instrucciones sobre el proceso de instalación:

- Instalación de *Docker* ²
- Instalación de *Docker Compose* ³

B.3 Instrucciones de ejecución

Para ejecutar y desplegar la aplicación, deberá descargar el archivo **docker-compose** publicado en el repositorio de **Docker Hub**. Además, deberá tener un archivo **.env** ubicado en la carpeta de **docker**. Este archivo debe seguir la plantilla que se muestra en el **Código B.1**.

```

1  SPRING_PROFILES_ACTIVE=docker
2  SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/MLBPortal
3  SPRING_DATASOURCE_USERNAME=postgres
4  SPRING_DATASOURCE_PASSWORD=root
5  POSTGRES_PASSWORD=root
6
7  CLOUDINARY_CLOUD_NAME=${CLOUDINARY_CLOUD_NAME}
8  CLOUDINARY_API_KEY=${CLOUDINARY_API_KEY}
9  CLOUDINARY_API_SECRET=${CLOUDINARY_API_SECRET}
10
11 MAIL_USERNAME=mailusername@gmail.com
12 MAIL_PASSWORD=application_password

```

Código B.1: Plantilla del archivo **.env**

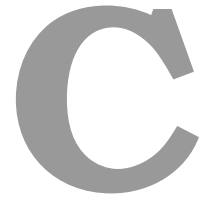
Para obtener las credenciales para utilizar el servicio de *Cloudinary* y la cuenta de correos oficial de la aplicación, deberá contactar con el administrador de la aplicación para solicitar su acceso.

Una vez creado el fichero, siga los siguientes pasos para ejecutar el contenedor de la aplicación.

1. Descargue el *docker-compose* del repositorio de *Docker Hub*: **docker compose pull**.
2. Ejecute el contenedor usando el *docker-compose* recién descargado: **docker compose up**.

²Instrucciones de descarga de *Docker* para *Linux*: <https://docs.docker.com/engine/install/ubuntu/>

³Instrucciones de descarga de *Docker Compose* para *Linux*: <https://docs.docker.com/compose/install/linux/>



Ejecución y edición de código

En este anexo se describirán las instrucciones a seguir para poder ejecutar la aplicación de manera local.

C.1 Clonación del repositorio

Para clonar el repositorio, se deberá ejecutar el siguiente comando:

```
git clone https://github.com/codeurjc-students/2025-MyMLB.git
```

Una vez clonado el repositorio, se deberá ubicar en la raíz del proyecto:

```
cd 2025-MLB
```

C.2 Requerimientos

En la **Tabla C.1**, se detallarán las mínimas versiones necesarias para poder ejecutar correctamente el proyecto:

Tabla C.1: Requisitos para la ejecución de la aplicación

Herramienta/ <i>Software</i>	Versión
<i>Java</i>	21
<i>Maven</i>	3.9
<i>Spring Boot</i>	3.5
<i>GraalVM</i>	21
Microsoft C/C++ Compiler	19.44
<i>Node</i>	22
npm	10.9
<i>PostgreSQL</i> (imagen <i>Docker</i>)	16.0
<i>Docker</i>	28
<i>Docker Compose</i>	2.34-desktop.1

C.3 Base de datos

La aplicación utiliza como base de datos principal ***PostgreSQL***. A continuación, se detalla la configuración necesaria para establecer una correcta conexión con ella en los diferentes perfiles que posee la aplicación:

C.3.1 Local - Perfil *dev*

El **Código C.1** muestra la configuración de la conexión con la base de datos en el entorno local.

```
1 spring.datasource.url=jdbc:postgresql://localhost:5432/MLBPortal
2 spring.datasource.username=postgres
3 spring.datasource.password=root
```

Código C.1: Configuración de la conexión en local con la base de datos

C.3.2 Docker - Perfil *docker*

El **Código C.2** muestra la configuración de la conexión con la base de datos en el entorno de contenedores.

```
1 spring.datasource.url=jdbc:postgresql://db:5432/MLBPortal
2 spring.datasource.username=postgres
3 spring.datasource.password=root
4 spring.datasource.driver-class-name=org.postgresql.Driver
```

Código C.2: Configuración de la conexión en *docker* con la base de datos

C.3.3 Producción - Perfil *prod*

El **Código C.3** muestra la configuración de la conexión con la base de datos en el entorno de producción (*Railway*).

```
1 # Profile used by Railway to deploy the application.
2
3 # Disable SSL and security and let Railway handles it
4 server.port=${PORT:8080}
5 server.ssl.enabled=false
6
7 # PostgreSQL DB Config (Let Railway manage the DB)
8 spring.datasource.url=${SPRING_DATASOURCE_URL}
9 spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
10 spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
11 spring.datasource.driver-class-name=org.postgresql.Driver
12
13 # JPA and Hibernate Config
14 spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
15 spring.jpa.hibernate.ddl-auto=update
16 spring.jpa.show-sql=false
```

Código C.3: Configuración de la conexión en producción con la base de datos

Todos estos perfiles utilizan un archivo de configuración común, en el que se especifican las configuraciones generales del proyecto, entre ellas, las credenciales del servicio *Cloudinary* y de la cuenta de correos

oficial de la aplicación. Dichas credenciales, se deberán inyectar mediante variables de entorno, que deberá solicitar al administrador de la aplicación.

El **Código C.4** muestra el fragmento del archivo de configuración global (*application.properties*) donde se establecen las configuraciones con el servicio de correos y *Cloudinary*.

```
1 # Spring Mail Config
2 spring.mail.host=smtp.gmail.com
3 spring.mail.port=465
4 spring.mail.username=mlbportal29@gmail.com
5 spring.mail.password=${SPRING_MAIL_PASSWORD}
6 spring.mail.properties.mail.smtp.auth=true
7 spring.mail.properties.mail.smtp.ssl.enable=true
8 spring.mail.properties.mail.smtp.ssl.trust=smtp.gmail.com
9 spring.mail.properties.mail.smtp.starttls.enable=false
10 spring.mail.properties.mail.smtp.connectiontimeout=30000
11 spring.mail.properties.mail.smtp.timeout=30000
12 spring.mail.properties.mail.smtp.writetimeout=30000
13
14 # Cloudinary
15 cloudinary.cloud_name=${CLOUDINARY_CLOUD_NAME}
16 cloudinary.api_key=${CLOUDINARY_API_KEY}
17 cloudinary.api_secret=${CLOUDINARY_API_SECRET}
```

Código C.4: Fragmento del archivo de configuración global de la aplicación

C.4 Servidor

Primero, debe situarse sobre la carpeta raíz del servidor:

```
cd backend
```

Una vez situado, podrá ejecutar el servidor de la aplicación mediante el siguiente comando:

```
mvn spring-boot:run -Dspring-boot.run.profiles=dev
```

Si todo se realizó de manera correcta, la aplicación se encontrará accesible en **<https://localhost:8443>**.

C.5 Cliente

Primero, debe situarse sobre la carpeta raíz del cliente:

```
cd frontend
```

Seguido de ello, deberá instalar todas las dependencias necesarias:

```
npm install
```

Finalmente, podrá ejecutar el cliente mediante el siguiente comando:

```
npm start
```

Si todo se realizó de manera correcta, la aplicación se encontrará accesible en **http://localhost:4200**.

Para utilizar la aplicación en conjunción (cliente y servidor), deberá primero ejecutar el servidor, y posteriormente, ejecutar el cliente mediante el siguiente comando:

```
npm run serve
```

Ahora accediendo a **http://localhost:4200**, la aplicación se podrá utilizar a su totalidad.

C.6 Generación del ejecutable

Para generar el ejecutable de la aplicación, deberá ejecutar el siguiente comando en la carpeta raíz del servidor:

mvn -Pnative native:compile

Si utiliza **Visual Studio Code**, deberá primero cumplir con los siguientes requisitos:

- *Visual Studio Community* 2022 (17.14).
- *Microsoft C/C++ Compiler* (19.44).

Dicho compilador viene incluido en la instalación de *Visual Studio Community 2022*, que la podrá realizar a través del enlace que encontrará en el pie de página ¹. En caso de que ya lo tenga instalado, ejecute el siguiente comando:

cl

Para ejecutar el comando anterior, es necesario que lo ejecute en la terminal de *x64 Native Tools Command Prompt for VS 2022*. En caso de no tenerla, por favor, siga las instrucciones que se detallarán a continuación:

1. Abra el **Instalador de Visual Studio**.
2. Haga clic sobre la sección **Modify** en *Visual Studio Community 2022*.
3. haga clic sobre la pestaña **Individual Components**.
4. Busque por **MSVC v143-VS 2022 C++ Build Tools for x64/x86 (latest)** y selecciónelo.
5. Haga clic en instalar.

¹Descarga de *Visual Studio Community 2022*: <https://visualstudio.microsoft.com/es/vs/community/>



Instrucciones de ejecución de las pruebas

En este anexo se describirán las instrucciones a seguir para la ejecución de las pruebas de la aplicación, tanto en el lado del cliente como en el servidor.

D.1 Servidor

Debe situarse sobre la carpeta raíz del servidor (*/backend*) y ejecutar el siguiente comando:

```
mvn test
```

D.2 Cliente

La ejecución en el cliente no se realiza con un único comando como en el servidor, se ejecutan con tres comandos por separado. Es importante que esté situado dentro la carpeta raíz del cliente (*/frontend*).

Para ejecutar las pruebas unitarias, tendrá que ejecutar:

```
npm run test:unit
```

Para ejecutar las pruebas de integración, tendrá que ejecutar:

npm run test:integration

Finalmente, para ejecutar las de interfaz de usuario, deberá primero ejecutar el cliente en su entorno de pruebas de sistema mediante el siguiente comando:

npm run e2e

Finalmente, ejecutar

npx cypress run

Si se opta por una ejecución sin interfaz gráfica. En caso de que se desee una ejecución con interfaz, ejecutar:

npx cypress open



Endpoints de la *API REST* de la aplicación

En este anexo se describirán los **79 endpoints** que ofrece la *API* de la aplicación, divididos por recursos, y mostrando en cada uno el formato de respuesta

E.1 Recurso Authentication

- **GET /api/v1/auth/me:** Obtiene el usuario activo.

```
{
  "username": "string",
  "roles": [
    "string"
  ]
}
```

- **POST /api/v1/auth/login:** Inicio de sesión de un usuario. Recibe como cuerpo de la petición el nombre de usuario y la contraseña asociada.

```
{
  "status": "string",
  "message": "string"
}
```

- **POST /api/v1/auth/register:** Registro de un usuario. Recibe como cuerpo de la petición el correo electrónico, nombre de usuario y contraseña.

```
{
  "status": "string",
  "message": "string"
}
```

- **POST /api/v1/auth/logout:** Cierra la sesión del usuario activo.

```
{
  "status": "string",
  "message": "string"
}
```

- **POST /api/v1/auth/refresh:** Refresca el *token* de sesión del usuario activo, para así prolongar su sesión.

```
{
  "status": "string",
  "message": "string"
}
```

- **POST /api/v1/auth/reset-password:** Establece una nueva contraseña para el usuario activo. Recibe como cuerpo de la petición el código enviado por correo electrónico y la nueva contraseña.

```
{
  "status": "string",
  "message": "string"
}
```

- **POST /api/v1/auth/forgot-password:** Envía al usuario activo el código por correo para poder reiniciar la contraseña de su cuenta.

```
{
  "status": "string",
  "message": "string"
}
```

- **DELETE /api/v1/auth:** Elimina la cuenta del usuario activo.

```
{
  "status": "string",
  "message": "string"
}
```

E.2 Recurso Users

- **GET /api/v1/users?page={page}&size={size}**: Obtiene todos los usuarios registrados de la aplicación. Solo disponible para administradores

```
{
  "content": [
    {
      "username": "string",
      "email": "string"
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **GET /api/v1/users/profile**: Obtiene el perfil del usuario activo.

```
{
  "email": "string",
  "picture": {
    "url": "string",
    "publicId": "string"
  },
  "enableNotifications": boolean
}
```

- **GET /api/v1/users/favorites/teams**: Obtiene los equipos favoritos del usuario activo

```
[
  {
    "name": "string",
    "abbreviation": "string",
    "league": "string",
    "division": "string"
  }
]
```

- **GET /api/v1/users/tickets**: Obtiene los *tickets* comprados del usuario activo.

```
[
  {
    "id": long,
    "awayTeamName": "string",
    "homeTeamName": "string",
    "stadiumName": "string",
    "price": double,
    "matchDate": "date",
    "sectorName": "string",
    "seatName": "string"
  }
]
```

- **POST /api/v1/users/picture:** Añade una nueva foto de perfil a la cuenta del usuario activo. Recibe como cuerpo de la petición la imagen que se desea subir.

```
{
  "url": "string",
  "publicId": "string"
}
```

- **POST /api/v1/users/favorites/teams/teamName:** Añade el equipo pasado por parámetro a la lista de favoritos del usuario activo.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **PATCH /api/v1/users:** Edita el perfil del usuario activo. Recibe como cuerpo de la petición los campos del perfil que se desean editar, que pueden ser: correo, contraseña y habilitar/deshabilitar notificaciones.

```
{
  "username": "string",
  "email": "string"
}
```

- **DELETE /api/users/picture:** Elimina la foto de perfil del usuario activo.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **DELETE /api/v1/users/favorites/teams/teamName:** Elimina el equipo pasado por parámetro de la lista de equipos favoritos del usuario activo.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

E.3 Recurso Teams

- **GET** /api/v1/teams?page={page}&size={size}: Obtiene todos los equipos.

```
{
  "content": [
    {
      "name": "string",
      "abbreviation": "string",
      "league": "string",
      "division": "string",
      "totalGames": integer,
      "wins": integer,
      "losses": integer,
      "pct": "string",
      "gamesBehind": double,
      "lastTen": "string"
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **GET** /api/v1/teams/teamName: Obtiene el equipo pasado por parámetro.

```

{
  "teamStats": {
    "name": "string",
    "abbreviation": "string",
    "league": "string",
    "division": "string",
    "totalGames": integer,
    "wins": integer,
    "losses": integer,
    "pct": "string",
    "gamesBehind": double,
    "lastTen": "string"
  },
  "city": "string",
  "generalInfo": "string",
  "championships": [
    integer
  ],
  "stadium": {...},
  "positionPlayers": [
    {...}
  ],
  "pitchers": [
    {...}
  ]
}

```

- **GET /api/v1/teams/teamName/rivals:** Obtiene los rivales del equipo pasado por parámetro. La respuesta es la misma que la del *endpoint* anterior.
- **GET /api/v1/teams/available?page={page}&size={size}:** Obtiene los equipos disponibles. Un equipo es disponible si su plantilla tiene menos de 26 jugadores.

```

{
  "content": [
    {
      "name": "string",
      "abbreviation": "string",
      "league": "string",
      "division": "string"
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}

```

- **GET /api/v1/teams/standings:** Devuelve la clasificación de la temporada actual.

```
{
  "name": "string",
  "abbreviation": "string",
  "league": "string",
  "division": "string",
  "totalGames": integer,
  "wins": integer,
  "losses": integer,
  "pct": "string",
  "gamesBehind": double,
  "lastTen": "string"
}
```

- **GET /api/v1/teams/{teamName}/analytics/wins-per-rival?rivalTeamNames={rivals}:** Obtiene las victorias del equipo pasado por parámetro vs los rivales seleccionados.

```
[
  {
    "rivalTeamName": "string",
    "gamesPlayed": integer,
    "wins": integer
  }
]
```

- **GET /api/v1/teams/teamName/analytics/win-distribution:** Obtiene la distribución de victorias (locales y visitante) del equipo pasado por parámetro:

```
{
  "teamName": "string",
  "homeGames": integer,
  "homeWins": integer,
  "roadGames": integer,
  "roadWins": integer,
  "homeWinPct": double,
  "roadWinPct": double
}
```

- **GET /api/v1/teams/analytics/runs-per-rival?teams={teams}:** Obtiene las carreras anotadas y permitidas de los equipos pasados por parámetro.

```
[
  {
    "teamName": "string",
    "runsScored": integer,
    "runsAllowed": integer
  }
]
```

- **GET /api/v1/teams/analytics/historic-ranking?teams={teams}&dateFrom={date}**: Obtiene la posición de los equipos pasados por parámetro de sus respectivas divisiones a lo largo de la temporada.

```
{
  "teamName": [
    "teamName": "string",
    "matchDate": date,
    "rank": integer
  ]
}
```

- **POST /api/v1/teams/sync**: Actualiza manualmente las clasificaciones.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **POST /api/v1/teams/hydrate**: Obtiene la posición divisional de cada equipo desde el primer día de temporada. Únicamente se ejecuta si hay un vacío de datos para alguna jornada, y es exclusivo para administradores.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **PATCH /api/v1/teams/teamName**: Actualiza la información del equipo. Exclusivo para administradores y únicamente permite modificar la ciudad, información general, campeonatos y estadio. Dicha información vendrá en el cuerpo de la petición.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

E.4 Recurso Players

- **GET** /api/v1/players?playerType={type}&playerName={name}&teamName={teamName}&page={page}&size={size}: Devuelve todos los jugadores según los filtros activos, en caso de no haber ninguno, devuelve todos.

```
{
  "content": [
    {
      "name": "string",
      "playerNumber": integer,
      "teamName": "string",
      "position": "string",
      "games": integer,
      "wins": integer,
      "losses": integer,
      "era": double,
      "inningsPitched": integer,
      "totalStrikeouts": integer,
      "walks": integer,
      "hitsAllowed": integer,
      "runsAllowed": integer,
      "whip": double,
      "saves": integer,
      "saveOpportunities": integer,
      "picture": {
        "url": "string",
        "publicId": "string"
      },
      "apiDataSource": boolean
    },
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **GET** /api/v1/players/rankings?playerType={type}&stat={stat}: Obtiene la clasificación de jugadores de la métrica pasada por parámetro.

```
{
  "content": [
    {
      "name": "string",
      "picture": "string",
      "stat": number
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **GET /api/v1/players/rankings/all?playerType=type:** Obtiene todas las clasificaciones de las métricas disponibles para el tipo de jugador pasado por parámetro (*pitcher* o posición). El formato de respuesta es un mapa, donde cada *ranking* viene agrupado por la métrica correspondiente.

```
{
  "statName": [
    {
      "name": "string",
      "picture": "string",
      "stat": integer
    }
  ]
}
```

- **POST /api/v1/players/playerName/pictures:** Actualiza la foto de perfil del jugador. Exclusivo para administradores.

```
{
  "url": "string",
  "publicId": "string"
}
```

- **POST /api/v1/players/sync:** Actualiza las clasificaciones de jugadores. Exclusivo para administradores.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **POST /api/v1/players/position-player:** Crea un nuevo jugador de posición. En el cuerpo de la petición se pasarán los campos necesarios para la

creación que son el nombre, número, nombre del equipo y posición. Exclusivo para administradores.

```
{
  "name": "string",
  "playerNumber": integer,
  "teamName": "string",
  "position": "string",
  "atBats": integer,
  "walks": integer,
  "hits": integer,
  "doubles": integer,
  "triples": integer,
  "homeRuns": integer,
  "rbis": integer,
  "average": double,
  "obp": double,
  "ops": double,
  "slugging": double,
  "picture": {
    "url": "string",
    "publicId": "string"
  },
  "apiDataSource": false
}
```

- **POST /api/v1/players/pitcher:** Crea un nuevo *pitcher*. En el cuerpo de la petición se pasarán los campos necesarios para la creación que son el nombre, número, nombre del equipo y posición. Exclusivo para administradores.

```
{
  "name": "string",
  "playerNumber": integer,
  "teamName": "string",
  "position": "string",
  "atBats": integer,
  "walks": integer,
  "hits": integer,
  "doubles": integer,
  "triples": integer,
  "homeRuns": integer,
  "rbis": integer,
  "average": double,
  "obp": double,
  "ops": double,
  "slugging": double,
  "picture": {
    "url": "string",
    "publicId": "string"
  },
  "apiDataSource": false
}
```

- **PATCH /api/v1/players/position-players/playerName:** Actualiza los datos de un jugador de posición. En el cuerpo de la petición se envían los campos que se desean actualizar, que pueden ser el nombre del equipo, el número, la posición, las métricas de la temporada, y si los datos se obtienen de la API oficial de MLB o se van a actualizar manualmente. Exclusivo para administradores.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **PATCH /api/v1/players/pitchers/playerName:** Actualiza los datos de un *pitcher*. En el cuerpo de la petición se envían los campos que se desean actualizar, que pueden ser el nombre del equipo, el número, la posición, las métricas de la temporada, y si los datos se obtienen de la API oficial de la MLB o se van a actualizar manualmente. Exclusivo para administradores.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

E.5 Recurso Stadium

- **GET** `/api/v1/stadiums?page={page}&size={size}`: Obtiene todos los estadios.

```
{
  "content": [
    {
      "name": "string",
      "openingDate": integer,
      "teamName": "string",
      "pictures": [
        {
          "url": "string",
          "publicId": "string"
        }
      ],
      "pictureMap": {
        "url": "string",
        "publicId": "string"
      }
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **GET** `/api/v1/stadiums/stadiumName`: Obtiene el estadio por nombre.

```
{
  "name": "string",
  "openingDate": integer,
  "teamName": "string",
  "pictures": [
    {
      "url": "string",
      "publicId": "string"
    }
  ],
  "pictureMap": {
    "url": "string",
    "publicId": "string"
  }
}
```

- **GET** `/api/v1/stadiums/available?page={page}&size={size}`: Obtiene todos los estadios disponibles. Un estadio está disponible si no tiene

ningún equipo asociado.

```
{
  "content": [
    {
      "name": "string",
      "openingDate": integer,
      "teamName": "string",
      "pictures": [
        {
          "url": "string",
          "publicId": "string"
        }
      ],
      "pictureMap": {
        "url": "string",
        "publicId": "string"
      }
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **GET /api/v1/stadiums/stadiumName/pictures:** Obtiene las fotos del estadio pasado por parámetro.

```
[
  {
    "url": "string",
    "publicId": "string"
  }
]
```

- **POST /api/v1/stadiums:** Crea un nuevo estadio. Los campos necesarios para la creación se pasarán en el cuerpo de la petición y serán el nombre y la fecha de apertura. Exclusivo para administradores.

```
{
  "name": "string",
  "openingDate": integer,
  "pictures": [
    {
      "url": "string",
      "publicId": "string"
    }
  ]
}
```

- **POST /api/v1/stadiums/stadiumName/pictures:** Añade una nueva foto para el estadio pasado por parámetro. Únicamente se podrá añadir una nueva foto si el total de fotos del estadio es menor a cinco. Exclusivo para administradores.

```
{
  "url": "string",
  "publicId": "string"
}
```

- **POST /api/v1/stadiums/stadiumName/pictures/map:** Añade/modifica la foto del mapa del estadio. Exclusivo para administradores.

```
{
  "url": "string",
  "publicId": "string"
}
```

- **DELETE /api/v1/stadiums/{stadiumName}/pictures?publicId={publicId}:** Elimina una foto del estadio pasado por parámetro. Para borrar dicha foto, será necesario pasar por parámetro el id de dicha foto.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

E.6 Recurso Matches

- **GET** `/api/v1/matches/matchId`: Obtiene el partido según su ID.

```
{
  "id": long,
  "homeTeam": {
    "name": "string",
    "abbreviation": "string",
    "league": "string",
    "division": "EAST"
  },
  "awayTeam": {
    "name": "string",
    "abbreviation": "string",
    "league": "string",
    "division": "EAST"
  },
  "homeScore": integer,
  "awayScore": integer,
  "date": date,
  "status": "string",
  "stadiumName": "string"
}
```

- **GET** `/api/v1/matches/today?page={page}&size={size}`: Obtiene los partidos del día actual.

```
{
  "content": [
    {
      "id": long,
      "homeTeam": {
        "name": "string",
        "abbreviation": "string",
        "league": "string",
        "division": "EAST"
      },
      "awayTeam": {
        "name": "string",
        "abbreviation": "string",
        "league": "string",
        "division": "EAST"
      },
      "homeScore": integer,
      "awayScore": integer,
      "date": date,
      "status": "string",
      "stadiumName": "string"
    },
    {
      "page": {
        "size": integer,
        "number": integer,
        "totalElements": integer,
        "totalPages": integer
      }
    }
  ]
}
```

- **GET** /api/v1/matches/team/{teamName}?location={location}&page={page}&size={size}: Obtiene los partidos de local o visita (dependiendo del parámetro *location*) del equipo pasado por parámetro.

```
{
  "content": [
    {
      "id": long,
      "homeTeam": {
        "name": "string",
        "abbreviation": "string",
        "league": "string",
        "division": "EAST"
      },
      "awayTeam": {
        "name": "string",
        "abbreviation": "string",
        "league": "string",
        "division": "EAST"
      },
      "homeScore": integer,
      "awayScore": integer,
      "date": date,
      "status": "sttring",
      "stadiumName": "string"
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **GET** /api/v1/matches/team/{teamName}/calendar?year={year}&month={month}: Devuelve los partidos del equipo pasado por parámetro, para la fecha dada.

```
{
  [
    "id": long,
    "homeTeam": {
      "name": "string",
      "abbreviation": "string",
      "league": "string",
      "division": "EAST"
    },
    "awayTeam": {
      "name": "string",
      "abbreviation": "string",
      "league": "string",
      "division": "EAST"
    },
    "homeScore": integer,
    "awayScore": integer,
    "date": date,
    "status": "string",
    "stadiumName": "string"
  ]
}
```

- **POST** `/api/v1/matches/sync?scope={scope}`: Refresca los partidos dependiendo del *scope* pasado. Si es *today* refresca los partidos del día actual, si es *season* refresca todos los de la temporada actual. Este último es exclusivo para administradores.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

E.7 Recurso Events

- **GET /api/v1/events?page={page}&size={size}**: Devuelve todos los eventos disponibles. Exclusivo para usuarios registrados.

```
{
  "content": [
    {
      "id": long,
      "awayTeamName": "string",
      "homeTeamName": "string",
      "homeTeamAbbreviation": "string",
      "stadiumName": "string",
      "date": date,
      "pictureMap": {
        "url": "string",
        "publicId": "string"
      },
      "sectors": [
        {
          "id": long,
          "sectorId": long,
          "sectorName": "string",
          "price": double,
          "availability": integer,
          "totalCapacity": integer
        }
      ]
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **GET /api/v1/events/eventId**: Obtiene el evento por ID. Exclusivo para usuarios registrados.

```
{
  "id": long,
  "awayTeamName": "string",
  "homeTeamName": "string",
  "homeTeamAbbreviation": "string",
  "stadiumName": "string",
  "date": date,
  "pictureMap": {
    "url": "string",
    "publicId": "string"
  },
  "sectors": [
    {
      id": long,
      "sectorId": long,
      "sectorName": "string",
      "price": double,
      "availability": integer,
      "totalCapacity": integer
    }
  ]
}
```

- **GET** /api/v1/events/match/{matchId}: Obtiene el evento por ID del partido. Exclusivo para usuarios registrados.

```
{
  "id": long,
  "awayTeamName": "string",
  "homeTeamName": "string",
  "homeTeamAbbreviation": "string",
  "stadiumName": "string",
  "date": date,
  "pictureMap": {
    "url": "string",
    "publicId": "string"
  },
  "sectors": [
    {
      id": long,
      "sectorId": long,
      "sectorName": "string",
      "price": double,
      "availability": integer,
      "totalCapacity": integer
    }
  ]
}
```

- **GET** /api/v1/events/{eventId}/sectors: Obtiene los sectores del evento pasado por parámetro. Exclusivo para usuarios registrados.

```
{
  "id": long,
  "sectorId": long,
  "sectorName": "string",
  "price": double,
  "availability": integer,
  "totalCapacity": integer
}
```

- **GET /api/v1/events/eventId/sector/sectorId:** Obtiene los asientos disponibles de un evento y sector en concreto. Exclusivo para usuarios registrados.

```
{
  "id": long,
  "name": "string"
}
```

- **GET /api/v1/events/{eventId}/tickets?page={page}&size={size}:** Obtiene los *tickets* vendidos de un evento concreto. Exclusivo para administradores.

```
{
  "content": [
    {
      "id": long,
      "awayTeamName": "string",
      "homeTeamName": "string",
      "stadiumName": "string",
      "price": double,
      "matchDate": date,
      "sectorName": "string",
      "seatName": "string"
    }
  ],
  "page": {
    "size": integer,
    "number": integer,
    "totalElements": integer,
    "totalPages": integer
  }
}
```

- **POST /api/v1/events:** Crea un nuevo evento, para ello es necesario pasar en el cuerpo de la petición el número de sectores, junto al nombre, precio y número de asientos que tendrá cada sector. Exclusivo para administradores.

```
{
  "id": long,
  "awayTeamName": "string",
  "homeTeamName": "string",
  "homeTeamAbbreviation": "string",
  "stadiumName": "string",
  "date": date,
  "pictureMap": {
    "url": "string",
    "publicId": "string"
  },
  "sectors": [
    {
      "id": long,
      "sectorId": long,
      "sectorName": "string",
      "price": double,
      "availability": 1073741824,
      "totalCapacity": 1073741824
    }
  ]
}
```

- **PUT /api/v1/events:** Permite modificar un evento, para ello será necesario pasar por el cuerpo de la petición los nuevos precios de los sectores de dicho evento. Exclusivo para administradores.

```
{
  "id": long,
  "awayTeamName": "string",
  "homeTeamName": "string",
  "homeTeamAbbreviation": "string",
  "stadiumName": "string",
  "date": date,
  "pictureMap": {
    "url": "string",
    "publicId": "string"
  },
  "sectors": [
    {
      "id": long,
      "sectorId": long,
      "sectorName": "string",
      "price": double,
      "availability": 1073741824,
      "totalCapacity": 1073741824
    }
  ]
}
```

DELETE /api/v1/events/eventId: Elimina el evento pasado por parámetro. Exclusivo para administradores.

```
{
  "id": long,
  "awayTeamName": "string",
  "homeTeamName": "string",
  "homeTeamAbbreviation": "string",
  "stadiumName": "string",
  "date": date,
  "pictureMap": {
    "url": "string",
    "publicId": "string"
  },
  "sectors": [
    {
      "id": long,
      "sectorId": long,
      "sectorName": "string",
      "price": double,
      "availability": 1073741824,
      "totalCapacity": 1073741824
    }
  ]
}
```

E.8 Recurso Tickets

- **GET /api/v1/tickets/ticketId:** Obtiene un ticket por su ID. Exclusivo para administradores.

```
{
  "id": long,
  "awayTeamName": "string",
  "homeTeamName": "string",
  "stadiumName": "string",
  "price": double,
  "matchDate": date,
  "sectorName": "string",
  "seatName": "string"
}
```

- **GET /api/v1/tickets/ticketId/download:** Descarga el pdf del *ticket* pasado por parámetro, y devuelve el pdf en formato *byte[]*. Exclusivo para usuarios registrados.
- **POST /api/v1/tickets:** Compra de *tickets* para un evento concreto. Toda la información relacionada con la compra se pasará en el cuerpo de la petición, donde estarán la cantidad de asientos, el precio total, número de la tarjeta, dueño, CVV, y fecha de caducidad. Exclusivo para usuarios registrados.

```
{
  "id": long,
  "awayTeamName": "string",
  "homeTeamName": "string",
  "stadiumName": "string",
  "price": double,
  "matchDate": date,
  "sectorName": "string",
  "seatName": "string"
}
```

E.9 Recurso Support

- **POST /api/v1/support:** El usuario crea un *ticket* de soporte. En el cuerpo de la petición se enviará el *email* del usuario, el asunto y cuerpo del mensaje.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

E.10 Recurso Admin Support

- **GET /api/v1/admin/support/tickets:** Devuelve todos los *tickets* de soporte que se encuentren abiertos, es decir, sin resolver.

```
[
  {
    "id": long,
    "subject": "string",
    "userEmail": "string",
    "status": "string",
    "creationDate": date
  }
]
```

- **GET /api/v1/admin/support/tickets/ticketId/conversation:** Obtiene la conversación entre el administrador y el usuario del *ticket* de soporte pasado por parámetro.

```
[
  {
    "id": long,
    "senderEmail": "string",
    "body": "string",
    "fromUser": boolean,
    "creationDate": date
  }
]
```

- **POST /api/v1/admin/support/tickets/ticketId/reply:** Permite al administrador responder al usuario que envió el *ticket* pasado por parámetro. En el cuerpo de la petición se enviará, el *email* del administrador y el cuerpo del mensaje.

```
{
  "id": long,
  "senderEmail": "string",
  "body": "string",
  "fromUser": false,
  "creationDate": date
}
```

- **POST** `/api/v1/admin/support/tickets/ticketId/close`: Permite al administrador cerrar un *ticket*.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

E.11 Recurso Analytics

- **GET /api/v1/analytics/visibility?dateFrom={dateFrom}&dateTo={dateTo}:** Obtiene las estadísticas de visibilidad de la aplicación dentro del intervalo de tiempo especificado. Exclusivo para administradores.

```
[
  {
    "date": date,
    "visualizations": integer,
    "newUsers": integer,
    "deletedUsers": integer
  }
]
```

- **GET /api/v1/analytics/api-performance?dateRange={range}:** Obtiene las estadísticas sobre el rendimiento de la API dentro del rango de tiempo especificado, que puede ser: 1 hora, 1 día, 1 semana o 1 mes. Exclusivo para administradores.

```
[
  {
    "timeStamp": date,
    "totalRequests": integer,
    "totalErrors": integer,
    "totalSuccesses": integer,
    "averageResponseTime": double,
    "mostDemandedEndpoints": [
      {
        "uri": "string",
        "count": integer
      }
    ]
  }
]
```

- **GET /api/v1/analytics/fav-teams:** Obtiene las estadísticas sobre los equipos que más son añadidos como favoritos por los usuarios. Exclusivo para administradores.

```
[
  {"teamName": "count"}
]
```

- **POST /api/v1/analytics/visibility/visualizations:** Aumenta las visualizaciones de la aplicación.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **POST /api/v1/analytics/visibility/registrations:** Aumenta los usuarios registrados de la aplicación.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **POST /api/v1/analytics/visibility/losses:** Aumenta los usuarios perdidos de la aplicación.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

E.12 Recurso Cache

- **GET /api/v1/cache:** Obtiene todas las *caches* que tiene la aplicación. Exclusivo para administradores.

```
[
  {"name": "string"}
]
```

- **DELETE /api/v1/cache/name:** Restaura la *caché* pasada por parámetro. Exclusivo para administradores.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

- **DELETE /api/v1/cache:** Restaura todas las *caches* de la aplicación. Exclusivo para administradores.

```
{
  "status": "string",
  "message": "string",
  "error": "string"
}
```

F

Extensión arquitectura del servidor

En este anexo se expandirá la sección **4.3.5 Arquitectura del servidor**, con los diagramas de los módulos restantes que conforman el servidor.

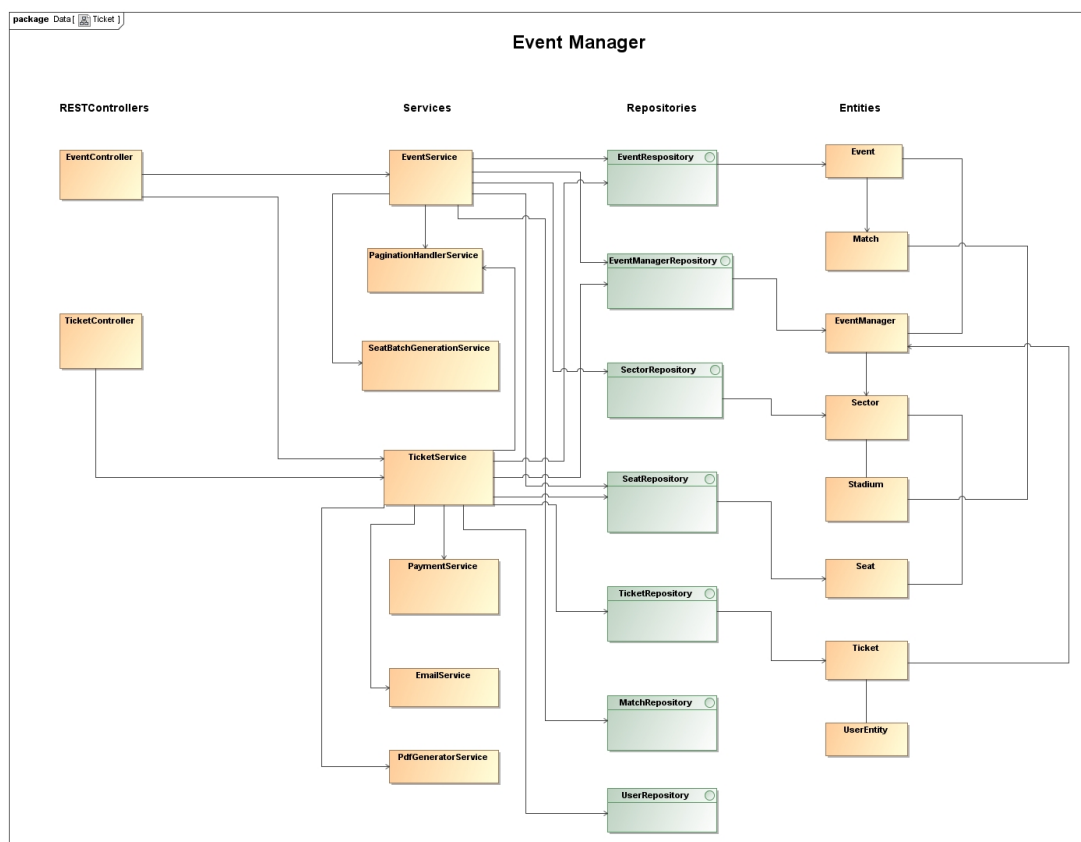


Figura F.1: Diagrama de clases del módulo de gestión de entradas en el servidor

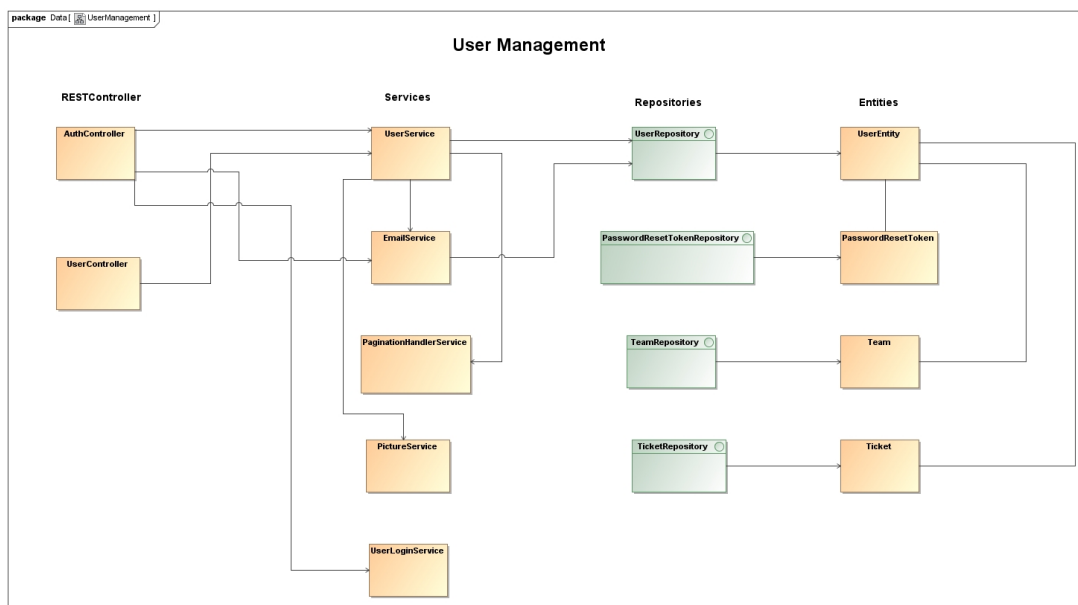


Figura F.2: Diagrama de clases del módulo de gestión de usuarios

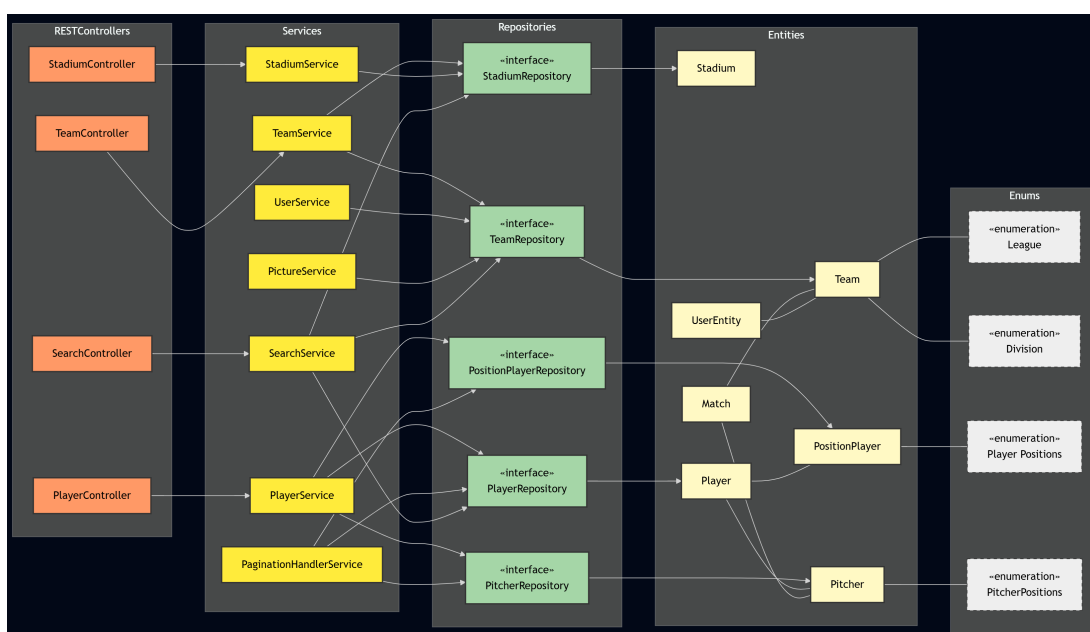


Figura F.3: Diagrama de clases del módulo de modificación de entidades en el servidor

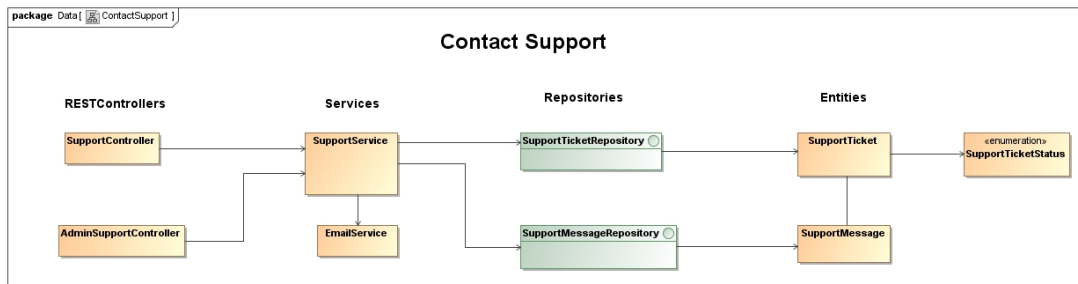


Figura F.4: Diagrama de clases del módulo de soporte en el servidor

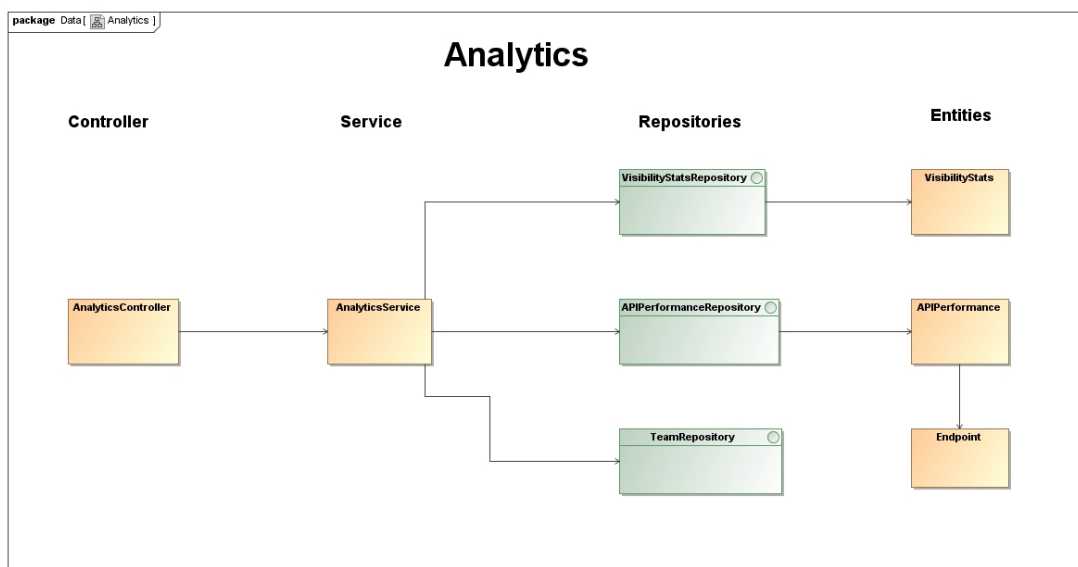


Figura F.5: Diagrama de clases del módulo de analíticas en el servidor

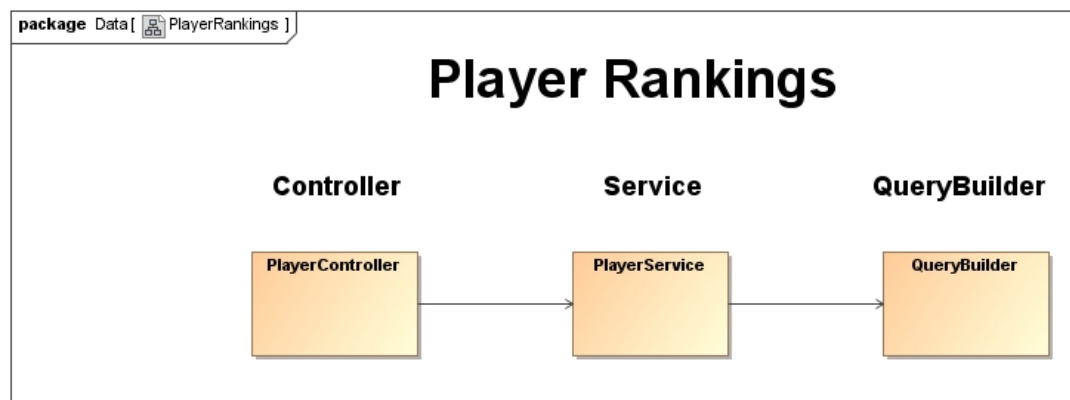


Figura F.6: Diagrama de clases del módulo de clasificación de jugadores en el servidor

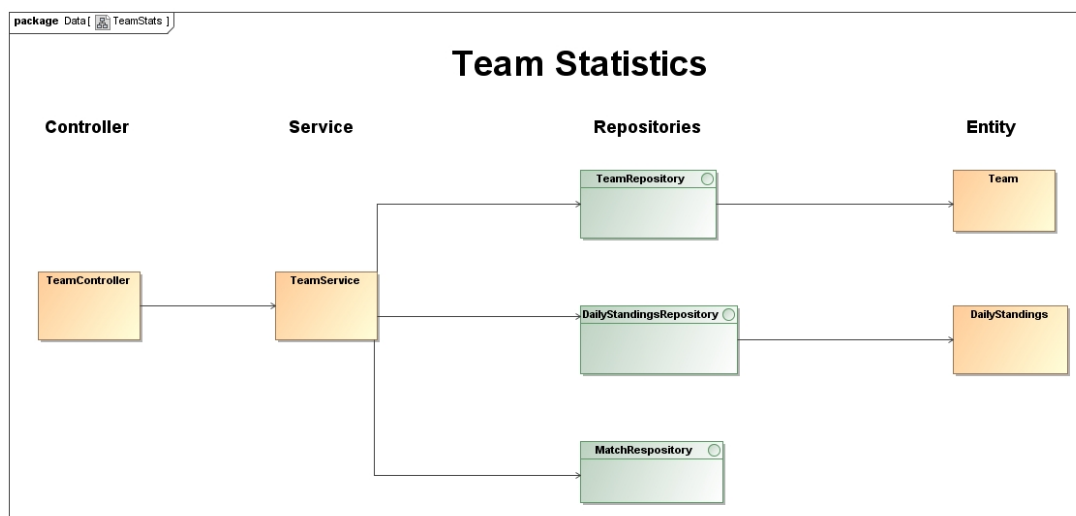


Figura F.7: Diagrama de clases del módulo de las estadísticas del equipo en el servidor



Extensión arquitectura del cliente

En este anexo se expandirá la sección **4.3.6 Arquitectura del cliente**, con los diagramas de los módulos restantes que conforman el cliente.

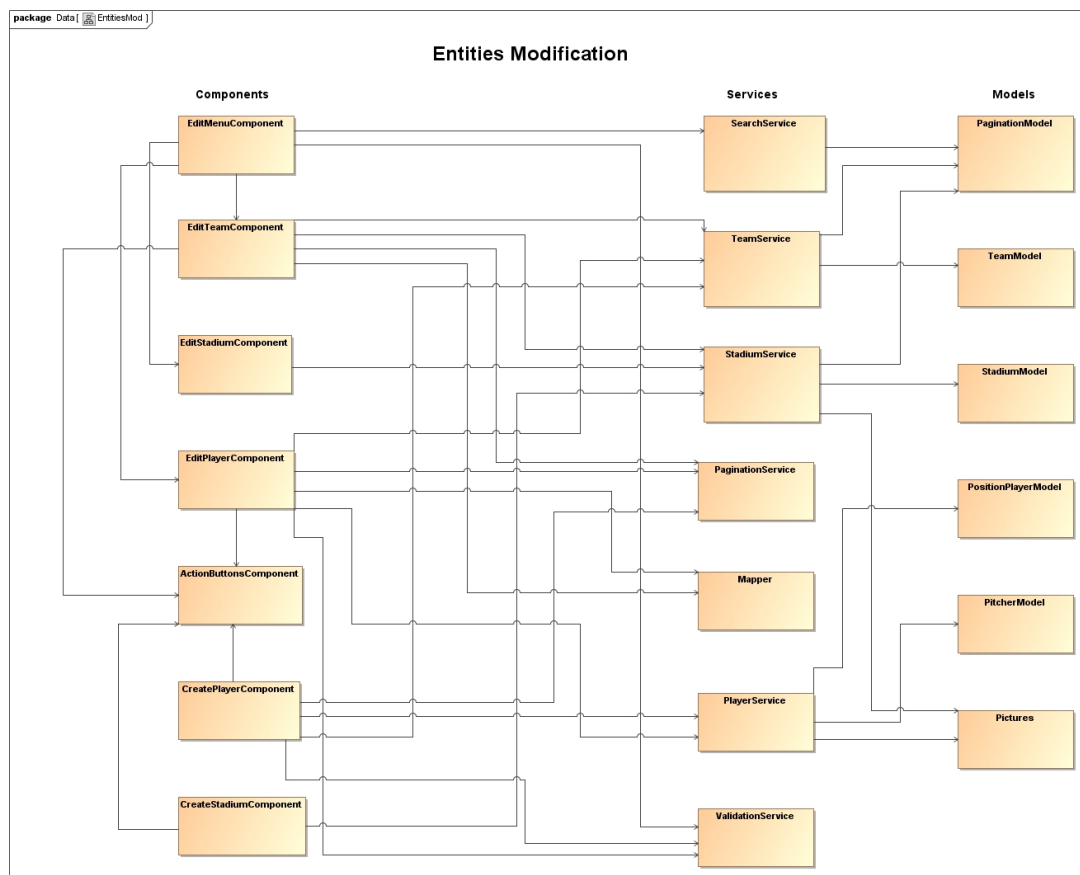


Figura G.1: Diagrama de clases del módulo de modificación de entidades en el cliente

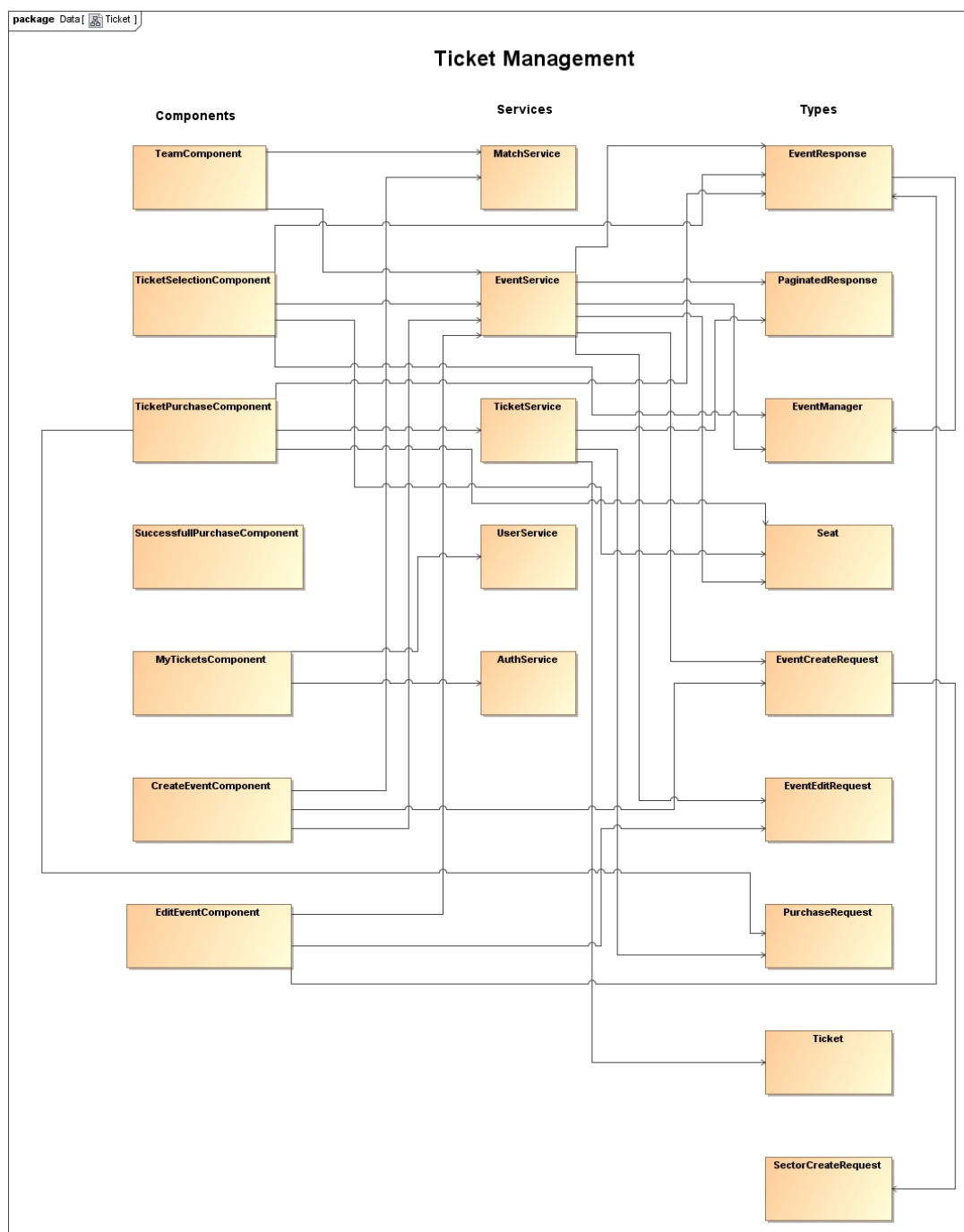


Figura G.2: Diagrama de clases del módulo de gestión de entradas en el cliente

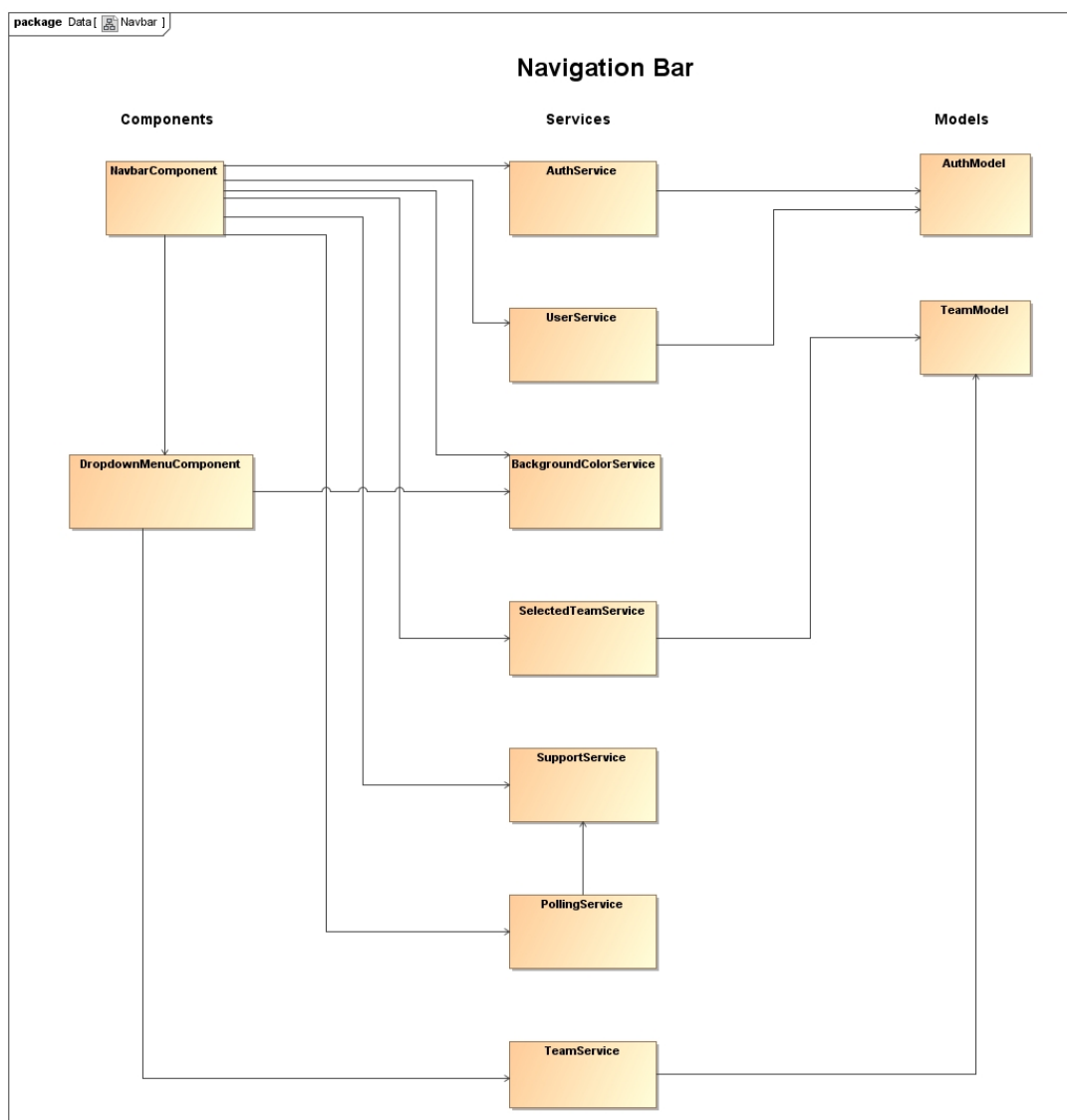


Figura G.3: Diagrama de clases del módulo de la barra de navegación

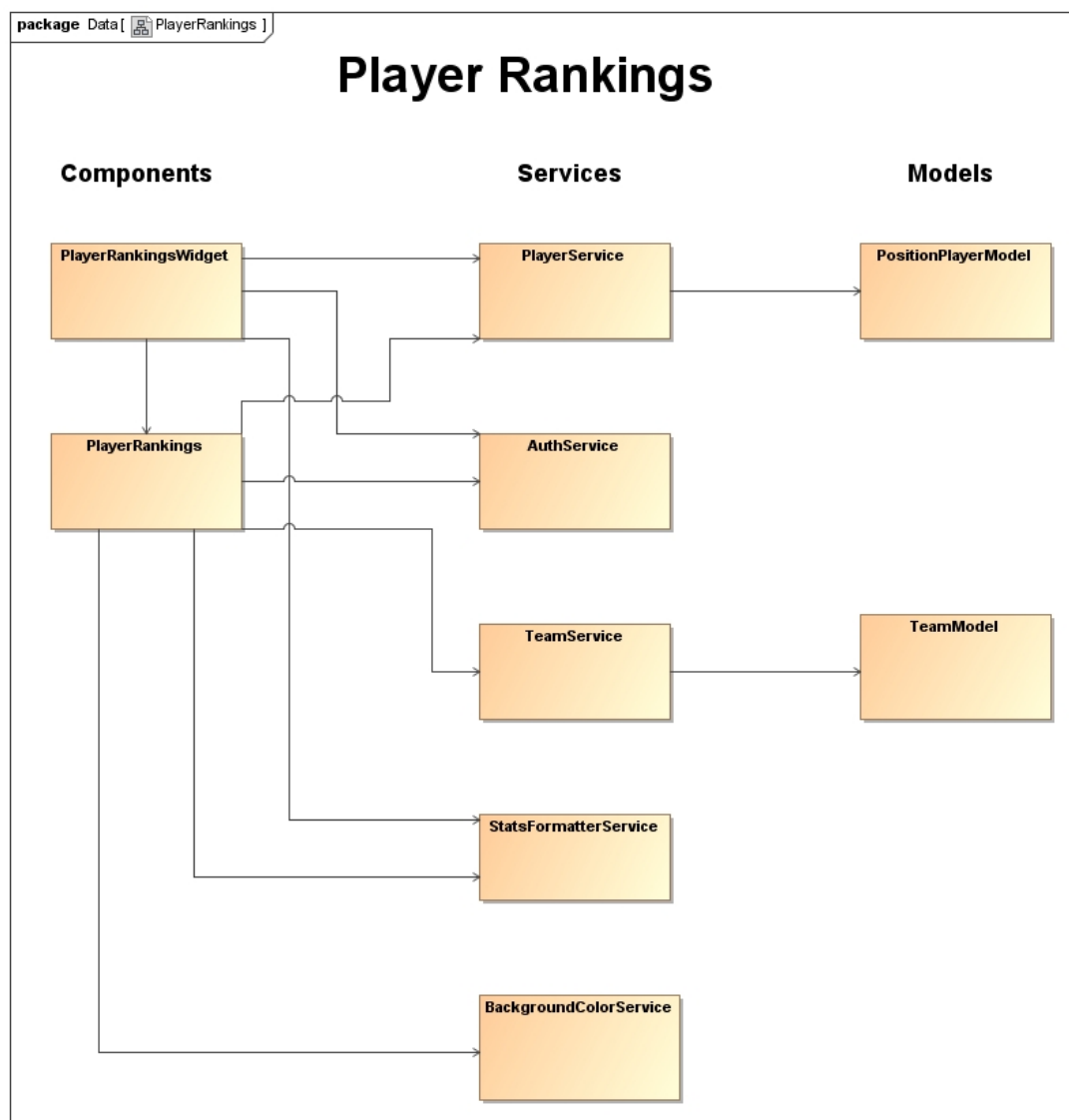


Figura G.4: Diagrama de clases del módulo de clasificación de jugadores en el cliente

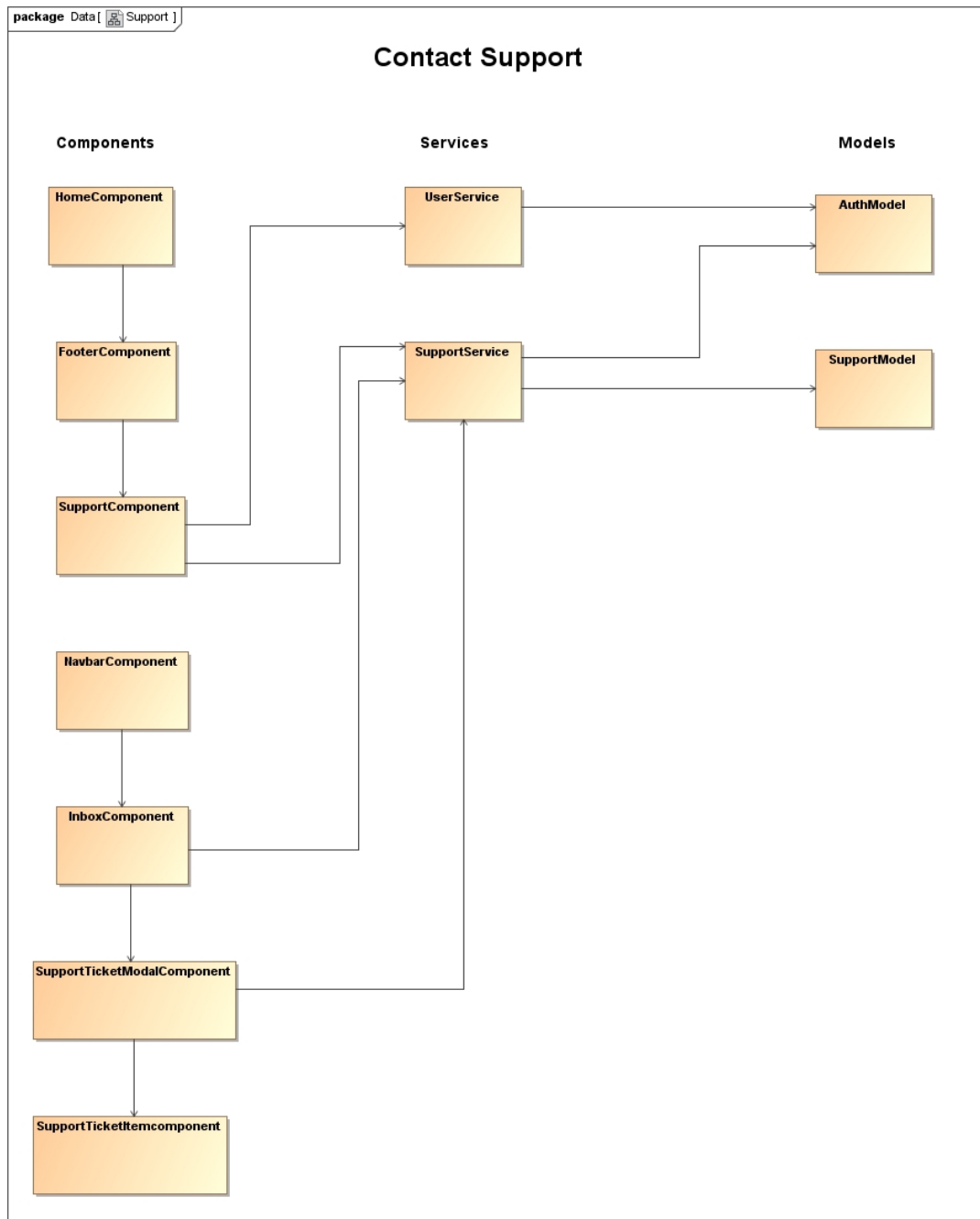


Figura G.5: Diagrama de clases del módulo de soporte en el cliente

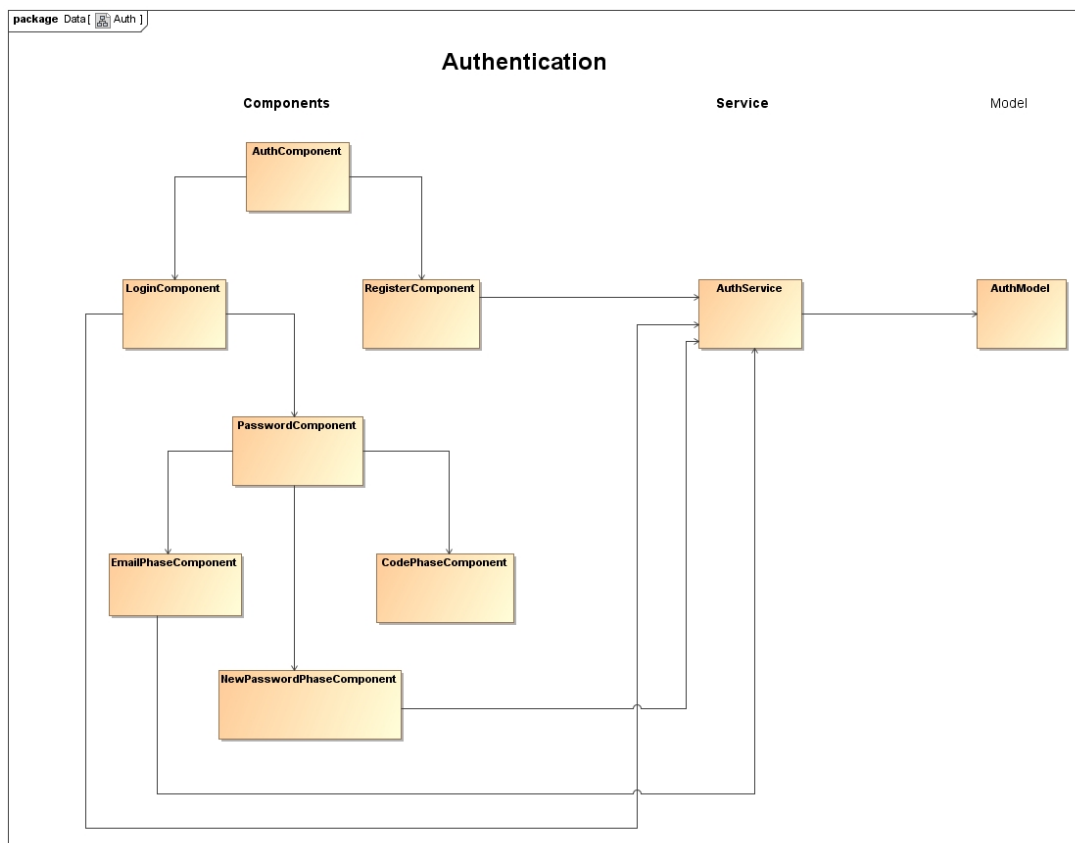


Figura G.6: Diagrama de clases del módulo de autenticación

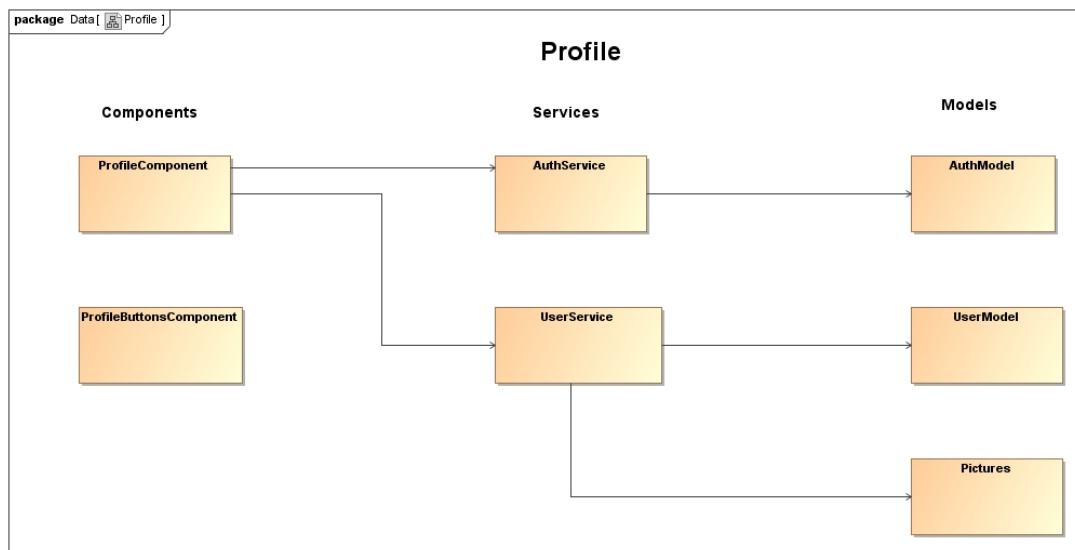


Figura G.7: Diagrama de clases del módulo de perfil de usuario

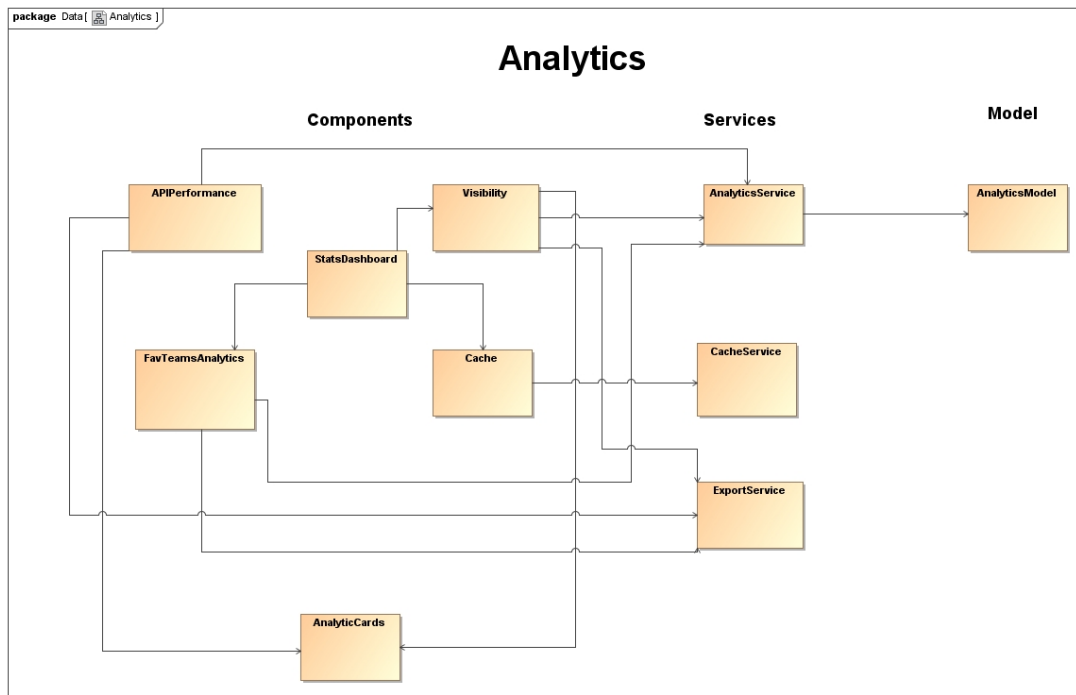


Figura G.8: Diagrama de clases del módulo de analíticas en el cliente

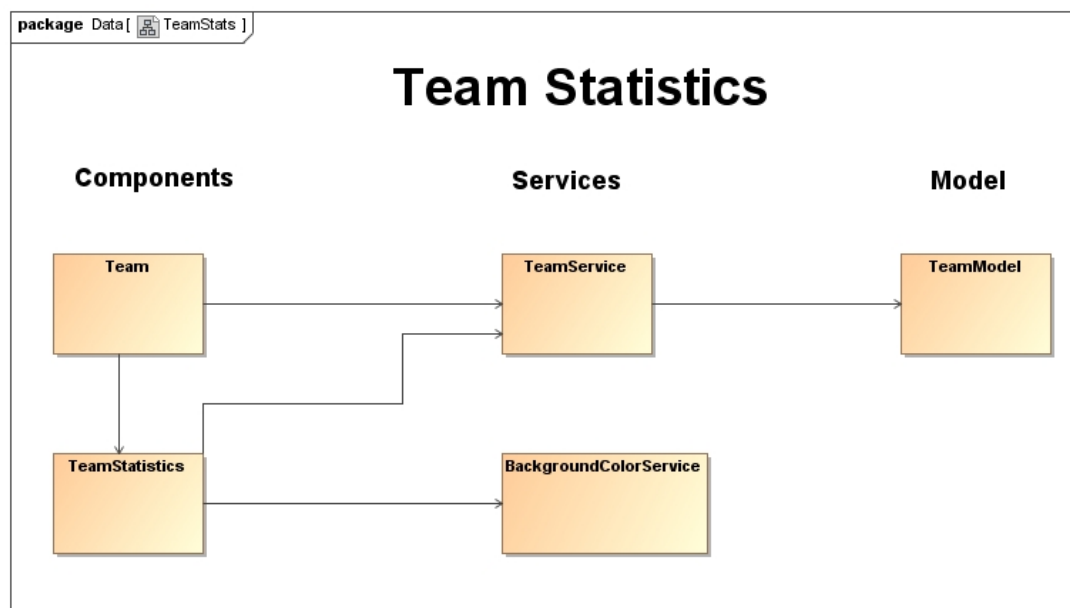
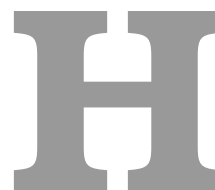


Figura G.9: Diagrama de clases del módulo de las estadísticas del equipo en el cliente



Implementación *MatchImportService*

En este anexo, se mostrará el código de los métodos principales del *MatchImportService* (Código H.1 y H.2) junto a su correspondiente *Scheduler MatchScheduler* (Código H.3).

```
1  @Transactional
2  @CircuitBreaker(name = "mlbApi", fallbackMethod = "fallbackMatches")
3  @Retry(name = "mlbApi")
4  public List<MatchDTO> getOfficialMatches(LocalDate start, LocalDate end) {
5      String url = "https://statsapi.mlb.com/api/v1/schedule?sportId=1" +
6          "&startDate=" + start + "&endDate=" + end;
7
8      ScheduleResponse response = this.restTemplate.getForObject(url,
9          ScheduleResponse.class);
10     if (response == null || response.dates() == null) {
11         return List.of();
12     }
13     log.info("Matches successfully retrieved!");
14     return response.dates().stream()
15         .flatMap(d -> d.games().stream())
16         .map(this::toMatchDTO)
17         .filter(this::isMlbTeamMatch)
18         .filter(dto ->
19             this.stadiumRepository.findByName(dto.stadiumName()).isPresent())
20         .map(this::saveMatch)
21         .toList();
22 }
```

Código H.1: Implementación de *getOfficialMatches*

```

1  @Transactional
2  @CircuitBreaker(name = "verifyMatchStatus", fallbackMethod =
    "fallbackVerifyStatus")
3  @Retry(name = "verifyMatchStatus")
4  @CacheEvict(value = {
5      "get-standings",
6      "historic-ranking",
7      "win-distribution",
8      "wins-per-rivals",
9      "get-home-matches",
10     "get-away-matches"
11 }, allEntries = true)
12 public void verifyMatchStatus() {
13     try {
14         LocalDate today = LocalDate.now(this.clock);
15         LocalDate yesterday = today.minusDays(1);
16         LocalDate endDate = today.plusDays(1);
17         String url =
18             "https://statsapi.mlb.com/api/v1/schedule?sportId=1&startDate=" +
19             yesterday + "&endDate=" + endDate;
20         ScheduleResponse response = this.restTemplate.getForObject(url,
21             ScheduleResponse.class);
22
23         if (response == null || response.dates() == null) {
24             log.info("No matches found for {}", today);
25             return;
26         }
27         for (DateEntry dateEntry : response.dates()) {
28             for (GameEntry gameEntry : dateEntry.games()) {
29                 MatchDTO apiMatch = this.toMatchDTO(gameEntry);
30                 this.matchRepository.findByStatsApiId(apiMatch.id()).ifPresent(match
31                     -> {
32                         MatchStatus oldStatus = match.getStatus();
33                         MatchStatus newStatus = apiMatch.status();
34                         match.setHomeScore(apiMatch.homeScore());
35                         match.setAwayScore(apiMatch.awayScore());
36
37                         if (!oldStatus.equals(newStatus)) {
38                             match.setStatus(newStatus);
39                             this.matchRepository.save(match);
40                             if (!oldStatus.equals(MatchStatus.FINISHED) &&
41                                 newStatus.equals(MatchStatus.FINISHED)) {
42                                 this.updateStatus(match);
43                             }
44                         }
45                     });
46             }
47         }
48         log.info("Successfully updated the match for the {} games", today);
49     }
50     catch (Exception ex) {
51         log.error("An error occur updating the status of the matches: {}",
52             ex.getMessage());
53     }
54 }

```

Código H.2: Implementación de *verifyMatchStatus*

```
1  @Async
2  @Schedules({
3      @Scheduled(cron = "0 */20 19-23 * * *", zone = "Europe/Madrid"),
4      @Scheduled(cron = "0 */20 0-8 * * *", zone = "Europe/Madrid")
5  })
6  public void updateStandings() {
7      log.info("Starting synchronization of the matches status");
8      try {
9          this.matchImportService.verifyMatchStatus();
10     }
11     catch (Exception ex) {
12         log.error("An error occur while synchronizing the matches status: {}",
13             ex.getMessage());
14     }
```

Código H.3: Implementación de *MatchScheduler*



Implementación *TeamImportService*

En este anexo, se mostrará el código del principal del *TeamImportService* (Código [I.1](#)) junto a su correspondiente *Scheduler* *TeamScheduler* (Código [I.2](#)).

```

1  @Transactional
2  @CircuitBreaker(name = "getTeamStats", fallbackMethod = "fallbackTeamStats")
3  @Retry(name = "getTeamStats")
4  @CacheEvict(value = {
5      "get-teams",
6      "get-standings",
7      "runs-per-rival",
8      "wins-per-rivals",
9      "win-distribution",
10     "historic-ranking",
11     "search-team"
12 }, allEntries = true)
13 public void getTeamStats() {
14     int currentYear = LocalDate.now().getYear();
15     LocalDate today = LocalDate.now();
16     String url =
17         "https://statsapi.mlb.com/api/v1/standings?leagueId=103,104&season="
18         + currentYear + "&standingsTypes=regularSeason";
19     try {
20         StandingsResponse response =
21             this.restTemplate.getForObject(url, StandingsResponse.class);
22         if (response == null || response.records() == null ||
23             response.records().isEmpty()) {
24             log.warn("No stats found for the {} season. Applying empty values",
25                 currentYear);
26             this.setEmptyStats();
27             return;
28         }
29         List<Team> teamsToSave = new ArrayList<>();
30         List<DailyStandings> dailyHistory = new ArrayList<>();
31         for (Records records : response.records()) {
32             if (records.teamRecords() != null) {
33                 for (TeamRecords teamRecord : records.teamRecords()) {
34                     Team team = this.updateTeamStats(teamRecord);
35                     teamsToSave.add(team);
36                     if (!this.dailyStandingsRepository.existsByeamAndMatchDate(team,
37                         today)) {
38                         dailyHistory.add(new DailyStandings(
39                             team,
40                             today, this.parseRankToInt(teamRecord.divisionRank())
41                         ));
42                     }
43                 }
44             }
45         }
46         this.teamRepository.saveAll(teamsToSave);
47         this.dailyStandingsRepository.saveAll(dailyHistory);
48         log.info("Rankings and Team Stats successfully updated!");
49     }
50     catch (Exception e) {
51         log.error("Error obtaining the stats from the API:{}", e.getMessage());
52         this.setEmptyStats();
53     }
54 }

```

Código I.1: Implementación de *getTeamStats*

```
1  @Async
2  @Scheduled(cron = "0 0 5 * * *", zone = "Europe/Madrid")
3  public void updateTeamStatsAndRankings() {
4      this.teamImportService.getTeamStats();
5      log.info("Team Stats Updated. Reset Team Cache");
6  }
```

Código I.2: Implementación de *TeamScheduler*

J

Implementación *PlayerImportService*

En este anexo, se mostrará el código del principal del *PlayerImportService* (Códigos J.1, J.2 y J.3) junto a su correspondiente *Scheduler PlayerScheduler* (Código J.4).

```
1  @Transactional
2  @CircuitBreaker(name = "playersFromStatsAPI", fallbackMethod =
   "fallbackGetTeamsRoster")
3  @Retry(name = "playersFromStatsAPI")
4  @CacheEvict(value = {
5      "get-players",
6      "all-stats-player-rankings",
7      "single-stat-player-rankings",
8      "search-player"
9  }, allEntries = true)
10 public void getTeamRoster() {
11     List<Team> teams = this.teamRepository.findAll();
12
13     for (Team team : teams) {
14         String url = "https://statsapi.mlb.com/api/v1/teams/" +
15             team.getStatsApiId() + "/roster";
16         RosterResponse response = this.restTemplate.getForObject(url,
17             RosterResponse.class);
18         if (response != null && response.roster() != null) {
19             for (RosterEntry entry : response.roster()) {
20                 try {
21                     this.getPlayerStatsFromAPI(entry.person().id(), team);
22                 } catch (Exception ex) {
23                     log.warn("Error fetching the data of player id: {} with the
24                         team : {}: {}", entry.person().id(), team.getName(),
25                         ex.getMessage());
26                 }
27             }
28         }
29     }
30     log.info("Rosters and Stats successfully retrieved");
31 }
```

Código J.1: Implementación de *getTeamRoster*

```

1  @CircuitBreaker(name = "playerStatsFromAPI", fallbackMethod =
    "fallbackGetPlayerStats")
2  @Retry(name = "playerStatsFromAPI")
3  private void getPlayerStatsFromAPI(int personId, Team team) {
4      String url = "https://statsapi.mlb.com/api/v1/people/" + personId +
        "?hydrate=stats(group=[hitting,pitching],type=[season])";
5      PlayerResponse response = this.restTemplate.getForObject(url,
        PlayerResponse.class);
6      if (response == null || response.people().isEmpty()) {
7          return;
8      }
9      PlayerDetailInfo player = response.people().getFirst();
10     StatData data = this.getData(player);
11
12     if ("1".equals(player.primaryPosition().code())) {
13         this.savePitcher(player, data, team);
14     }
15     else {
16         this.savePositionPlayer(player, data, team);
17     }
18 }

```

Código J.2: Implementación de *getPlayerStatsFromAPI*

```

1  @CircuitBreaker(name = "playerPictureFromAPI", fallbackMethod =
    "fallbackGetPlayerPicture")
2  @Retry(name = "playerPictureFromAPI")
3  private PictureInfo savePlayerPicture(int id) {
4      String url = "https://img.mlbstatic.com/mlb-photos/image/upload/" +
        "d_people:generic:headshot:67:current.png/v1/people/" + id +
        "/headshot/67/current";
5
6      PictureInfo pictureInfo = new PictureInfo();
7      pictureInfo.setUrl(url);
8      return pictureInfo;
9  }

```

Código J.3: Implementación de *savePlayerPicture*

```

1  @Async
2  @Scheduled(cron = "0 0 6 * * *", zone = "Europe/Madrid")
3  public void updateRostersAndPlayerStats() {
4      this.playerImportService.getTeamRoster();
5      log.info("Player Stats Updated. Reset Player Cache");
6  }

```

Código J.4: Implementación de *PlayerSecheduler*



Configuración de *Grafana* y *Prometheus*

En este anexo se expandirá la subsección de *Grafana* de la sección de **Implementación**, en donde se detallará la configuración necesaria para que el ecosistema de *Grafana* funcione correctamente.

K.1 Configuración de *Prometheus*

Ubicado en la carpeta *prometheus*, contiene el archivo *prometheus.yml*, que le indica a *Prometheus* dónde obtener los datos. En este caso, el proveedor de datos es *Spring Actuator*, y para acceder al *Actuator*, se conecta a la red interna del contenedor de la aplicación. El otro archivo que se encontrará es un *Dockerfile*, que permite a *Railway* crear el contenedor de esta base de datos.

K.2 Configuración de *Grafana*

En la otra parte de la carpeta, se encuentra la carpeta *grafana*, que contiene toda la configuración necesaria para que *Grafana* funcione. Esta carpeta es un poco más compleja que la de *Prometheus*, ya que no solo tiene su propia configuración, sino también la relacionada con la configuración de la fuente de datos. Está dividida de la siguiente manera:

- **Carpeta *Provisioning*:** Contiene la fuente de datos y la configuración de

los paneles.

- **Carpeta *Datasource*:** Contiene un archivo *datasource.yml* que configura *Prometheus* como proveedor de datos, conectando *Grafana* a su red interna.
- **Carpeta *Dashboards*:** Contiene dos archivos: *dashboard-config.yml*, que gestiona la configuración necesaria de los paneles (en este caso, solo uno), y *Diamond-Insights-Analytics.json*, que almacena una copia de seguridad de los datos mostrados en el panel.
- ***Dockerfile*:** Igual que para *Prometheus*, necesaria para que *Railway* pueda crear su contenedor.

K.3 Despliegue

Para que los administradores puedan acceder a este servicio en cualquier momento, también se desplegó en los servidores de *Railway*. Para acceder al panel, solo basta con seguir el enlace situado en el pie de página ¹, usando las credenciales que se muestran en la **Tabla K.1**.

Tabla K.1: Credenciales de acceso a *Grafana*

Usuario	admin
Contraseña	admin

La **Figura K.1** ilustra el mapa de despliegue resultante.

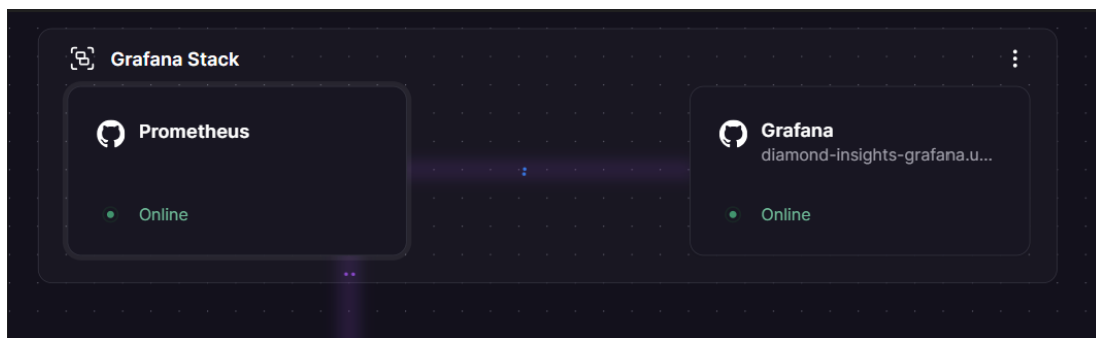


Figura K.1: Mapa de despliegue del ecosistema de *Grafana*

¹Enlace al panel de *Grafana*: <https://diamond-insights-grafana.up.railway.app/>

El panel de métricas de *Diamond Insights* es accesible desde la página de inicio o a través de la sección *Dashboards* del menú lateral izquierdo. Su interfaz presenta un aspecto similar al ilustrado en la **Figura K.2**.

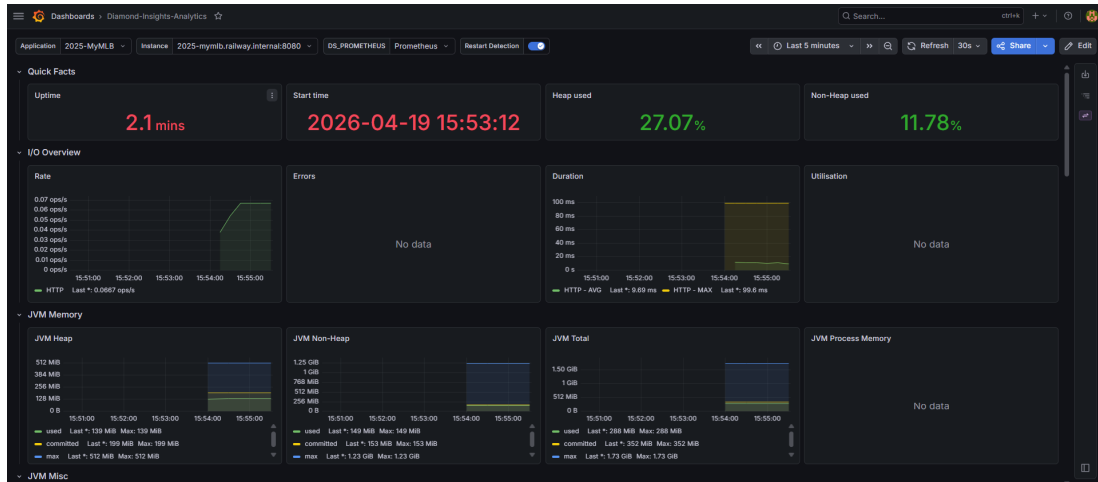


Figura K.2: *Dashboard* de *Grafana* de la aplicación